

Musica con PD

di Antonio Grande

31 agosto 2007

Indice

I	Sintassi e oggetti di controllo	5
1	Installare e configurare PD	6
1.1	PD in questo libro	6
1.2	Installare PD	7
1.2.1	Per i sistemi Windows	7
1.2.2	Per i sistemi Linux	7
1.2.3	Versione <i>estesa</i>	9
1.3	Configurazione	9
1.3.1	File	10
1.3.2	Find	11
1.3.3	Windows	11
1.3.4	Media	11
1.3.5	Help	13
1.4	Verso la prima patch	14
1.4.1	File	14
1.4.2	Edit	15
1.4.3	Put	16
1.4.4	Connessioni locali e non locali: <i>send</i> , <i>receive</i>	17
1.4.5	Modo Edit/Modo Run	18
1.4.6	Tipologie di box	18
1.4.7	Subpatches e Astrazioni	20
2	Primi passi	21
2.1	Symbol	22
2.2	Box symbol (di testo)	22
2.3	Proviamo la nostra patch	22
2.4	Ancora su Symbol	22
2.5	L'oggetto bang	23
2.6	Abbiamo studiato	24
3	Matematica con PD	25
3.1	Il box numerico	25
3.2	Oggetti matematici. I 2 inlet	26
3.2.1	Alcuni oggetti matematici	26
3.2.2	Altri oggetti matematici	27
3.3	Oggetti relazionali	27
3.3.1	Oggetti simili:	28
3.3.2	Altri oggetti relazionali	28

3.4	L'oggetto <i>random</i>	28
3.5	Conversioni	29
3.5.1	Altre conversioni	29
3.6	Ancora sui numeri	30
3.7	L'oggetto <i>clip</i>	30
3.8	Esercitazione	30
3.9	Abbiamo studiato	32
4	Manipolazione del tempo	33
4.1	Delay	33
4.2	Pipe	34
4.3	Metro	35
4.4	Misurare il tempo	36
4.5	L'oggetto <i>line</i>	37
4.6	Esercitazione 1: costruire un contatore	39
4.7	Esercitazione 2: costruire un orologio	40
4.8	Abbiamo studiato	41
5	Controllo sui dati	42
5.1	Filtri, smistamenti ed altro: <i>moses</i> , <i>spigot</i> , <i>change</i>	42
5.1.1	Moses	42
5.1.2	Spigot	42
5.1.3	Change	43
5.2	Ancora su trigger	44
5.3	Route	44
5.3.1	Gli argomenti di route	45
5.4	Controllo da tastiera	46
5.5	Abbiamo studiato	47
6	Le tabelle	48
6.1	Creazione di una tabella	48
6.1.1	L'oggetto <i>table</i>	48
6.2	Lavorare con una tabella	49
6.2.1	Scrivere con <i>tabwrite</i>	49
6.2.2	Leggere con <i>tabread</i>	50
6.2.3	Azzeramento della tabella	51
6.3	Per maggiori precisioni: <i>tabread4</i>	52
6.4	Selezionare fra varie tabelle	52
6.5	Inviare messaggi alle tabelle	52
6.5.1	Carichiamo una tabella	53
6.5.2	Ridimensioniamo la tabella: <i>bounds</i>	53
6.5.3	Inserire una gradazione	53
6.5.4	Numerare una tabella	54
6.6	Abbiamo studiato	54
7	MIDI	56
7.1	Un semplice strumento: uso di <i>makenote</i> e <i>noteout</i>	56
7.1.1	Digressione sulla notifica MIDI	58
7.2	Suonare da tastiera: <i>notein</i> e <i>stripnote</i>	59
7.3	Cambiamo suono: <i>pgmout</i>	60

<i>INDICE</i>	3
7.4 Il <i>Control Change</i>	61
7.5 Piccoli incrementi: <i>bendin</i> e <i>bendout</i>	63
7.6 Altri oggetti MIDI	64
7.7 Abbiamo studiato	65
8 Caricare librerie esterne	66
8.1 Installare <i>maxlib</i>	66
8.2 Alcuni oggetti di maxlib: <i>borax</i> , <i>pitch</i> e <i>chord</i>	67
8.2.1 Borax	67
8.2.2 Pitch	68
8.2.3 Chord	68
8.3 Installare <i>xeq</i>	69
8.4 Esercitazione. Ipotesi di analisi.	70
8.5 Installare <i>iemlib</i>	75
8.5.1 Installazione sotto Linux	75
8.6 Abbiamo studiato	75
II Gestione dell'audio	76
9 PD e l'audio. Per cominciare	77
9.1 Oggetti <i>adc~</i> e <i>dac~</i>	77
9.2 Un semplice oscillatore: <i>osc~</i>	79
9.3 Registrare files audio su Hard Disk	79
9.4 Leggere files audio da Hard Disk	81
9.5 Abbiamo studiato	81
10 Lettura e scrittura da tabelle	82
10.0.1 Caricare un file da Hard Disk: <i>soundfiler</i>	82
10.0.2 Acquisizione da microfono con l'oggetto <i>tabwrite~</i>	84
10.0.3 Caricare un suono in uscita da PD e rileggerlo con <i>tabplay~</i>	85
10.1 Gestire le rampe: <i>line</i> , <i>line~</i> e <i>vline~</i>	87
10.1.1 <i>line~</i>	87
10.1.2 <i>vline~</i>	87
10.1.3 Produrre un suono tramite rampe	88
10.2 Lettura di campioni: <i>tabosc4~</i> e <i>tabread4~</i>	89
10.2.1 Uso di <i>tabosc4~</i>	89
10.2.2 Uso di <i>tabread4~</i> con <i>phasor~</i>	90
10.2.3 Uso di <i>tabread4~</i> con <i>cos~</i>	92
10.2.4 Uso di <i>samphold~</i>	94
10.3 Abbiamo studiato	95
11 Filtri	96
11.1 Un po' di matematica	97
11.2 Poli e zeri	98
11.3 Teoria essenziale sui filtri	99
11.3.1 Passa alto, passa basso	99
11.3.2 Passa banda	100
11.3.3 Filtri passa tutto	101
11.3.4 Filtri a pettine (comb filters)	101

11.3.5	Filtri equalizzatori	102
11.4	I filtri più semplici in PD	103
11.4.0.1	<i>hip~</i> , <i>lop~</i> ed <i>env~</i>	103
11.4.1	<i>bp~</i> e <i>noise~</i>	104
11.4.2	<i>vcf~</i>	107
11.5	Filtri <i>grezzi</i>	108
11.5.1	<i>rzero~</i> , <i>rzero_rev~</i> , <i>rpole~</i>	108
11.5.2	Il filtro <i>biquad~</i>	109
11.6	Esempio di impiego di filtri <i>grezzi</i>	110
11.6.1	Filtro Shelving	110
11.7	Abbiamo studiato	112
12	Ritardi	114
12.1	Gli oggetti <i>delwrite~</i> e <i>delread~</i>	114
12.2	L'oggetto <i>vd~</i>	115
12.3	Ordine di computazione della coppia <i>lettura/scrittura</i>	116
12.3.1	Scrittura <i>prima</i> della lettura	116
12.3.2	Scrittura <i>dopo</i> la lettura	116
12.4	Realizzare un filtro <i>comb</i> con una linea di ritardo	117
12.4.1	Appendice matematica	117
12.4.2	Trasporre di un'ottava con un ritardo dinamico	119
12.5	Linee di ritardo come algoritmi di sintesi	123
12.5.1	Una patch di sintesi e uso di <i>block~</i>	123
12.6	Abbiamo studiato	126
13	Strumenti di analisi	127
13.1	Un po' di teoria	128
13.1.1	Un esempio per capire meglio	129
13.1.2	Conflitto tra risoluzione di frequenza e di tempo	131
13.1.3	Overlapping	131
13.2	Tipologie d'uso di <i>fft~</i> ed <i>rfft~</i>	132
13.2.1	<i>fft~</i>	132
13.2.2	Breve approfondimento: valori Reali e Immaginari	134
13.2.3	Finestra di Hann ed <i>rfft~</i>	135
13.2.4	Ricostruzione del segnale audio. Uso di <i>irfft~</i>	137
13.3	Un rilevatore di frequenza e di ampiezza: <i>fiddle~</i>	139
13.4	Esercitazioni con <i>fiddle~</i>	141
13.4.1	Tracciatore d'altezze e di picchi	142
13.4.2	Analisi e risintesi	143
13.5	Abbiamo studiato	144

Parte I

Sintassi e oggetti di controllo

Capitolo 1

Installare e configurare PD

1.1 PD in questo libro

PD è un ambiente di programmazione grafico real-time per il processamento dell'audio e della grafica. E' simile al noto e costoso programma commerciale Max/MSP, ma è più semplice e certamente più portabile. Rispetto al suo cugino commerciale è più spartano, soprattutto nella grafica, ma la sua potenza è fuori discussione. Inoltre, essendo un programma *open source*, PD può vantare di essere in continua evoluzione, con un numero imprecisato di sviluppatori sparsi in tutto il mondo.

Lo scopo di questo libro è la divulgazione del programma attraverso la creazione di una documentazione a carattere didattico in lingua italiana che potrà, nel tempo, diventare sempre più completa e approfondita. Per il momento il nostro obiettivo è stato quello di puntare ad una conoscenza base di PD per la gestione dell'audio digitale. Ciò attraverso un duplice percorso. In una prima parte, fino al Capitolo 8 compreso, viene spiegato il funzionamento di PD, la sua sintassi e tutta la gestione dei controlli, senza tuttavia affrontare le componenti audio. L'argomento della seconda parte, dal Capitolo 9 alla fine, è invece la gestione dell'audio: oltre agli strumenti più essenziali per la riproduzione o registrazione, vengono trattati i filtri, le linee di ritardo ed infine i processi di analisi in tempo reale del suono. Seppure in modo semplice, e per quanto possibile divulgativo, l'intento alla base di questa seconda parte è anche quello di fornire una piattaforma per lo studio di alcuni grandi temi del trattamento digitale del suono. In tale prospettiva, nelle intenzioni dell'autore, PD diventa un'ottimo strumento di laboratorio, versatile, avvincente e di grande utilità didattica.

Ne consegue che molti aspetti e possibilità di questo software sono rimasti fuori. Ad esempio non viene affrontata la tematica delle strutture di dati, nè quella legata alla gestione delle animazioni grafiche - la libreria GEM¹ - su cui in rete vi è una vasta letteratura. Ma, se il progetto doveva essere quello di costruire un insieme di competenze nel campo della musica digitale, non si tratta di una mancanza. Lo studente che avrà seguito il percorso proposto da questo libro avrà gli strumenti per poter affrontare in seguito, e da solo, i settori di impiego qui non trattati.

¹GEM, che sta per *Graphics Environment for Multimedia*, è un ambiente di programmazione visuale per la computer graphic in real-time scritto inizialmente da Mark Danks. In pratica è un insieme di *externals* da abbinare a PD.

1.2 Installare PD

Date le sue caratteristiche, PD può esser liberamente scaricato dalla rete. Bisognerà fare riferimento al proprio sistema operativo perchè il programma gira sotto vari sistemi come Linux, Microsoft Windows, Mac OS e Irix. Faremo riferimento in questo manuale alla installazione per Windows e Linux.

Per il download possiamo collegarci al sito ufficiale, che è poi quello del suo autore:

<http://www.crcs.ucsd.edu/~msp/software.html>

Qui troveremo anche la documentazione in inglese visionabile in formato HTML. Va detto però che una volta scaricato e decompresso il file, la documentazione in lingua inglese sarà presente in una cartella del nostro computer.

1.2.1 Per i sistemi Windows

La procedura più semplice è scaricare il file di installazione autoestraente, riconoscibile peraltro per l'estensione *.exe*. In alternativa c'è il classico file in formato compresso *.zip* da scompattare.

In entrambi i casi, seguendo la procedura di installazione, il sistema chiederà in quale cartella installare il programma. Ad esempio potrebbe essere: C:/pd.

Va subito detto che Pd non è uno dei soliti programmi cui è abituato l'utente windows, con tanto di icona posta sul desktop per facilitarne il lancio. Il programma di installazione si limita a scompattare i vari files e creare una rete di sottocartelle. E' consigliabile, dunque, andare subito a vedere cosa contiene la nostra nuova cartella *../pd*. Vediamo che vi si trovano 4 sottocartelle: *bin*, *doc*, *extra* e *lib*. Il programma eseguibile per lanciare PD si trova nella cartella *bin*. Apriamola. A questo punto non resta che fare il doppio click sul file *pd.exe* ed è fatta.

E' consigliabile, ovviamente, facilitarci la vita e associare il lancio del programma ad un'icona di nostra scelta da porre sul desktop. In tal caso seguiamo la tipica procedura di Windows: cliccare su uno spazio non attivo del desktop con il pulsante destro, quindi su *Nuovo*, e poi su *Collegamento*. A questo punto si aprirà una finestra in cui si richiede di immettere il percorso del file cui ci si vuole collegare. Noi mettiamo: *c:/pd/bin/pd.exe* (sempre nell'ipotesi che abbiamo installato PD come detto sopra). Quindi cliccando su *Avanti* ci verrà chiesto di assegnare un nome al collegamento: può andar bene scrivere PD.

Se vogliamo fare di meglio potremmo anche scegliere anche un'icona grafica più attraente seguendo la seguente procedura. Clicchiamo con il pulsante destro del mouse sull'icona provvisoria appena creata sul desktop, quindi scegliere *Proprietà* dal menù a comparsa. Nella finestra che si apre clicchiamo sul bottone *Cambia icona ...*, quindi scegliamo fra le tante icone presenti. Effettuata la scelta basta agire sul pulsante *Applica* (in basso alla finestra) e il lavoro è fatto.

1.2.2 Per i sistemi Linux

La procedura di installazione sotto Linux può dipendere fortemente dal tipo di distribuzione in uso. Oggi per fortuna anche Linux dispone di procedure più o meno automatiche di installazione a cui ovviamente è il caso di fare riferimento. Ad esempio se la distribuzione lo permette - come in Debian, ma anche in Fedora - è tutto molto semplice potendo usare il gestore APT (*Advanced Packaging Tool*). Si tratta di una procedura che va installata e che in questa sede non discuteremo.

Procedure amichevoli: *apt-get* e *yum* Il concetto alla base alla base di APT è che un qualunque sistema locale può connettersi con una o più repository remote dove poter andare a scaricare il programma interessato con tutte le dipendenze di cui necessita. Lo stato sia dei

programmi installati che delle dipendenze va tenuto costantemente aggiornato in modo che ogni volta che il sistema deve installare un nuovo programma può muoversi nella maniera più adeguata.

Se tutto è a posto, l'operazione da fare è semplicissima. Basta aprire un terminale e digitare:
`apt-get install puredata`

Per completezza, va detto che anche la procedura *apt-get* è oggi resa ulteriormente intuitiva tramite il supporto grafico offerto da *Synaptic*.

E' importante aver configurato il sistema per connettersi alle repository di un centro di ricerca che porta avanti il progetto Pd. Nel nostro caso si consiglia di fare riferimento al ben noto *Planet CCRMA* (da pronunciare 'carma'), dell'Università di Stanford, USA. Vedremo dunque in sequenza, una volta interrogato il database, non una sola versione di PD, quella standard, ma molte versioni corrispondenti alle numerose librerie scritte da altri sviluppatori che vengono ad integrare il pacchetto base. Comunque sia, anche installate tutte le librerie, PD partirà con l'opzione base a meno di non impostare uno script di partenza che indichi a PD di caricare le librerie esterne. Di questo però parleremo più avanti.

Un altro semplice procedimento di installazione richiede l'uso di *yum*, che si comporta come l'*apt-get*, sempre che naturalmente si sia caricato l'indirizzo del repository di nostro interesse (nel nostro caso di *Planet CCRMA*). A tal fine si può procedere in questo modo, supponendo di avere installata una distribuzione Fedora 7²: digitare da terminale (come root): `rpm -Uv` seguito da

`http://ccrma.stanford.edu/planetccrma/mirror/fedora/linux/planetccrma/7/i386/planetccrma-repo-1.0-3.fc7.ccrma.noarch.rpm`

a questo punto digitare:

`yum install planetccrma-core`

e

`yum install planetccrma-apps`

Ora, per installare PD, procedere con

`yum install pd.`

Ad installazione avvenuta, per lanciare il programma è sufficiente digitare *pd* da una finestra di terminale. Ovviamente anche sotto Linux si può procedere ad impostare un collegamento ad un'icona posta sul desktop, con una procedura che è molto simile a quella descritta per il Sistema Operativo Windows.

Tuttavia, talvolta, è necessario specificare con maggiore dettaglio l'architettura audio usata che oggi, almeno dal kernel n. 2.6, è generalmente ALSA (*Advanced Linux Sound Architecture*). Pertanto lo script di lancio sarà più precisamente: *pd-alsa*. Se inoltre abbiamo installato, come è auspicabile, il server Jack occorrerà per prima cosa lanciarlo (all'uopo può essere utile usare il front-end grafico *qjackctl*) quindi digitare sul terminale *pd -jack*. In tal caso, aprendo la grafica della rete di dispositivi audio in *qjackctl*, vedremo che è comparsa anche PD, sia in entrata che in uscita verso la scheda audio. Consigliamo peraltro di lanciare PD con il server Jack per avere un corretto e immediato funzionamento del sistema MIDI.

Procedura classica Nel caso, invece, di installazione più tradizionale la procedura è un po' più complessa. Scarichiamo il file con estensione *.tar.gz* che include tutto il pacchetto di files relativi a PD in formato compresso. Quindi, aperta una shell, digitare:

`tar xvf [nome-file].src.tar.gz`

Ciò crea una cartella di nome [nome-del-file], ad esempio: *pd-0.39.3* con dentro i files di installazione. Cercare il file *config* e lanciarlo, digitando dalla shell:

`./config`

²Nel caso della release 6, o inferiore, sostituire opportunamente nella path il numero */7/* con */6/* o altro.

e, subito dopo, i classici comandi:

```
make e  
make install.
```

Fare attenzione che PD usa le librerie *tk* e *tcl*, quindi se non presenti nel sistema, andranno installate. Si può effettuare il download all'indirizzo:

<http://tcl.sourceforge.net/>.

Si raccomanda l'attenta lettura delle informazioni fornite a conclusione del lancio del primo comando, *./config* dove compariranno eventuali richieste di dipendenze non soddisfatte.

1.2.3 Versione *estesa*

Per avere una versione ricca di tutte le librerie esterne (almeno quelle più conosciute) con relativa documentazione, c'è oggi anche un pacchetto comprensivo detto *versione estesa* di PD (pd-extended). Può essere utile se si vuole lavorare appoggiandosi ai moltissimi oggetti scritti da vari programmatori in tutto il mondo e finalizzati alle più svariate operazioni, sia audio che grafiche. In questo libro tuttavia, per ragioni didattiche, faremo riferimento quasi esclusivamente alla dotazione standard di PD quindi non ci soffermeremo a descrivere in dettaglio le librerie che vengono agganciate con l'installazione della versione estesa. Si tenga conto, però, che la libreria di cui si parlerà nel cap. 8 relativo al MIDI (di nome *xeq*) non è fra quelle comprese in tale versione. Per cui per usarla non c'è altro modo che scaricarla separatamente e installarla. Si rimanda all'uopo alle pagine relative.

1.3 Configurazione

Sotto Linux, una volta lanciato PD, compare la finestra indicata in fig. 1.1 con 5 menù a discesa e, separatamente, un *terminale*, ossia la finestra principale di PD. Da Windows, invece, si aprirà solo la finestra principale. In entrambi i casi troveremo i cinque menù che ora andiamo ad esaminare singolarmente.

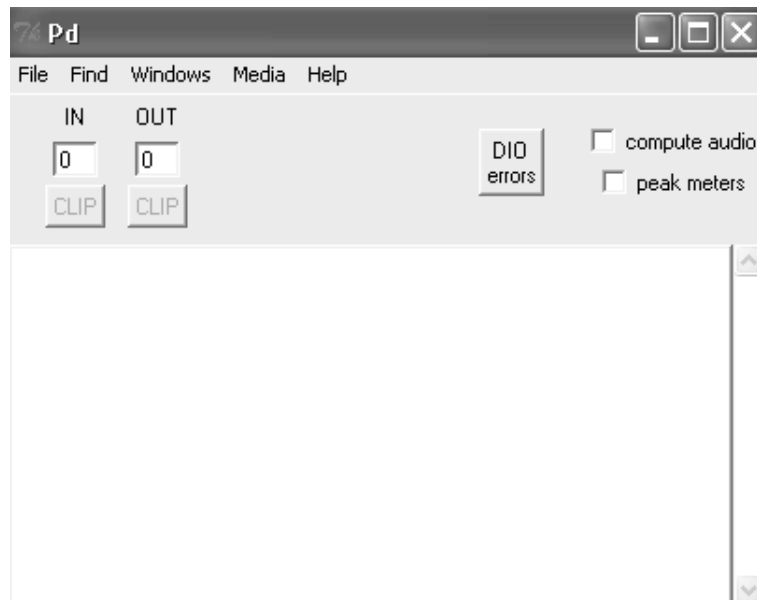


Figura 1.1:

1.3.1 File

Comprende: **New**, **Open**, **Message**, **Path...**, **Startup**, **Quit**

In particolare:

New Serve per creare una nuova patch PD. Si aprirà una piccola finestra su cui è possibile andare a disporre box ed oggetti di cui abbiamo bisogno con le relative connessioni.

Open Fa aprire una finestra per la scelta di un file PD presente in Hard Disk o altro supporto. Agire secondo le normali procedure del vostro Sistema Operativo.

Message Scegliendo questa opzione si apre una finestra molto sottile con l'etichetta **send-panel**. In pratica si può digitare un comando che verrà inviato da PD.

Path... Qui è possibile dichiarare il percorso completo in cui PD andrà a cercare i files che intendiamo caricare. Può essere dunque una utile scorciatoia per evitare di fare numerosi click quando si lavora all'interno di una certa cartella.

Startup Per *Startup* si intende lo script iniziale con cui si intende lanciare PD. Ciò si rivela utile quando vogliamo caricare, oltre al pacchetto base, anche una o più librerie esterne. Salvando lo script corretto in questo spazio non dovremo digitarlo tutte le volte che vogliamo lanciare il programma. In ogni caso, anche se apriamo PD da shell (o tramite un'icona, secondo quanto descritto prima) lo script di lancio verrà copiato in questo spazio.

Inserire dunque una (eventuale) libreria addizionale esterna per ogni linea. In questo spazio è inoltre possibile inserire comandi che riguardano il lancio di PD in modo non realtime (circostanza che potrebbe talvolta essere utile) oppure con particolari flags.

Ecco l'insieme dei flags:

A) Flags di configurazione audio: -r – specifica la frequenza di campionamento (*r* sta per *rate*, velocità)

-audioindev ... – dispositivi di *audio in*. Per es. "1, 3" sta per primo e terzo

-audiooutdev ... – dispositivi di *audio out* (come sopra)

-audiodev ... – specifica sia l'input che l'output

-inchannels ... – canali di input audio

-outchannels ... – numero di canali audio di uscita

-channels ... – specifica sia i canali di input che di output

-audiobuf – specifica l'ampiezza del buffer audio in msec

-blocksize – specifica l'ampiezza del blocco I/O dell'audio in campioni

-nodac – sopprime l'output audio

-noadc – sopprime l'input audio

-noaudio – sopprime sia l'input che l'output audio (ossia: nessun suono)

-listdev – chiede una lista di dispositivi sia Audio che MIDI

-pa – chiede l'uso di Portaudio API (l'audio API di default per questa piattaforma: portaudio)

B) Flags di configurazione MIDI: -midiindev ... – chiede una lista di dispositivi di MIDI In. Per es.: "1, 4" per il primo e il quarto

- midioutdev ... – chiede una lista di dispositivi di MIDI Out. Vedi sopra
- mididev ... – specifica -midioutdev e -midiindev insieme
- nomidiin – sopprime il MIDI input
- nomidiout – sopprime il MIDI output
- nomidi – sopprime sia il MIDI input che l'output

C) Altri flags: -path – aggiunge un percorso di ricerca per i files

- helppath – aggiunge un percorso di ricerca per il file help
- open – apre file(s) alla partenza
- lib – carica libreria(e) di oggetti
- font – specifica l'ampiezza di default dei fonts in punti
- verbose – commenti extra al lancio o quando si cercano files
- version – ci informa della versione di Pd, senza lanciare il programma
- d – specifica il livello di errore
- noloadbang – sopprime tutti i loadbangs
- nogui – non fa partire il supporto GUI
- guicmd "cmd..." – sostituisce un altro programma GUI
- send "msg..." – invia un messaggio al lancio (dopo che sono caricate le patches)
- rt or -realtime – usa la priorità real-time
- nrt – non usa la priorità real-time

Quit Serve per uscire da PD.

1.3.2 Find

Find contiene solo l'opzione *Find last error*, un archivio dell'ultimo errore in cui PD è incorso. Cliccandovi viene riportato sul terminale l'indicazione dell'errore in oggetto.

1.3.3 Windows

Al momento questa opzione non contiene nulla perchè non abbiamo ancora aperto una finestra intesa come vera e propria area di lavoro. PD distingue infatti tra varie gerarchie di finestre. Quella principale del programma, che è una sorta di terminale, dove PD dialoga con l'utente e quella propria della patch (*parent window*), secondo una struttura di filiazione. Infatti da questa finestra possiamo aprirne un'altra, una figlia, che è collegata alla prima e ad essa dipendente. Vedremo più avanti il funzionamento di questa gerarchia.

1.3.4 Media

E' una delle opzioni più importanti di PD perchè serve a configurare la parte Audio e MIDI. In particolare contiene i seguenti sottomenù: **audio ON**, **audio OFF**, **standard (MMIO)**, **ASIO (via portaudio)**, **Audio settings**, **MIDI settings**, **Test Audio and MIDI**, **Load Meter**.

Audio ON/Audio OFF Apre o chiude, rispettivamente, il flusso verso la scheda audio. E' da notare che, per maggior praticità, quasi sempre si darà il comando usando un messaggio all'interno di una patch. Tuttavia può risultare comodo in molti casi effettuare lo switch da menù esterno. Quando l'Audio è acceso PD computa campioni in real time con riferimento alla patch aperta, indipendentemente se visibile o meno. Ciò è da tener presente per avere sempre coscienza dell'onere di calcolo cui è sottoposta la CPU.

Standard (MMIO) In informatica la sigla MMIO sta per *Memory Mapped I/O* ossia una procedura di controllo tramite la quale la CPU comunica con le periferiche esterne, nel nostro caso (la o) le schede audio. Più in particolare riguarda una mappatura di memoria con la quale si indirizzano dei dispositivi hardware. Attraverso essa la CPU legge o scrive per mandare comandi o ricevere informazioni.

PD apre una finestra (fig. 1.2) contenente informazioni su Frequenza di campionamento (*Sample Rate*) e latenza (*delay*). Questi dati possono essere lasciati come sono in quanto contengono valori genericamente adeguati.



Figura 1.2:

Ricordiamo che per latenza si intende il tempo richiesto perchè venga ottenuto un certo risultato, a seguito di una certa operazione. In ambito audio, ad esempio, si intende il lasso di tempo che intercorre tra quando il suono viene creato (via software, tramite una nostra azione) e quando può essere ascoltato. Di norma, per prevenire errori o anomalie nell'audio, si usano dei buffers che mantengono la regolarità di flusso laddove la CPU è impegnata in altre operazioni. Naturalmente vi deve essere un giusto rapporto perchè un buffer troppo ampio può creare malfunzionamenti, ma ciò vale anche se esso è troppo piccolo. Diciamo che 70 msec. di latenza, come indicato in figura, è un valore molto alto. Su un buon sistema e in funzione, ovviamente, del sistema operativo, si può arrivare a meno di 5 msec. di latenza.

In ambiente Windows è importante usare i drivers ASIO; sotto Linux si usano gli ormai abituali drivers ALSA

Più in basso sono indicati i dispositivi audio attualmente agganciati da PD, sia in ingresso che in uscita. Se ve ne sono più di uno si può cliccare sul bottone relativo e selezionare quello che ci interessa.

ASIO (via portaudio) ASIO e Portaudio sono entrambe librerie per la gestione I/O dell'Audio. **Portaudio** è una libreria *open-source*, dunque libera pensata per essere il più possibile indipendente dalla piattaforma. Viceversa **ASIO** (Audio Streaming Input Output) è una tecnologia proprietaria sviluppata dalla softerhouse tedesca Steinbeg, quella - per intenderci - che ha prodotto il celebre Cubase. ASIO controlla lo *streaming audio* e, più in generale, la gestione della scheda, il tutto con una latenza molto bassa.

Questa opzione apre una finestra sostanzialmente analoga alla precedente, con le medesime opzioni. Non figura tuttavia il pulsante “use multiple devices”.

Audio settings... Anche questa opzione ci apre una finestra simile alle precedenti (in particolare simile allo Standard MMIO). Si rimanda dunque a quelle.

MIDI settings... Questo menù ci permette di configurare la parte MIDI del nostro sistema. Come per l’Audio possiamo selezionare, anche separatamente, i dispositivi di Ingresso e di Uscita a seconda delle schede di cui disponiamo.



Figura 1.3:

Nel nostro esempio (fig. 1.3) abbiamo collegato un dispositivo (device) di ingresso MIDI, ossia una tastiera, di tipo USB. Per l’uscita invece utilizziamo il banco suoni MIDI interno della scheda audio del computer.

In ambiente Linux, per evitare problemi con il MIDI, consigliamo di utilizzare un sintetizzatore virtuale in coppia con il server *jack*. Più in dettaglio procedere così:

- attivare *jack*, usando anche il front-end grafico *qjackctl*
- lanciare PD con lo script *pd -jack* in modo da caricare l’ambiente audio *jack*
- attivare un synth virtuale, come potrebbe essere *qsynth* (eventualmente caricare i font audio)
- dopo aver verificato la presenza di PD nell’ambiente grafico delle connessioni di *jack* (sempre supponendo *qjackctl*) provvedere ad agganciare, nella parte MIDI, il synth con PD.

Fatto. Non resta che verificare che tutto funzioni con il Test di cui andiamo a parlare.

Test Audio and MIDI Questa è forse la sezione più importante per la configurazione di PD nel nostro computer: cliccando su questa opzione si aprirà una finestra da cui è possibile effettuare un test sia per la parte Audio che per il MIDI.

Per l’**Audio** è sufficiente cliccare sulle caselle bianche di *Test Signal*, tarate su -20 e -40 dB, oppure su *noise tone* (anche in questo caso sulla casella bianca). Se tutto è a posto dovremmo sentire l’audio funzionare. Nel caso si disponga di una scheda a più uscite separate possiamo effettuare il test anche dei singoli canali di uscita.

Nella parte bassa della finestra invece si farà un analogo test per il **MIDI**. Cliccare sulla casella indicata con MIDI OUT. Più avanti si chiarirà il significato degli oggetti e altre parti grafiche ‘attive’ di questa patch. Per ora limitiamoci ad effettuare il nostro test.

1.3.5 Help

Infine l’Help che contiene 3 sottomenù: **About PD**, **Html...**, **Browser...**

About PD Ci fornisce alcune indicazioni utili per identificare la versione di PD che stiamo usando, il nome del file che contiene la licenza d'uso del programma, come pure quello contenente la documentazione (in lingua inglese) per ora ufficiale. Di quest'ultimo si fornisce anche l'indirizzo web. Concludono il materiale altri riferimenti relativi a PD.

Html... E' un link all'indice della documentazione presente nelle cartelle create al momento dello scompattamento. E' sostanzialmente il medesimo contenuto che troviamo al seguente indirizzo:

http://www.cerca.ucsd.edu/~msp/Pd_documentation/index.htm

Browser... Non fa che aprire il contenuto della cartella */doc* presente nei files copiati al momento dell'installazione. Qui sono presenti diverse sottocartelle (dovrebbero essere 5) ognuna delle quali contiene vario materiale di documentazione. Si va dal materiale in lingua inglese cui si giunge cliccando su *Html...*, (vedi sopra), agli esempi d'uso in materia di controlli, Audio, strutture di dati, e altro ancora.

1.4 Verso la prima patch

Dopo aver esaminato l'area dei menù, diamo un ulteriore sguardo ai vari componenti della finestra (fig. 1.4) di 'monitoraggio' del programma, compare solo nella versione Linux.

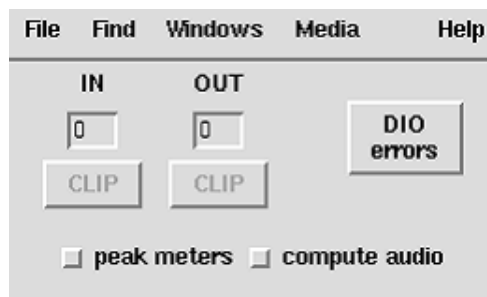


Figura 1.4:

In basso, il bottone a fianco di *compute audio* serve per attivare o disattivare l'Audio.

Il bottone *DIO errors* - che sta per Digital I/O - controlla il flusso audio. Si accende di rosso o lampeggia se qualcosa non va, ad esempio se vi è un errore di sincronizzazione per l'ingresso o l'uscita audio, se vi è un ritardo di computazione, ecc. Se poi si clicca sul bottone appare la lista degli ultimi errori rilevati.

Ma questo non è ancora l'inizio. Per usare PD dobbiamo creare una nuova finestra (*parent window*) su cui costruire ciò che si chiama una *patch*, vero elemento di base. Una patch consiste di una serie di **box** interconnessi: si tratta di elementi che attendono messaggi inviati loro, possono produrre processi e sono capaci di generare messaggi da inviare ad altri oggetti.

Dal menù *File* della finestra principale clicchiamo su *New*. Si aprirà una finestra vuota che è lo spazio dove andremo a collocare gli oggetti. Scorriamo velocemente la barrà dei menù di questa finestra (fig. 1.5).

1.4.1 File

New Per creare una patch nuova

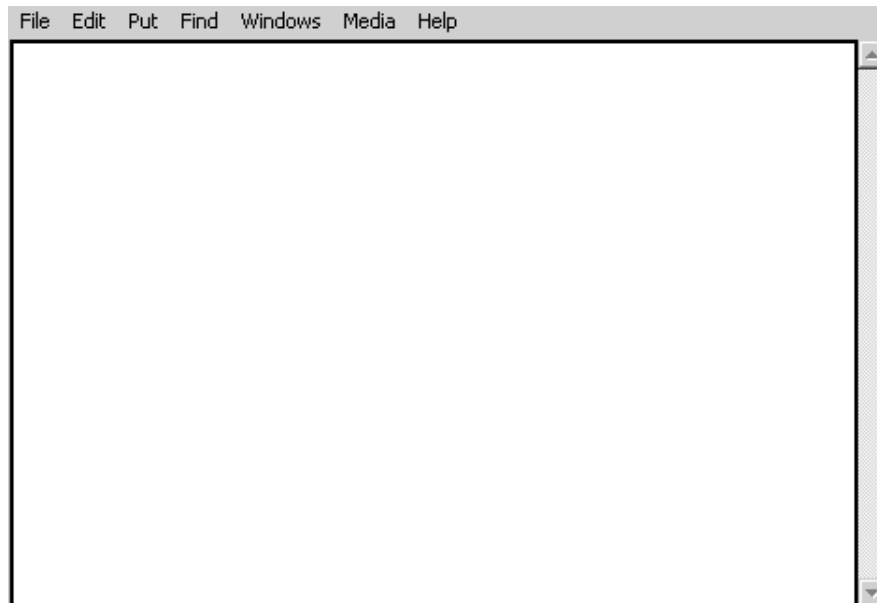


Figura 1.5:

Open Per aprire un file PD memorizzato in Hard Disk o altro supporto

Message, Path e Startup Hanno la stessa funzione di quelli già visti in precedenza

Close, Save, Save as Sono i classici comandi presenti nei vari softwares. Rispettivamente chiude la patch mantenendo però aperta la sessione di PD; salva la patch in un file (se già creato in precedenza); salva la patch in un file assegnando un nome

Print Con questa opzione non si stampa la patch tramite un dispositivo esterno. Piuttosto la stampa avviene su file - di cui si richiede un nome - con estensione *ps* (*Postscript*).

Quit Per uscire dal programma

1.4.2 Edit

Anche questo menù a discesa presenta numerose opzioni assai familiari a chiunque utilizzi il computer; dunque ci occuperemo solo di alcune di esse.

Text editor Cliccando su questa opzione si apre un riquadro che offre un semplice editor di testo. Può essere utile quando si vuole scrivere un box di commento all'interno di una patch. In tal caso dal menù *Put* (vedi più avanti) selezionare *Comment*. Comparirà sul piano di lavoro un riquadro con la parola *comment* evidenziata. A quel punto selezionare *Text Editor*: ciò che scriveremo nell'editor comparirà nello spazio dove abbiamo aperto il commento.

Font Con questa opzione controlliamo la grandezza dei caratteri. Ma può rivelarsi utile anche per rimpicciolire nel suo complesso la patch, nel senso che diventeranno più piccoli o più grandi anche i box ivi presenti.

Find Utile per trovare un certo oggetto disposto sul piano della patch o una qualunque stringa di testo presente entro i vari box. Il box in cui è presente la stringa di ricerca assumerà un colore differente. Analogamente per **Find Again** che cercherà ulteriori occorrenze di quella stringa.

Edit mode Permette il passaggio dal modo Autore (Edit) al modo Utente (vedi più avanti).

1.4.3 Put

E' il menù più importante per il nostro lavoro. Da qui possiamo creare i componenti che, interconnessi tra loro, formeranno la nostra patch. Non spiegheremo in dettaglio i singoli elementi che formano questo menù, rimandando al loro utilizzo pratico nel corso dei prossimi capitoli, ma ne daremo solo una panoramica generale.

Si tratta di effettuare la scelta con il mouse: immediatamente verrà creata un'istanza di quel box sull'area di lavoro, ossia una copia di un prototipo più generale che tuttavia in molti casi dovrà ancora essere definita tramite l'immissione di un simbolo iniziale. Se ad esempio vogliamo istanziare un oggetto 'oscillatore' creiamo sull'area di lavoro l'istanza di un oggetto generico - scegliendo l'opzione **Object** - , poi dobbiamo scrivervi dentro *osc~*. La successiva opzione, **Message**, crea subito un box messaggio, ovviamente vuoto e da riempire. **Number** crea un box numerico, inizializzato a 0, **Symbol** un box in grado di ricevere valori alfabetici e **Comment** introduce uno spazio nell'area di lavoro inizialmente con la scritta *comment*, ma pronta per essere sostituita con quanto scriveremo da tastiera.

Quelli che vengono dopo sono una serie di oggetti di natura grafica (GUI) già pronti per l'uso. Dire che sono pronti per l'uso significa che potremmo crearli ugualmente selezionando genericamente un oggetto e scrivendovi dentro il nome particolare dell'oggetto. Ad esempio se voglio creare un oggetto *bang*, uno degli oggetti fondamentali di tutto il linguaggio di PD, posso scegliere l'opzione **Bang** che trovo nella lista. In tal caso si formerà un'istanza di questo oggetto in formato grafico: una sorta di bottone che si illumina quando lo attiviamo. Ma posso anche creare un oggetto *bang* da **Object** scrivendovi dentro **bng**, per avere il medesimo oggetto grafico tipo bottone, oppure proprio *bang*, e avere un'istanza 'base' dell'oggetto, più spartana, ma funzionante esattamente allo stesso modo.

Toggle è una sorta di interruttore acceso/spento, **Number2** è una versione di tipo grafico, quindi più amichevole e personalizzabile, della classe *number* che abbiamo già visto, ma con le medesime proprietà. Seguono degli *sliders* di tipo verticale (V) oppure orizzontale (H), e analogamente dei set di bottoni, **Vradio** e **Hradio**, assai utili per effettuare varie tipologie di scelte o di selezioni. Il GUI **Canvas**, infine, disegna una cornice grafica sul piano di lavoro: possiamo usarla per colorare particolari aree della patch, per una migliore visibilità anche di tipo funzionale.

Infine **Graph** e **Array** producono degli *array* (matrici) vuoti, ossia contenitori di dati con proprietà grafiche. Ne vedremo l'uso in un prossimo capitolo dedicato.

Il menù **Find** è simile a quello già visto in precedenza, mentre **Window** presenta due opzioni: **PD window** richiama la finestra principale di PD, un terminale (tipico della filosofia UNIX) dove il programma dialoga con l'utente, in cui ad esempio possiamo ricevere informazioni, messaggi di errore o anche dati elaborati da PD. Invece **parent window** è la finestra che abbiamo chiamato area di lavoro.

Gli ultimi due menù, **Media** ed **Help**, sono del tutto identici a quelli incontrati prima per la finestra principale di PD.

1.4.4 Connessioni locali e non locali: *send*, *receive*

Consideriamo un semplice esempio (fig. 1.6) che mostra una tipica catena di box e di connessioni. La patch si spiega da sè.

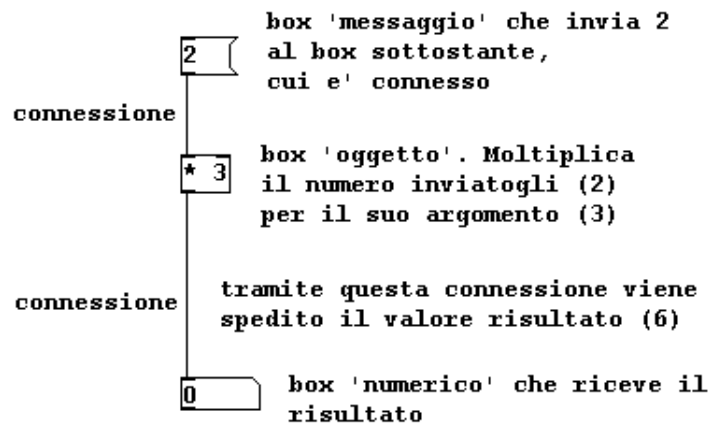


Figura 1.6:

Va aggiunto però che esiste un altro tipo di connessione. Per favorire la leggibilità, quindi per non appesantire con eccessivi tratti di connessione una patch, è possibile interconnettere i box fra loro anche *a distanza*, ossia senza effettuare un collegamento via mouse trascinando l'outlet di un box nell'inlet dell'altro. Per fare questo si usano due oggetti chiamati rispettivamente **send** ed **receive**. Considerando la patch precedente, possiamo riscriverla come in fig. 1.7 usando connessione non locali.

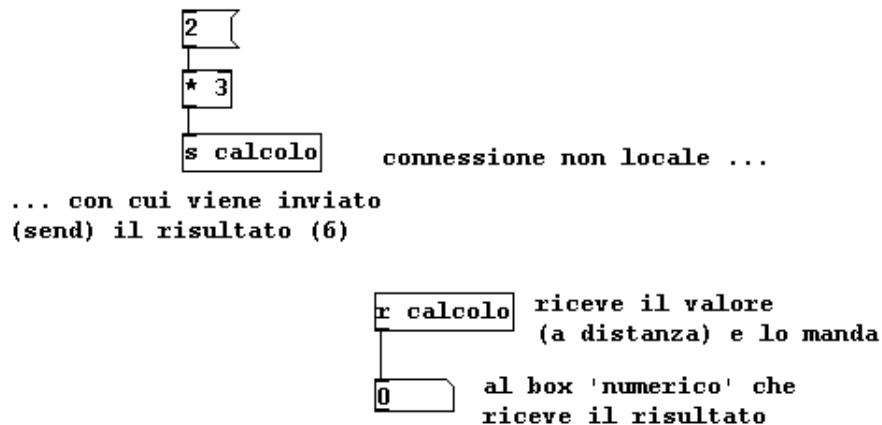


Figura 1.7:

La **s** dell'oggetto più in alto sta per *send*, mentre la **r** per *receive*. Certo, se la patch è molto semplice non si riscontra un apprezzabile miglioramento. Se invece è complessa, se ad esempio

i box da connettere sono lontani tra loro ed una linea di connessione dovrebbe attraversare altri box, allora l'uso di connessioni a distanza diventa molto più pratico.

1.4.5 Modo Edit/Modo Run

Configurandosi nella doppia veste di programma Autore e programma Utente, PD presenta due tipologie d'uso. Dal *Menù* cliccare sull'ultima opzione, *Edit Mode* (oppure *Ctrl+e*). Il puntatore si trasformerà in una piccola mano: in questo stato è possibile spostare i box, cliccarvi dentro e scrivere con la tastiera, configurare tutte le possibili connessioni tra i vari box, ecc. In breve: è possibile costruire tutta la patch. Il *modo Utente* è invece la modalità con cui la patch 'si usa', in una performance ad esempio. Solo in questa modalità vengono generati i messaggi e inviati al loro destinatario. Per tornare al *modo Utente* è sufficiente ripetere il *Ctrl+e* (o da *Edit Mode*): il puntatore tornerà ad essere una freccia.

1.4.6 Tipologie di box

In PD vi sono 4 tipologie di *box*:

- oggetti
- messaggi
- GUI
- commenti

Oggetti Per creare un oggetto cliccare su *Put*, quindi su *Object*. Disposto il box nello spazio della patch è possibile digitare il nome dell'oggetto che vogliamo creare. Ad esempio *int*, che sta per 'intero' (vedremo più avanti il significato più preciso di questi oggetti). L'unità minima di script, in PD si chiama *atomo*. Dentro l'oggetto scrivo più di un atomo, ciascuno separato da spazio, o *a capo*. Il primo atomo dice a PD che oggetto voglio creare. Successivamente inserisco gli *argomenti* che dicono a PD come inizializzare l'oggetto.

Ovviamente, a seconda dell'oggetto che creo, il box si dispone ad avere uno o più *inlet* e uno o più *outlet*. Alcune volte il numero è fisso per ciascun oggetto, altre volte dipenderà dagli argomenti di inizializzazione. Inoltre può variare la qualità: ad esempio gli oggetti *tilde* (vedi più avanti) si predispongono ad avere in uscita connessioni audio, un oggetto matematico, come il citato *int*, invece no. Dunque esistono 2 tipi di connessioni:

- di segnale (audio), riconoscibili graficamente perchè più spesse e
- di controllo, graficamente più sottili.

Vi è incompatibilità fra i due tipi. Non si può mandare una connessione di segnale dove l'oggetto si aspetta una connessione di controllo; e viceversa. Tuttavia esistono alcuni oggetti che possono ricevere, indifferentemente, dati controllo o dati segnale. Talvolta si rende necessario effettuare una conversione tra dati di controllo e dati segnale. All'uopo PD dispone di una serie di oggetti che più avanti esamineremo. Più in generale ogni classe di oggetti dispone di un insieme fisso di messaggi che possono ricevere. Per fare un esempio, gli oggetti di classe *float* rispondono solo a *bang* o a *float*. Se inviamo loro messaggi *simbolo* vi sarà un errore.

Messaggi I box messaggio interpretano il testo come un messaggio da inviare ad un altro box. Nel modo più semplice ciò avviene tramite una opportuna connessione. Il messaggio viene inviato cliccandovi sopra con il mouse.

Nei messaggi vi sono alcune regole sintattiche.

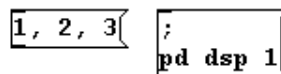


Figura 1.8:

La *virgola* serve a separare 'pezzi' di contenuto del messaggio. Così, nell'esempio a sinistra in figura 1.8, i numeri 1, 2 e 3 verranno inviati in successione come se fossero spediti da altrettanti singoli box messaggio.

Il *punto e virgola* invece significa che l'atomo di testo che lo segue è il destinatario del messaggio. Nell'esempio, il punto e virgola è seguito da *pd*, dunque sta ad intendere che il messaggio è inviato a PD. Nella fattispecie si comanda a PD di accendere l'audio.

Talvolta può tornare comodo usare una variabile dentro un box messaggio da riempire di volta in volta con un valore specifico. PD usa all'uopo il simbolo del dollaro, \$, seguito da un numero (progressivo). Ad esempio:

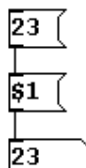


Figura 1.9:

Il numero 23 entra come argomento della variabile \$1 che a sua volta trasmette quel numero al box sottostante. Il simbolo \$ può servire ugualmente bene per numeri o per simboli.

GUI Si tratta dei box numerici, oppure di oggetti grafici come gli *sliders* o altri. La caratteristica di questi oggetti è che sono sensibili al movimento del mouse. In un box numerico, trascinando il mouse su o giù si vedranno scorrere in su o in giù, corrispondentemente, i numeri. Ricorrendo a librerie esterne è possibile trovare ulteriori e più specifici oggetti GUI. Se ne riparlerà più avanti.

Commenti Per rendere più leggibili le patch è buona abitudine corredarle di brevi commenti testuali. In PD esiste un box proprio per i commenti: qui la gestione del testo non è proprio ottimale, anzi è proprio un po' spartana, ma è tuttavia preziosissima.

Alcune osservazioni pratiche: talvolta capita che dopo aver cancellato un commento, alla successiva riapertura della patch si trovi la scritta 'comment' dove prima risultava il commento. Per evitare ciò è sufficiente selezionare con il mouse un'area appena più grande di quella in cui compara questa scritta e quindi usare il backspace per cancellare. In tal modo verrà cancellata tutta l'area.

1.4.7 Subpatches e Astrazioni

Sempre per garantire una migliore leggibilità - ma, come vedremo più avanti, anche per ragioni connesse al controllo del flusso audio e per conseguenti sgravi di calcolo - PD mette a disposizione una procedura che permette di scorporare una sezione di patch assegnandola ad una *subpatch*, che risulta così nascosta, pur continuando a lavorare in background. In altri casi è possibile assegnare un certo compito (in linguaggio informatico: una *routine*) ad una patch esterna a quella principale. Questa patch esterna, detta *astrazione*, avrà un suo nome; quando se ne ha bisogno, basterà creare un oggetto che abbia quel nome per argomento ed essa verrà istanziata nella patch chiamante con tutte le sue funzioni.

Si rimanda ad un successivo capitolo la spiegazione di come creare ed utilizzare subpatches ed astrazioni.

Capitolo 2

Primi passi

Come vuole la tradizione presentiamo al mondo PD con il classico *Hello World*. Per fare questo introduciamo alcuni box, in particolare:

- un box oggetto: Symbol
- un box Messaggio
- un box di testo

Un documento PD si chiama patch ed è composto di un certo numero di box connessi tra loro tramite segmenti che entrano nella parte superiore (Inlet o Ingresso) o escono dalla parte inferiore (Outlet o Uscita).

La patch che ci interessa è semplicissima e appare così (fig. 2.1):

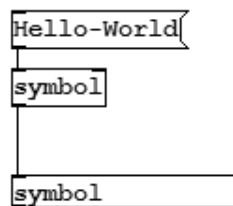


Figura 2.1:

Dopo aver aperto una nuova patch (*File* → *Nuovo*), clicchiamo su *Put*, nella barra in alto: vedremo scendere un menù con 16 nomi. Sono gli elementi che posso creare (da una tipologia di classi) e disporli nell'area della mia patch. Prendo dunque 1) il box Messaggio e lo dispongo in alto. Quindi 2) prelevo un 'oggetto' (object): vicino al puntatore, nell'area di lavoro, compare un rettangolino senza nome. Inseriamo un nome - nel nostro caso **symbol** - e di colpo l'oggetto corrispondente viene creato, con tutte le sue caratteristiche. Infine 3) agganciamo il box **Symbol** e lo disponiamo sotto gli altri due box.

Per prima cosa notiamo la forma dei singoli box.

Quello più in alto è un **box messaggio** (d'ora in poi Messaggio) e si riconosce perchè termina, a destra, con una leggera rientranza. Serve per inviare messaggi. In questo caso invia il messaggio "Hello-World". L'invio avviene cliccandovi sopra con il mouse. Dove va il

messaggio? Come si intuisce facilmente, il messaggio viene indirizzato nell'oggetto sottostante, che qui si chiama *symbol*.

2.1 Symbol

Symbol è un oggetto. Gli oggetti si riconoscono graficamente perché sono dei rettangolini senza ulteriori rifiniture a destra. Esso riceve un contenuto di testo (notare la connessione al suo Input) e lo memorizza. Nel nostro caso oltre a memorizzarlo lo manda in uscita verso un altro box. Vedremo tra breve un uso in cui i due effetti, memorizzazione e invio in uscita, sono separati. L'oggetto *Symbol* è connesso con l'input di un altro box, un box *symbol*.

2.2 Box symbol (di testo)

Il **box symbol** è, nella sostanza, un box di testo e si riconosce perché ha l'angolo superiore destro un po' tagliato. In ciò si presenta graficamente uguale al box numerico, che vedremo fra breve, con l'unica differenza che qui è scritta la parola riservata *symbol*. Tale box è abilitato a ricevere contenuto testuali dall'oggetto **symbol**. Occorre quindi non fare confusione: che differenza c'è tra *symbol* e un *box symbol*? Il primo è un 'oggetto' e compie attivamente un'azione, prima di tutto quella di memorizzare una informazione. Il *box symbol* non è un oggetto e si limita a ricevere informazioni e a mostrarle. Non ha, per così dire, una funzione attiva.

2.3 Proviamo la nostra patch

E' sufficiente cliccare con il mouse sul messaggio e nel box di testo sottostante si ripeterà il testo in esso contenuto.

2.4 Ancora su Symbol

Avrete notato che l'oggetto *symbol* possiede in realtà due porte di ingresso (si riconoscono per un leggero ingrossamento sul lato superiore dell'oggetto). Nella patch precedente abbiamo usato solo l'inlet di sinistra. A cosa serve quella di destra? Ricordiamo che l'azione del messaggio provocava il duplice effetto di 1) memorizzare l'informazione e 2) ordinarne l'uscita. Ecco dunque la differenza: se inviamo l'informazione all'inlet di destra avremo solo il primo effetto, la memorizzazione. Occorrerà una seconda azione per inviare l'ordine di uscita. Facciamo subito un esempio (fig. 2.2).

Se clicchiamo i box sulla destra non vedremo alcun effetto. Ciò che è accaduto è che l'informazione contenuta nel box è stata memorizzata in *symbol*. Per mandarla in uscita occorre cliccare su **bang**, un tipo di messaggio che ha il solo compito di attivare una certa attività (e non di trasferire il testo "bang"). In tal senso, *bang* è una parola riservata in PD. D'altra parte nel nostro caso si tratta solo di mandare in uscita un contenuto di memoria già presente in *symbol*. Noterete che nei box di testo sulla destra, che si connettono all'inlet di destra dell'oggetto sottostante, il testo da inviare è preceduto dalla parola *symbol*. In questo caso *symbol* è una dichiarazione e serve ad informare PD quale tipologia di dati sta per ricevere (dati testo).

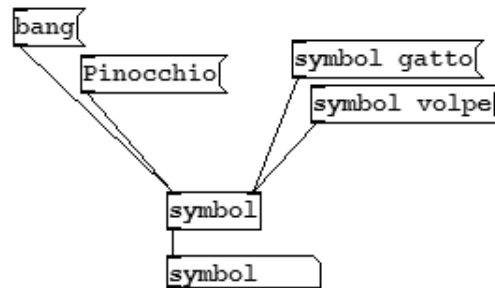


Figura 2.2:

2.5 L'oggetto bang

Abbiamo visto prima l'uso del messaggio contenente, in argomento, la parola *bang*. Poiché si tratta di un messaggio assai tipico e dunque ricorrente, PD dispone di un oggetto che ha proprio il compito di mandare in uscita un bang. Ma ha in più un vantaggio. Si colora, per un momento di nero, per poi ritornare bianco. In altre parole: quando è attivo si fa vedere. Ciò può rivelarsi assai prezioso quando, ad esempio, dobbiamo monitorare un certo processo. In tal caso ci accorgiamo che qualcosa è accaduto grazie al breve cambio di colore dell'oggetto.

Nella semplicissima patch in fig. 2.3 un *bang messaggio* è connesso ad un *bang oggetto*. Cliccando sul primo si attiverà il secondo, colorandosi di nero.

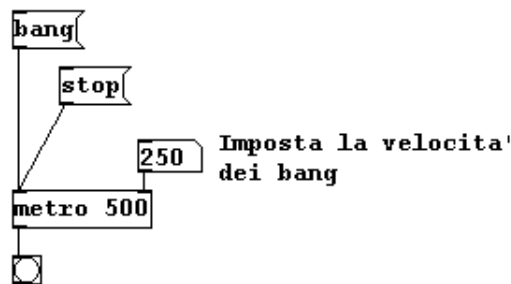


Figura 2.3:

Quindi riassumiamo: l'oggetto **bang** non è solo uno strumento per *inviare* un bang, ma anche per *monitorare* che è stato raggiunto da un bang. In altre parole, la figura sopra poteva anche essere pensata come in fig. 2.4 con l'oggetto impegnato nei 2 ruoli.

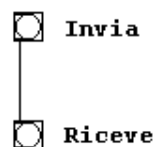


Figura 2.4:

2.6 Abbiamo studiato

symbol, bang, box messaggio, box symbol

Capitolo 3

Matematica con PD

Iniziamo ad usare PD per fare un po' di calcoli e prendere maggiore confidenza con la filosofia del programma.

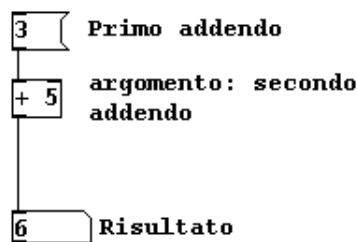


Figura 3.1:

Nella patch in Figura facciamo fare a PD il calcolo: $3 + 5$. Ecco come:

- a) il primo box, quello più in alto, già lo conosciamo: è il *box messaggio* che contiene il numero 3
- b) al centro troviamo un oggetto (lo riconosciamo perché non ha lati tagliati o concavi) che presenta sulla sinistra un “+” e a fianco, sulla destra, un 5. Il “+” specifica la classe di oggetto, in questo caso un oggetto che fa le addizioni. Sulla destra un argomento che dice all’oggetto: somma con 5 il numero che ti viene inviato all’inlet di sinistra
- c) in basso un *box numerico*, ossia un box che contiene solo tipi numerici (in PD di tipo *float*, ossia numeri reali). Si noterà che la forma del box numerico è identica a quello del box di testo.

3.1 Il box numerico

Approfondiamo le caratteristiche del box numerico: poniamolo nella nostra area di lavoro passando quindi alla modalità Utente. Si noterà che, cliccandovi (senza rilasciare il mouse) e quindi dragando, si porta avanti o indietro una numerazione per numeri interi. Se occorre una definizione più selettiva, per numeri decimali, è sufficiente effettuare il dragaggio tenendo abbassato il tasto *Shift* (quello del Maiuscolo). Il box numerico è utile anche per ricevere informazioni misurative. Ad esempio: vogliamo sapere quanti millisecondi sono passati dopo

un certo evento. Un apposito oggetto che calcola il tempo (*timer*, lo vedremo più avanti) verrà direzionato proprio su un box numerico, da cui potremo leggere i risultati. Naturalmente gli usi sono davvero tanti.

Una particolarità del box numerico si rivelerà assai utile: dopo averne creata un'istanza sul nostro ambiente di lavoro, clicchiamovi sopra con il pulsante destro. Si apre la finestra per le impostazioni di Proprietà (*Properties*). Impostiamo ad 1 l'ampiezza del box (*width*). A questo punto il box si comporterà come un commutatore: ad un click del mouse diventa 1, ad un altro click 0. E così via.

3.2 Oggetti matematici. I 2 inlet

Perché l'oggetto matematico “+” ha due inlet? Nell'esempio precedente l'oggetto possedeva un argomento, ossia aveva incorporato il numero con il quale si doveva sommare l'ingresso sinistro. Se però vogliamo cambiare il valore di tale argomento ci è utile l'inlet destro. Consideriamo l'esempio seguente:

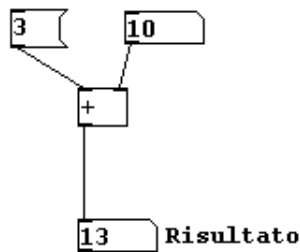


Figura 3.2:

Nell'oggetto “+” è scomparso l'argomento in quanto ora lo diamo con il box numerico connesso all'input destro. Se su quest'ultimo scorriamo i numeri non compare alcun risultato in basso. Se invece clicchiamo sul box messaggio connesso a sinistra compare in basso il risultato. Dunque: i valori inviati a sinistra aggiornano anche il risultato, quelli inviati a destra no, servono solo ad aggiornare la memoria dell'oggetto. Per avere l'uscita si può allora procedere in 2 modi:

1. clicchiamo sul box messaggio di sinistra
2. inviamo un bang all'oggetto “+” (fig. 3.2).

Il bang forza l'oggetto a restituire il valore contenuto nella sua memoria.

Riflettiamo un momento sulla diversità dei box in alto: a sinistra un box messaggio, a destra un box numerico. Se mettiamo un box numerico anche a sinistra, ogni volta che cambia il numero (con il dragaggio del mouse) verrà aggiornato il risultato. Che succede se lasciamo un argomento all'interno dell'oggetto “+”? L'argomento viene semplicemente aggiornato nel momento in cui attiviamo il box numerico connesso a destra.

3.2.1 Alcuni oggetti matematici

- **pow** elevamento a potenza
- **sin** funzione seno di un numero

- **cos** funzione coseno di un numero
- **tan** funzione tangente di un numero
- **atan** funzione arcotangente di un numero
- **exp** esponenziale
- **log** logaritmo naturale

L'oggetto **pow** compie un'operazione di elevamento a potenza in cui la base è un numero inviatogli all'inlet sinistro e l'esponente è il suo argomento o altro numero inviatogli all'inlet destro. Invece l'oggetto **exp** calcola il valore del numero e (numero di Eulero: 2.71828...) elevato per l'esponente dato dal valore del suo inlet. Analogamente l'oggetto **log** calcola il logaritmo naturale, ossia di base e . Naturalmente, servendosi delle usuali formule matematiche, possiamo ottenere con PD i logaritmi in qualunque base. Ad esempio, se vogliamo calcolare il Log_{10} in base 10 di 6 usiamo la formula indicata in figura 3.3:

$$\text{Log}_{10} x = \log x / \log 10$$

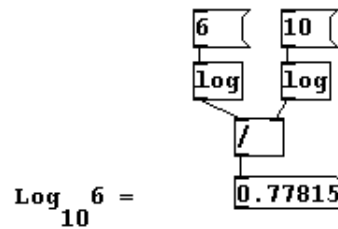


Figura 3.3:

3.2.2 Altri oggetti matematici

Altri oggetti matematici sono **mod**, **div** e **abs**:

mod è l'operatore modulo e va usato con un argomento. Esso produce il resto della divisione tra il numero in ingresso e l'argomento. Ad esempio, se l'argomento è 12, avremo: $5 \bmod 12 = 5$, $13 \bmod 12 = 1$, e così via.

div è il contrario di *mod*. Ossia restituisce la parte intera della divisione tra il numero in ingresso e l'argomento. Ad es. $5 \div 3 = 1$, $26 \div 3 = 8$, ecc.

abs restituisce il valore assoluto del numero in ingresso. Per cui a -7, *abs* manda in uscita 7.

3.3 Oggetti relazionali

Nella patch di fig. 3.4, utilizziamo l'oggetto relazionale ">".

Avrete certamente intuito il suo funzionamento: nell'inlet di sinistra inseriamo il numero da confrontare (ancora una volta usando un messaggio e un box numerico), a destra inseriamo invece l'argomento, ossia il numero sul quale effettuare il confronto. Nella patch un click su messaggio contenente 3 ha dato in uscita 0, perché è *falso* che 3 sia minore di 16. Ricordiamo infatti, come è tipico nei linguaggi informatici, che 1 significa *Vero*, mentre 0 significa *Falso*.

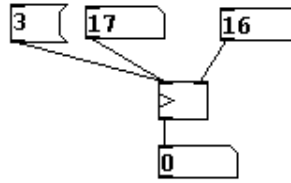


Figura 3.4:

3.3.1 Oggetti simili:

- `==` è uguale a
- `!=` è diverso da
- `>` è maggiore
- `<` è minore
- `<=` è minore o uguale

3.3.2 Altri oggetti relazionali

In modo simile, ma con qualche sfumatura di differenza, si comportano gli oggetti **max** e **min** (vedi fig. 3.5).

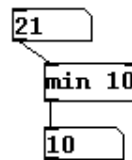


Figura 3.5:

Anche l'oggetto **min** effettua un confronto tra il numero che gli inviamo, tramite l'inlet sinistro, e il suo argomento (o il numero inviato all'inlet destro). Ma in questo caso invece di avere in uscita *Vero* o *Falso* ci restituirà il numero più piccolo tra i due. Nell'esempio di Fig. 6, l'oggetto ha emesso 10 perché è il numero più piccolo tra 21 e 10 (argomento). Analogamente, ma in modo inverso, si comporta l'oggetto **max** che restituisce il maggiore tra il numero da confrontare e l'argomento.

3.4 L'oggetto random

L'oggetto **random** è un generatore di numeri casuali (o, per meglio dire, pseudocasuali) entro un range impostato dal valore dell'argomento. In dettaglio, i valori in uscita andranno da 0 a N-1 se l'argomento è N.

La figura 3.6 ci mostra la modalità d'uso. L'oggetto contiene un argomento, in questo caso 10, ma tale numero può essere aggiornato a piacere prevedendo un box numerico da connettere all'inlet destro. Con tale impostazione sappiamo che avremo in uscita numeri da 0 a 9 (N-1). Per attivare l'oggetto basta inviare un bang all'inlet sinistro. In uscita l'oggetto ha prodotto 2.

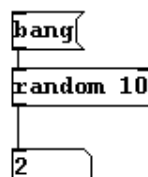


Figura 3.6:

3.5 Conversioni

In molti casi, per esprimere l'altezza, è utile servirsi dei valori di nota MIDI. Esistono dunque, allo scopo, oggetti che compiono le dovute conversioni dal valore MIDI al corrispondente valore in Hz. Ad esempio l'oggetto **mtof**, che è appunto la contrazione di *Midi To Frequency*, di cui vediamo in fig. 3.7 un semplice esempio:

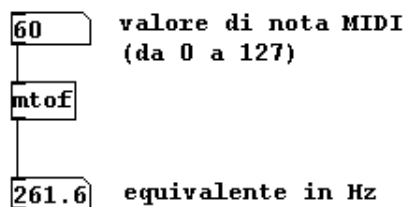


Figura 3.7:

L'oggetto non accetta argomenti (non ha infatti una seconda porta di ingresso). Nell'esempio il valore di nota 60 - corrispondente, come si sa, al Do centrale del pianoforte - viene convertito in 261.6 che esprime proprio la frequenza in Hz. di quella nota. E' possibile impostare valori più piccoli dell'intero effettuando il dragaggio con il pulsante del Maiuscolo premuto. Si ottiene in tal modo una risoluzione in centesimi. Analogamente, ma in modo inverso, si comporta l'oggetto **fptom**.

3.5.1 Altre conversioni

Altri tipi di conversione che possono risultare utili sono quelle che interessano la trasformazione da una misurazione in decibel (dB) e una in Rms (valore efficace), e viceversa. E' quanto fanno, rispettivamente, gli oggetti **dbtorms** e **rmstodb**. Ricordiamo che il valore Rms (*Root Mean Square*, radice della media dei quadrati) rappresenta una particolare media delle variazioni di pressione dell'onda nel tempo. A 100 dB il valore Rms equivale ad 1.

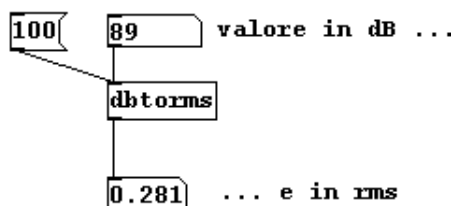


Figura 3.8:

Nell'esempio di fig. 3.8, a 89 dB corrisponde un valore di 0.281 in Rms.

3.6 Ancora sui numeri

Gli oggetti **int** e **float** sono come dei magazzini di numeri, rispettivamente interi e reali. Come già si è visto per gli oggetti matematici, come ad esempio “+”, la differenza tra i due inlet (sinistro e destro) è che a quello sinistro è associata una doppia azione: si memorizza il numero e si forza la sua uscita (nell’outlet). Ciò che è inviato nell’inlet destro, invece, rimane nella memoria dell’oggetto e verrà mandato in uscita solo se l’oggetto sarà raggiunto dal messaggio bang.

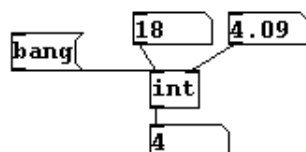


Figura 3.9:

Come si vede in fig. 3.9, agendo sull’inlet di destra abbiamo bisogno di un successivo *bang* per avere un’uscita. Che peraltro darà un risultato troncato al solo intero, tralasciando il decimale.

3.7 L’oggetto clip

Infine consideriamo **clip**, un oggetto che confronta il numero in ingresso con i suoi due argomenti e lo trasmette in uscita solo se è compreso tra essi. Se eccede - più grande o più piccolo - l’oggetto rilascia il numero rispettivamente più alto o più basso tra i suoi argomenti. Questi possono essere integrati nell’oggetto o inseriti tramite i due inlet più a destra.

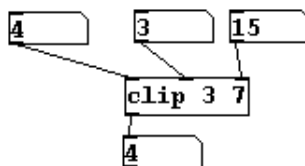


Figura 3.10:

Nell'esempio di fig. 3.10 gli argomenti sono 3 e 15 (il 15 sovrascrive il secondo argomento integrato nell’oggetto). Il 4 inviato all’inlet sinistra darà in uscita 4 perché è un numero compreso fra 3 e 15.

3.8 Esercitazione

Il rapporto tra l’universo acustico e quello psicoacustico riporta sempre in primo piano la necessità di adattare i criteri di grandezza fisici con quelli, più familiare, alla quale la notazione musicale ci ha abituati. Un caso tipico è la relazione tra la scala *lineare* delle frequenze e quella

logaritmica delle scale diastematiche (le note, nel senso musicale del termine). Il protocollo MIDI, proprio per venire incontro al musicista, ha adottato il criterio di scegliere un sistema di rappresentazione basato sulla scala temperata e in tal senso anche i softwares musicali più a basso livello, come lo stesso PD, hanno scelto di introdurre i criteri di rappresentazione del MIDI. A questo si dedicherà più avanti un intero capitolo.

Per ora ci interessa utilizzare gli strumenti matematici per proporre un esempio di conversione fra la dimensione delle frequenze, espressa in Hz. ed un criterio di segmentazione del continuo secondo i più familiari steps dei dodici semitoni per ottava. Il problema è: partendo da una qualsiasi altezza, come trovare, rispetto a quella, le frequenze poste a distanza di n semitoni, intesi questi nel senso temperato del termine. Va da sé, peraltro, che una volta trovato un principio di calcolo - e averlo reso facile e automatico grazie a PD - potremo comodamente piegarlo verso interessi più orientati alla ricerca e allo studio. Ad esempio potremo ipotizzare una segmentazione dell'ottava in un numero diverso dai classici 12 gradi.

L'argomento della nostra esercitazione trova spunto da un fattore costante utilizzato da Miller Puckette - autore di PD - nel suo tutorial al programma (cfr. la patch G09.pitchshift.pd). Grazie a questo fattore, che è 0.05776, inserito in un particolare algoritmo di calcolo, è possibile ottenere la trasposizione di una qualsiasi frequenza ad una distanza di n semitoni (temperati). Ci proponiamo di scoprire come si ottiene questo numero e di realizzare una patch che ci fornisca il valore in Hz della frequenza desiderata.

Partendo dalla frequenza del La = 440, e sfruttando le proprietà del numero e , possiamo impostare la seguente equazione:

$$440 * e^{nx} = \text{frequenza desiderata}$$

dove n rappresenta il numero dei semitoni di distanza dal La del diapason ed x il fattore costante in oggetto.

E' evidente che per $n = 0$, ossia quando non c'è alcuna trasposizione, il risultato è 440; quando invece $n = 12$, si avrà:

$$440 * e^{12x} = 880$$

Risolviamo dunque l'equazione esponenziale trovando la x . Semplificando si ha:

$$e^{12x} = 2$$

$$\log(e^{12x}) = \log 2$$

$$12x \log e = \log 2$$

poichè $\log e = 1$, si ha:

$$12x = \log 2$$

$$x = \frac{\log 2}{12} \tag{3.1}$$

$$x = \frac{0.69314}{12}$$

$$x = 0.05776$$

fermandosi alla 5a. cifra decimale.

A questo punto, trovato il fattore costante, possiamo calcolare, ad esempio, la frequenza (in Hz) di una 4a sopra il La del diapason (ossia il Re5) risolvendo il calcolo:

$$440 * e^{5*0.05776} = 440 * e^{0.2888} = 440 * 1.33482 = 587.323$$

oppure, per avere la frequenza del Do centrale, devo impostare $n = -9$ (9 semitoni sotto il La del diapason), per cui si avrà:

$$440 * e^{-9*0.05776} = 440 * e^{-0.51984} = 440 * 0.594616 = 261.631.$$

Realizziamo ora tutto questo in PD, proponendo la seguente patch:

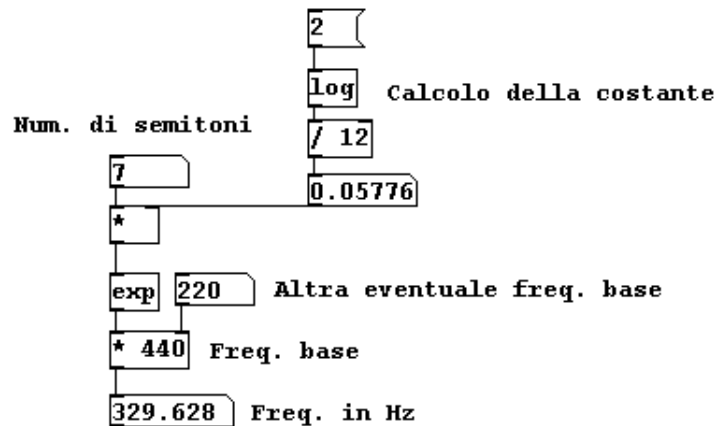


Figura 3.11:

In alto a destra si svolgono i calcoli per ottenere il fattore di moltiplicazione 0.05776 in base alla formula 3.1. Il valore ottenuto viene quindi moltiplicato per il numero di semitoni per i quali intendiamo trasportare una frequenza base. Il tutto entra in **exp** la cui uscita è uno dei fattori dell'oggetto ***** che ha per argomento la frequenza di riferimento (qui 440 Hz). Tale argomento, tuttavia, può essere aggiornato a piacere agendo sull'apposito box numerico: nell'esempio abbiamo inserito 220, il La sotto il Do centrale.

3.9 Abbiamo studiato

box numerico, oggetti di matematica di base (**+**), oggetti di matematica trigonometrica (**sin**, **cos**, **tan**, **atan**), **exp**, **log**, **mod**, **div**, **abs**, oggetti relazionali, **random**, **clip**, conversioni (**mtof**, **dbtorms**, **rmstodb**), contenitori (**float**, **int**).

Capitolo 4

Manipolazione del tempo

PD dispone di molti oggetti per la manipolazione del tempo che si rivelano assai preziosi. Strumenti per ritardare il compiersi di un'azione, per misurare il tempo, per gestire il verificarsi periodico e regolare di un evento, fino ad arrivare a monitorare il tempo della CPU, e molto altro ancora. Cominciamo con illustrare l'oggetto *delay*.

4.1 Delay

Quando **delay** riceve un bang (inlet sinistro) manda in uscita il medesimo bang dopo che è trascorso un lasso di tempo, in millisecondi, determinato dal suo argomento. Nel caso in fig. 4.1:

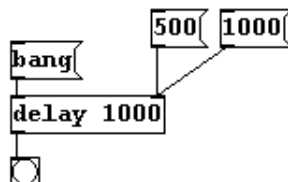


Figura 4.1:

l'argomento è 1000 (1 secondo). Se clicchiamo sul messaggio *Bang* connesso all'inlet sinistro, dovremo aspettare un secondo prima di vedere attivarsi l'oggetto *bang* connesso all'uscita.

Come sempre l'inlet destro è destinato a ricevere un argomento. Nel caso in figura, l'argomento 1000 è integrato all'oggetto, ma possiamo cambiarlo cliccando sul messaggio 500, o riportarlo a 1000, cliccando sul messaggio corrispondente.

In attesa che si compia il ritardo (delay) possiamo interrompere il processo inviando, nell'inlet sinistro, un messaggio di Stop. Come illustra la fig. 4.2:

Si ricorda che *delay* manda in uscita solo un bang. Quindi sarebbe un errore impostare con *delay* l'uscita ritardata di un simbolo. Pensiamo ad esempio di dover impostare la comparsa della stringa "Risultato" dopo 3 secondi che il programma ha compiuto il seguente calcolo: 12 elevato alla 3a. L'operazione è certo inutile, ma a noi serve per fare una semplice esercitazione.

Come si vede in fig. 4.3, all'uscita di *delay* si ha un bang che entra nell'oggetto *bang* - il cui scopo è solo quello di monitorare l'avvenuta azione. Il messaggio viene quindi inviato all'oggetto *symbol* - e non direttamente al campo testo (*symbol*). All'interno dell'oggetto *symbol*

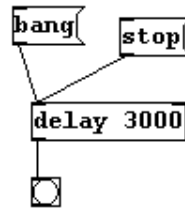


Figura 4.2:

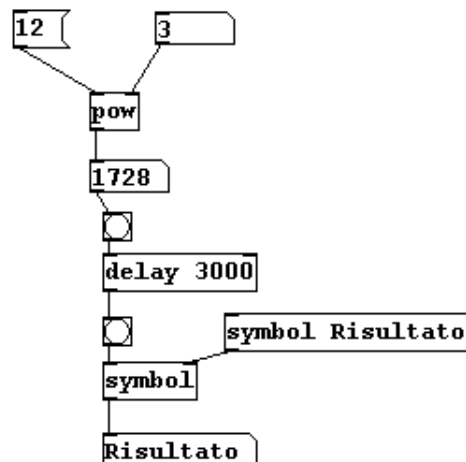


Figura 4.3:

vi è la stringa “Risultato” e ciò è stato ottenuto inviando un opportuno messaggio nell’inlet destro dell’oggetto.

4.2 Pipe

L’oggetto **pipe** è molto simile a *delay*: ritarda l’uscita di ciò che è entrato nel suo inlet di un certo ammontare di tempo. Il nome in inglese vuole dire ‘tubo’ ed infatti l’oggetto ricorda il comportamento di una conduttura dove l’acqua entra da una parte e, secondo la lunghezza, fuoriesce dall’altra parte dopo un certo lasso di tempo. La differenza con *delay* è che *pipe* non si limita ad emettere un *bang*, ma ritarda un vero e proprio messaggio, che può essere un numero, dei simboli, un puntatore.

Il cosa viene ritardato dipende dall’argomento e anche il tempo di ritardo può essere impostato con un valore di argomento (che in ogni caso può essere aggiornato tramite l’inlet destro). Dunque, nel caso più semplice, *pipe* presenta due argomenti, come nell’esempio in fig. 4.4, *s 800*, che possono interpretarsi: “aspettati dati di tipo simbolico (una stringa di caratteri) e ritardane l’uscita dopo 0.8 secondi”.

Se l’oggetto ha solo un argomento questo deve essere numerico e si riferisce alla durata del ritardo.

Esaminiamo invece i due casi di fig. 4.5:

Entrambe le patch dichiarano, come argomento, **f** ed **s** oltre che *1000*. Se quest’ultimo si riferisce alla durata del ritardo, le prime due lettere indicano, in forma compatta, il tipo di dati

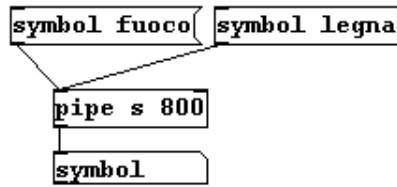


Figura 4.4:

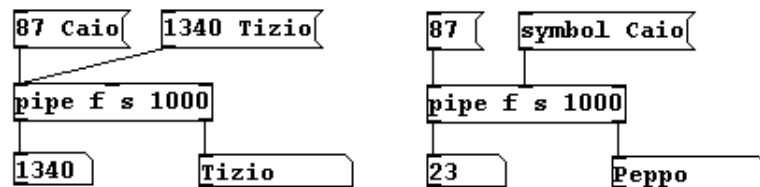


Figura 4.5:

attesi nei due inlet di ingresso, rispettivamente *float* (numero) e *symbol* (stringa di caratteri). Tuttavia vi sono alcune differenze: nella scrittura e nel comportamento. Quanto alla scrittura, nella patch di sinistra entrambi i dati entrano nel primo inlet, nella seconda entrano ciascuno in un proprio inlet. Inoltre nella patch di destra è necessario dichiarare anche nel messaggio la parola *symbol*. La differenza nel comportamento risiede invece nel fatto che solo il messaggio inviato a sinistra produce un effetto; quello mandato all'altro (o, eventualmente, altri) inlet si limita a memorizzare il dato nell'oggetto senza che venga spedito all'uscita.

L'oggetto *pipe*, infine, risponde ad altre tipologie di messaggi, da inviarsi all'inlet sinistro:

- **flush**, forza l'uscita dei messaggi in memoria, prima - cioè - che sia trascorso il tempo di ritardo
- **bang**, manda in uscita i messaggi in memoria. Può essere utile quando *non* si è inviato un messaggio nell'inlet di sinistra
- **clear**, cancella tutti i messaggi in memoria.

4.3 Metro

L'oggetto **metro** manda in uscita una serie di bang periodici, secondo una velocità costante stabilita nel suo argomento.

Vediamo in fig. 4.6 come si usa *metro*. Un bang iniziale attiva l'oggetto, quindi inizia ad uscire un flusso periodico di bang secondo una frequenza impostata dall'argomento. Nell'esempio l'oggetto ha per argomento 500 (msec.): ciò vuol dire che uscirà un bang ogni mezzo secondo. Posso sempre impostare, dall'inlet di destra, una velocità differente con apposito messaggio (in figura è usato uno box numerico per facilità d'uso). Per interrompere l'azione dell'oggetto basta inviare, all'inlet sinistro, un messaggio di *stop*. Come in altri casi in PD, il bang di attivazione può essere sostituito con 1 e il messaggio di *stop* può essere sostituito con 0.

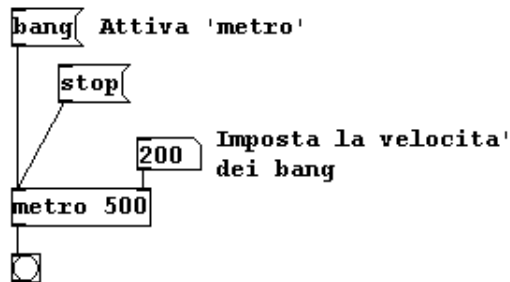


Figura 4.6:

Dunque la patch di prima può essere riscritta, con il medesimo funzionamento, come indicato in fig. 4.7:

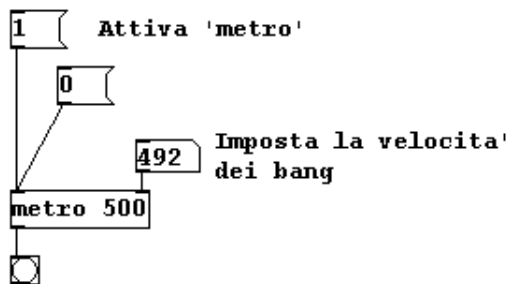


Figura 4.7:

4.4 Misurare il tempo

PD dispone di tre oggetti specificamente pensati per misurare il tempo:

1. **timer**
2. **cputime**
3. **realtime**

Può sembrare strano: il tempo non è sempre lo stesso? In effetti vi sono delle piccole differenze: **timer** misura il cosiddetto tempo logico, quello che non è influenzato dal mondo reale e dalle leggi della fisica.

cputime misura il tempo che il processore (la CPU appunto) impiega a svolgere il compito richiestogli. Sappiamo che la CPU controlla una serie di processi simultaneamente (o quasi), nel senso che dedica una frazione di tempo ad un compito, poi salta ad un altro, poi ad un altro ancora, e dopo un po' ritorna al processo iniziale. E così di seguito. In tal modo i tempi della CPU possono risultare diversi da quelli effettivi relativi al singolo processo.

realtime, infine, misura il cosiddetto tempo reale, quello del mondo fisico, come potrebbe fare il nostro orologio. E lo fa sulla base del clock interno del computer.

Nella sua forma più semplice una patch che usi uno di questi oggetti (scegliamo *realtime*) si presenta come in fig. 4.8, lavorando come può fare un cronometro che facciamo partire e fermare, leggendo il tempo trascorso tra le due operazioni.

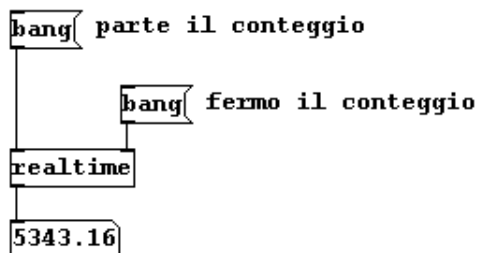


Figura 4.8:

Nell'esempio il tempo trascorso, indicato in msec., è di poco più di 5 secondi. In modo analogo si usano gli altri due oggetti, *timer* e *cputime*.

4.5 L'oggetto *line*

L'oggetto **line** è un generatore di rampe che si rivela utilissimo nella gestione di numerosi parametri del suono, primi fra tutti l'involuppo e il rilascio. Si basa essenzialmente su due valori:

- un valore *target*, ossia a quale valore numerico deve arrivare
- un *tempo*, che esprime quanto velocemente si arriverà al valore target

Come indica la fig. 4.9, il tempo incide sulla pendenza della rampa: a valori di tempo più brevi la rampa risulta più ripida.

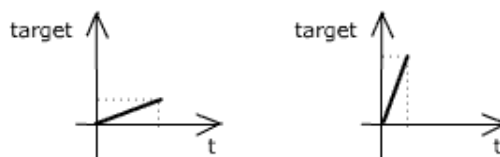


Figura 4.9:

L'esempio che segue (fig. 4.10) mostra un semplice impiego di *line*.

E' un'occasione per introdurre (sia pure provvisoriamente) un altro oggetto, **trigger** che per comodità si semplifica con la sola **t**. Come dice il termine, *trigger* si comporta come un interruttore che attiva un processo di messa in sequenza di messaggi in base ai suoi argomenti. In particolare esso accetta degli argomenti che specificano il tipo di dati da trasmettere. In questo caso i due argomenti sono delle **b**, lettera che sta per *bang*. Dunque, anche inviandogli un solo bang, dai suoi 2 outlet *trigger* manderà in uscita due *bang*, uno per ciascuno dei due box messaggio. Il numero degli argomenti si riflette dunque sul numero di outlet presenti nell'oggetto istanziato.

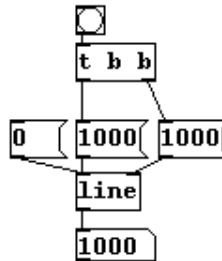


Figura 4.10:

Scendendo nella patch, a *line* giungono due valori, 1000 (che è il valore target) e 1000 (la durata della rampa, in msec.). Bisogna fare attenzione che, provata la patch e raggiunto il valore target di 1000, ora *line* contiene 1000 e non ricorda altro. Per cui se proviamo ad attivare ancora il processo non succederà nulla. E' chiaro: *line* non può arrivare dove già è arrivato. Ecco perchè nella patch abbiamo messo anche un messaggio 0 (sulla sinistra). Poichè si tratta di un solo valore, *line* raggiunge il target (0) immediatamente: la rampa, in questo caso, è paragonabile ad un segmento verticale. Se ora riproviamo ad azionare il *bang* in alto la patch riprende a funzionare come prima.

Un altro modo, più compatto, di usare *line* è quello di inviare un messaggio contenente entrambi i valori che l'oggetto si aspetta (fig. 4.11).

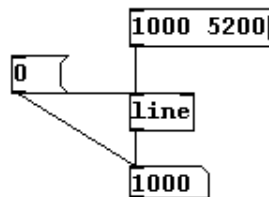


Figura 4.11:

Una variante alla patch precedente può essere quella di fig. 4.12, che presenta un nuovo oggetto, **unpack**:

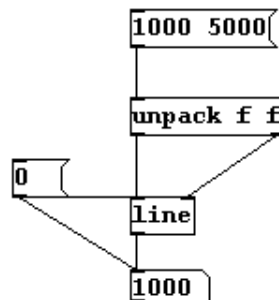


Figura 4.12:

Unpack spacchetta in messaggi singoli i dati che gli vengono inviati. Gli argomenti, integrati all'oggetto, specificano il tipo di dati che riceverà e che manda in uscita (dopo averli separati): qui **f** sta per *float*, ossia numeri reali. Rispetto alla precedente, nella patch di fig. 4.12 *line* riceve un valore per entrambi i suoi inlet.

Per completezza accenniamo anche all'inverso di *unpack* che è **pack**, un oggetto che serve ad impacchettare e concatenare in una singola lista un insieme separato di messaggi. A seconda di quanti argomenti possiede l'oggetto, al momento della creazione, vi saranno altrettanti inlet per ricevere i dati. Se i messaggi inviati ai rispettivi inlet non corrispondono alla tipologia degli argomenti PD risponde segnalando l'errore.

4.6 Esercitazione 1: costruire un contatore

Costruiamo un semplice contatore che serva a contare il verificarsi di particolari eventi. Nel caso più semplice: che conti quante volte abbiamo cliccato su un pulsante (l'oggetto *bang*). Per risolvere il problema abbiamo bisogno di un oggetto che memorizzi un numero e di un altro che incrementi quel valore in memoria di un'unità. Come si vede dalla fig. 4.13 abbiamo scelto, per il primo caso, l'oggetto **int** (con argomento 0) e per il secondo l'oggetto **+** (con argomento 1).

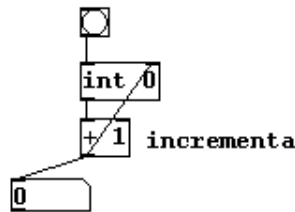


Figura 4.13:

Ogni volta che clicchiamo sul pulsante in alto l'oggetto **int** viene forzato a mandare in uscita il contenuto della sua memoria (inizialmente 0). Questo numero viene incrementato di 1 (dall'oggetto **+**) e quindi mandato nel box numerico per la visualizzazione del risultato. Ma, secondariamente, dall'oggetto **+** esce un'altra connessione. Essa fa tornare indietro il valore incrementato proprio nell'oggetto *int* che memorizzerà il nuovo valore. Pertanto, quando cliccheremo nuovamente sul pulsante, verrà mandato in uscita il valore incrementato. A questo punto il gioco è fatto: il processo continuerà con ulteriori incrementi di 1.

Perfezioniamo il nostro contatore introducendo la funzione di azzeramento per un successivo utilizzo. Per fare questo inviamo un messaggio contenente 0 al box numerico del risultato, come illustrato in fig. 4.14. Il box contiene ora lo 0.

Tuttavia, se proviamo la nuova patch, ci accorgiamo che non è soddisfacente. Il box in basso, dopo l'azzeramento, non ricomincia da 1, ma dal numero con cui era terminata la prova precedente. Ciò è facilmente spiegabile: abbiamo azzerato solo il box numerico del risultato e non la memoria dell'oggetto *int* che continua a contenere il vecchio numero. Dobbiamo dunque inviare il messaggio di 0 anche ad *int*.

In fig. 4.15 appare dunque la patch definitiva:

Il messaggio di azzeramento ora raggiunge anche *int* che manda in uscita direttamente sul box numerico (mentre prima il box riceveva il valore dall'oggetto **+**).

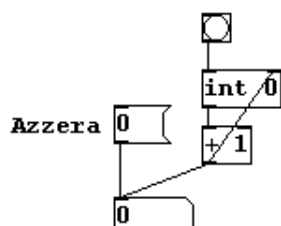


Figura 4.14:

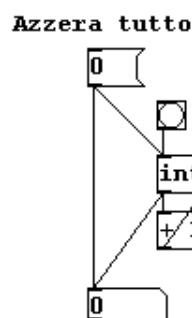


Figura 4.15:

4.7 Esercitazione 2: costruire un orologio

Durante una performance con PD, può essere comodo avere nell'ambiente di lavoro un timer che ci informi sul tempo trascorso dall'inizio del concerto o da un certo particolare evento. Costruiamo allora un orologio che conti i minuti e i secondi (rimandiamo al lettore il compito di impostare anche il computo delle ore). Conosciamo quasi tutti i 'pezzi' necessari tranne due, che ora presentiamo. Il primo oggetto si chiama **toggle**, graficamente appare come un quadratino bianco e si basa su due soli stati. Quando vi clicchiamo sopra si riempie con una crocetta; cliccandovi nuovamente ritorna ad essere bianco e così via. A tali stati grafici corrisponde, sul piano logico 1 o 0 che può interpretarsi anche - a seconda dei casi - come Sì/No oppure come bang/stop. (Ma il fatto che "possa interpretarsi" non vuol dire che *sia* un bang: se lo colleghiamo a monte di alcuni oggetti che ammettono solo un bang - come ad esempio *timer* - riceveremo un messaggio di errore).

Nella patch del nostro orologio (fig. 4.16, *toggle* si trova in alto a tutto e serve per attivare il dispositivo:

Cliccando dunque su *toggle* inviamo 1 all'oggetto *metro* sottostante che inizia a scandire un flusso di *bang* alla frequenza di 1 al secondo. Poco sotto possiamo riconoscere la coppia *int* e '+' che serve a creare un primo contatore (quello dei secondi) con incremento di 1.

Scendendo troviamo il secondo nuovo oggetto, di nome **select**. Si tratta di un oggetto che effettua un filtraggio: confronta i numeri in arrivo rispetto al suo argomento e se il confronto è positivo manda all'uscita *sinistra* un bang, se invece è negativo copia il numero inviatogli all'uscita *destra*. Dunque ricordiamo: a destra esce un *numero*, a sinistra un bang (si veda, nel prossimo capitolo, la differenza con *moses* da cui escono invece, opportunamente smistati, due numeri). Nel nostro caso il flusso di numeri che raggiunge *select* viene controllato rispetto a 60 (l'argomento): i numeri da 0 a 59 usciranno tutti da destra mentre, quando giunge 60,

Capitolo 5

Controllo sui dati

Tra i compiti più importanti di un programma è il controllo che può esercitare sul flusso dei dati che gestisce. In una patch musicale vi saranno svariate necessità di controllare i dati: ad esempio controllare una soglia di volume o di altezza, monitorare il superamento di certi valori e poter interrompere il flusso, ripartire secondo particolari criteri gli eventi, e così via.

5.1 Filtri, smistamenti ed altro: **moses**, **spigot**, **change**

Abbiamo già introdotto un oggetto fondamentale nel controllo sui dati, **select**. Cominciamo col presentare un oggetto assai simile, **moses**.

5.1.1 Moses

L'oggetto **moses** smista l'uscita a destra o sinistra dei numeri in ingresso, confrontandoli con il suo argomento: quelli minori dell'argomento escono a sinistra, quelli maggiori o uguali escono a destra. Come sempre l'argomento di confronto può essere aggiornato tramite l'inlet destro. Vi è dunque una differenza rispetto a **select** che smista i dati in ingresso producendo due tipologie di uscite: bang a sinistra e dati grezzi (quelli entrati) a destra.

5.1.2 Spigot

Anche **spigot** agisce come un filtro sui dati, controllando ciò che arriva al suo ingresso. Più precisamente si comporta come una vera e propria porta che può essere aperta o chiusa. Per aprire o chiudere la porta si usa un messaggio numerico inviato all'inlet destro: se è 0 *spigot* non lascia passare niente, se è un altro numero fa passare tutto. Per comodità si ricorre alla proprietà del box numerico di cui si è parlato: se la sua ampiezza (: *proprietà*, *width*) è impostata ad 1, si comporta come un commutatore. Ad un click diventa 1, al successivo diventa 0 (o viceversa).

Quindi possiamo pensare una patch come quella di fig. 5.1:

All'inlet sinistro inviamo i dati da sottoporre al controllo di *spigot*, a quello destro abbiamo il commutatore tra 0 ed 1. Nella posizione in figura *spigot* non lascia passare niente, se invece commutiamo ad 1, la porta è aperta.

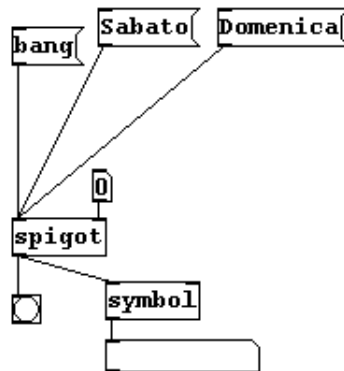


Figura 5.1:

5.1.3 Change

L'oggetto **change** effettua un controllo su dati in ingresso fermando tutte le ridondanze. Più in dettaglio: memorizza il dato inviatogli sull'inlet sinistro e lo manda in uscita. Quando gli arriva un successivo dato, verifica che non sia uguale a quello in memoria. Se è uguale non lo lascia passare, se è differente manda in uscita il nuovo dato e nel contempo lo fissa in memoria per i controlli successivi. E' possibile, tuttavia, forzare l'uscita anche di un dato ridondante: basta inviare a *change* un *bang* sull'inlet destro.

L'oggetto *change* può anche essere istanziato con un suo argomento: in tal caso i dati che gli vengono inviati saranno confrontati con quello.

Talvolta può essere utile cambiare argomento a *change* durante una performance, ossia rimanendo nel modo Utente. All'uopo occorre prevedere un messaggio *set* con argomento variabile. Consideriamo il caso in fig. 5.2:

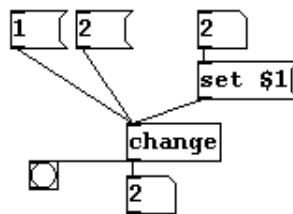


Figura 5.2:

Sulla destra, il messaggio *set* ha per argomento una variabile, riconoscibile per il simbolo del dollaro *\$1*. In sostanza *\$1* vuole dire: ciò che viene immesso tramite l'inlet del *box* diventa il mio argomento. La lettera dollaro *\$* è dunque una lettera di PD dotata di un suo significato specifico. Il numero 1 a fianco è solo il nome di quella variabile. Una seconda variabile sarebbe *\$2* e così via. Il messaggio *set* serve appunto ad impostare un oggetto, ossia a fornire i dati di cui ha bisogno per lavorare (o per cambiare le sue impostazioni iniziali), normalmente un argomento.

Dragando sul box ci siamo fermati sul numero 2; questo numero viene memorizzato nella variabile *\$1* e inviata a *change*. A questo punto ciò *change* riceve sul lato sinistro verrà

confrontato con 2. Cliccando, ad esempio, sul box contenente 2 non avremo nulla in uscita perchè *change* ha fermato il messaggio.

5.2 Ancora su trigger

L'oggetto **trigger**, che abbiamo già introdotto, merita un ulteriore approfondimento. Abbiamo visto che esso attiva una messa in sequenza di messaggi, in funzione degli argomenti con cui è istanziato. Ciò vuol dire che trigger compie anche, se necessario, una conversione del tipo di messaggi che gli vengono inviati. Non solo, ma tali messaggi usciranno secondo un ordine che è tipico di PD, da destra a sinistra. Questo ordine può tornare comodo per ridurre al minimo le istruzioni della nostra patch. Consideriamo l'esempio seguente:

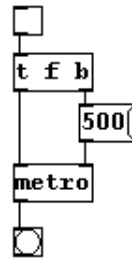


Figura 5.3:

In alto abbiamo l'oggetto *toggle* che manda in uscita - alternativamente - 0 o 1. Con una sola azione (un semplice click) otteniamo un duplice effetto:

- attivare il box numerico (che ha per argomento 500) e
- attivare l'oggetto *metro*.

Quest'ultimo per partire, lo ricordiamo, si aspetta di ricevere un *bang* oppure un 1. Per fermarsi deve ricevere uno 0. Esaminando gli argomenti di *trigger* (qui solo **t**) notiamo una **f** e una **b** che stanno, rispettivamente, per *float* e *bang*. Ciò vuol dire che l'oggetto converte il messaggio in ingresso (un numero: 1 o 0) in altrettanti messaggi: numerico (per *f*) e di *bang* (per *b*). E lo fa nell'ordine destro/sinistro: quindi il *bang* raggiunge il box numerico *prima* (almeno sul piano logico) che il numero 1 raggiunga l'oggetto *metro*. In tal modo quando *metro* viene attivato già contiene un argomento (500) e quindi produrrà un treno di *bang* ad intervalli di mezzo secondo.

5.3 Route

Anche l'oggetto **route** effettua un controllo dei dati in ingresso, ma ha un comportamento più articolato mostrandosi utile specialmente quando, invece che un dato solo, si invia una lista di dati. Consideriamo di inviare a *route* un messaggio contenente una lista di numeri. Per prima cosa l'oggetto valuta il primo elemento della lista confrontandolo con i suoi argomenti. Gli argomenti possono essere numeri o simboli, oppure possono essere semplici tipi di dati (ad es.: *symbol*, *list*, *float*, *bang*), ma di questo parleremo dopo. In base agli argomenti vi saranno tanti

outlet più 1, quest'ultimo è l'uscita di scarto, dove sono direzionati i dati che non incontrano corrispondenza. Quindi, per fare un esempio, con 3 argomenti si avranno 4 outlet.

Caso 1: *route* ha un solo argomento. Se trova la corrispondenza (dato/argomento) manda in uscita nell'outlet a sinistra, altrimenti a destra.

Caso 2: *route* ha più argomenti. La corrispondenza da trovare è sempre tra il *primo* della lista con ciascuno degli argomenti dell'oggetto. Se l'incontro è *primo/primo* si avrà l'uscita sul primo outlet (da sinistra) della lista, con esclusione del primo elemento (quello della corrispondenza). Se l'incontro è *primo/secondo* l'uscita sarà sul secondo outlet (da sinistra) sempre con esclusione del primo elemento. Idem se l'incontro è *primo/terzo* e così via. Ovviamente se non c'è alcuna corrispondenza (sempre tra il primo della lista con uno qualunque degli argomenti) esce tutto dall'outlet più a destra.

Ma facciamo un esempio per capire meglio il meccanismo (fig. 5.4):

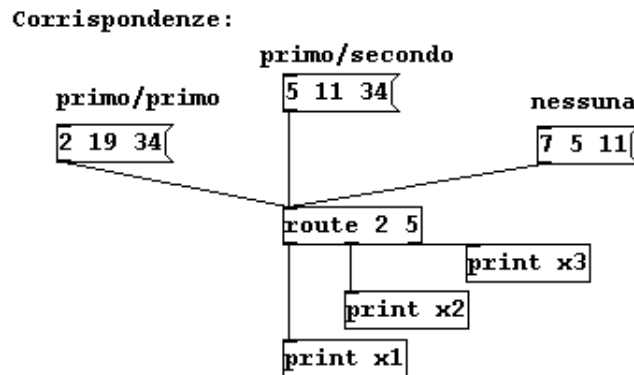


Figura 5.4:

Qui *route* ha 2 argomenti e quindi 3 sono gli outlet.

La prima lista, quella più a sinistra, incontra una corrispondenza con il primo argomento, pertanto l'uscita sarà l'outlet più a sinistra - quello che corrisponde al primo argomento - con la sequenza 19 34.

La seconda lista (al centro) ha la corrispondenza con il secondo argomento, dunque l'uscita avverrà sull'outlet centrale (il secondo da sinistra) con la sequenza 11 34.

La terza lista non ha corrispondenze: infatti, pur essendo presente il numero 5 tra gli argomenti, ciò è irrilevante perchè *route* controlla solo il primo valore della sua lista.

5.3.1 Gli argomenti di *route*

Nell'esempio precedente *route* aveva argomenti numerici, ma può avere anche argomenti di tipo *symbol*. In sostanza in base alla tipologia di argomenti che mettiamo al momento della creazione dell'oggetto, *route* si aspetterà un certo tipo di dati. Se non vi è corrispondenza quanto a tipologia, *route* ridirige tutto nell'outlet di destra.

Un caso particolare è l'uso di *route* per controllare proprio la tipologia dei dati. In tal caso mettiamo come argomenti non dei valori ma dei tipi, come *float*, *symbol*, *bang*, *list*.

Nell'esempio di fig. 5.5 si nota che può esserci una lista sia di numeri che di simboli: non fa alcuna differenza. Esaminiamo alcuni casi in cui il messaggio contiene:

- un numero: *route* lo interpreta come numero (*float*);

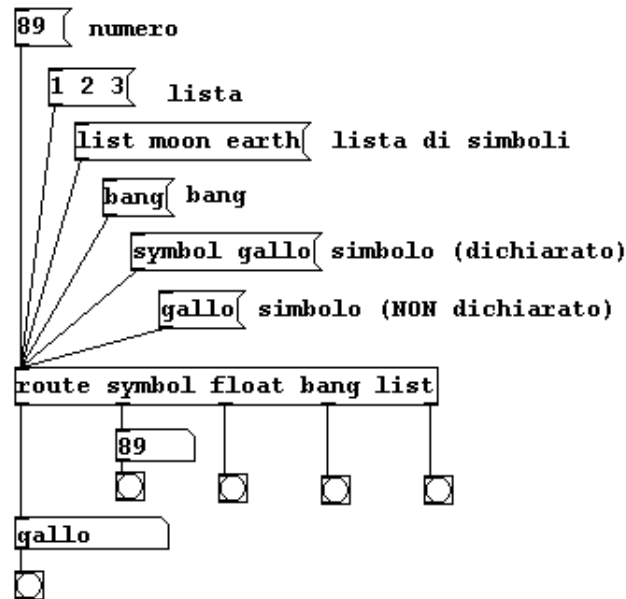


Figura 5.5:

- una lista di numeri: *route* la interpreta come *list*;
- una lista di stringhe (di testo) preceduta dalla parola riservata *list*: *route* la interpreta come *list*;
- una stringa di testo 'dichiarata' come *symbol*, (ossia preceduta dalla parola riservata *symbol*): *route* lo interpreta come *symbol*;
- una stringa di testo non dichiarata: *route* non la riconosce e la passa sull'ultima uscita a destra.

5.4 Controllo da tastiera

Anche se può sembrare fuori luogo, discutiamo in questo capitolo un'altra tipologia di oggetti in cui il controllo non viene effettuato da PD, ma dall'utente. PD infatti ha la possibilità, che si rivela in molti casi davvero utilissima, di gestire le patch usando la tastiera del computer (al posto del solito mouse). Ciò permette una interazione assai più immediata e una semplificazione delle operazioni più elementari, come attivare un *sinth*, prendere una misurazione, ridisegnare una tabella, e così via.

Quando si abbassa un tasto PD lo riconosce come numero. Si tratta del numero identificatore del tasto, da non confondere con i numeri impressi sul tasto e che vengono scritti sul terminale. Ad esempio il tasto che scrive 1 ha un ID uguale a 49; 7 è invece 55; e così via. Sono questi i numeri che PD cattura. Il controllo avviene separando gli eventi di 'abbassamento' e di 'ritorno' del tasto: in particolare **key** agisce quando si abbassa un tasto, mentre **keyup** quando il tasto è rilasciato. In entrambi i casi va in uscita il numero ID corrispondente. Infine, **keyname** riunisce entrambe le funzioni, in uscita sull'outlet sinistro, e in più riporta sull'outlet

destro la lettera o il numero che è effettivamente associato a quel tasto (ossia ciò che viene scritto da quel tasto).

Si propone un esempio in cui la patch viene guidata tramite tastiera (fig. 5.6). Il tasto P (il cui identificativo è 112) agisce sull'oggetto *metro*, mentre l'azzeramento del box numerico in basso avviene cliccando su un qualunque altro tasto (ovviamente compresa la barra spaziatrice).

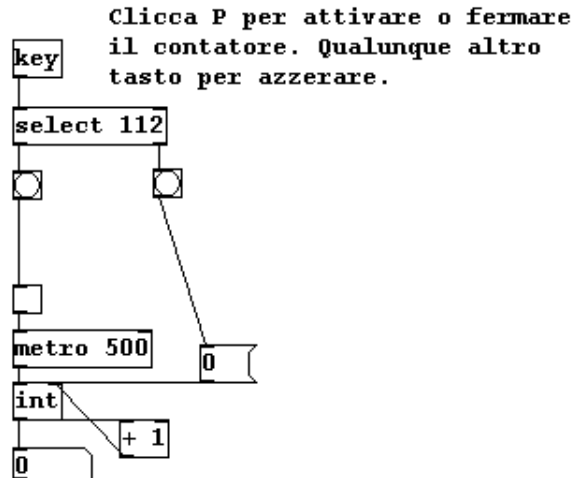


Figura 5.6:

Si noterà l'uso di *select* che smista a sinistra o a destra a seconda della corrispondenza tra l'ID del tasto e il numero 112. Quando clicchiamo su P *select* apre la porta a sinistra che attiva un *bang* (che, a dire il vero, ha solo una funzione di monitoraggio), quindi l'oggetto *toggle* diventa 1 (appare la relativa crocetta) e ciò fa partire *metro*. Al successivo click su P *toggle* darà zero (scompare la crocetta) e *metro* si interrompe. Un qualunque altro tasto fa aprire a *select* la porta di destra. Ciò attiva il messaggio 0 che viene inviato ad *int*, con conseguente azzeramento della memoria.

A proposito: come si fa a sapere quale numero ID ha un certo particolare tasto? La risposta ci è data sempre da PD. Basta porre sul nostro spazio di lavoro l'oggetto *key* connesso con un box numerico. Cliccando sul tasto (o la barra spaziatrice) rileveremo con facilità il numero ad esso associato.

5.5 Abbiamo studiato

- Oggetti: **moses**, **spigot**, **change**, **trigger**, **route**, **key**, **keyup**, **keyname**.
- Messaggi: **set**

Capitolo 6

Le tabelle

Chi si occupa di informatica musicale conosce bene l'importanza delle tabelle, che in inglese si chiamano arrays. Si tratta di contenitori indicizzati di dati di qualunque tipo. Il caso più tipico di impiego di tabelle, in ambito audio, è quello dell'oscillatore digitale. Vi è una fase di *scrittura*, come ad esempio nel processo di campionamento, in cui il suono digitalizzato riempie opportunamente una tabella che diventa il contenitore di tutte le sue specifiche e le sue trasformazioni nel tempo. E vi è una fase di *lettura* in cui dati contenuti nella tabella vengono letti secondo certe modalità e inviati al convertitore DAC che ci restituisce il suono.

Ma anche al di fuori dell'audio, tornando all'argomento di cui ci occupiamo in questa prima parte, le tabelle sono di grande utilità. Ad esempio possiamo scrivere in una tabella i dati di monitoraggio di un certo evento per studiare la frequenza o la densità di certe grandezze. E gli esempi potrebbero moltiplicarsi.

6.1 Creazione di una tabella

Il modo migliore per creare una tabella è cliccare sul menù *Put*, quindi *Array*. Si aprirà una finestra in cui è possibile inserire il nome che intendiamo assegnare alla tabella e la sua dimensione. Di default la dimensione è 100 e poiché l'indicizzazione degli array in PD va da 0 a $N - 1$, la nostra tabella avrà segmenti di memoria numerati da 0 a 99.

Tra le altre proprietà che possiamo settare nella finestra è il "save contents": in pratica, dopo aver lavorato con una tabella, possiamo chiedere a PD che i suoi valori rimangano memorizzati insieme alla patch alla chiusura, oppure che vengano cancellati. Nel caso di tabelle audio, memorizzare i contenuti vorrà dire appesantire di non poco la grandezza del file della patch ma, in alcuni casi, ciò può rivelarsi utile. Possiamo dunque servirci di questa opzione.

Altre possibili spunte riguardano le caratteristiche grafiche dell'array: per capire le differenze la cosa migliore è crearsi una tabella e provare a selezionare le varie voci. Lo vedremo più avanti.

Infine, le altre opzioni ci permettono di scegliere se, alla creazione di una nuova tabella, vogliamo creare anche un nuovo grafico oppure, se c'è, usare quello creato in precedenza.

6.1.1 L'oggetto **table**

Un altro modo per creare una tabella è usare l'oggetto **table** che si crea in modo analogo a tutti gli altri oggetti (*Put* → *Object*, quindi scrivere *table* dentro l'oggetto vuoto). A vederlo nella patch, *table* non presenta nè inlet nè outlet. Esso crea una sottopatch con dentro l'array

il cui nome può essere determinato al momento della creazione facendolo seguire alla parola *table*. Posso inoltre aggiungere, come secondo argomento al momento della creazione, anche la dimensione della tabella. Quindi se scrivo nell'oggetto `<table archivio 5>` significa che la tabella dentro la subpatch si chiamerà *archivio* e conterrà 5 punti (fig. 6.1).

In PD le subpatch non appaiono nella patch principale se non sotto forma di oggetto. Ciò è fatto per migliorare la presa visiva globale della patch, semplificandone i contenuti e lasciando in ombra i sottoprocessi. Se poi vogliamo vederli basterà un semplice click per aprire un taglio prospettico di secondo livello. E via di seguito (si possono infatti annidare anche differenti subpatch, una nell'altra).

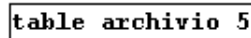


Figura 6.1:

Apriamo quindi *table*, cliccandovi sopra: vedremo una tabella con una riga orizzontale. Ciò vuol dire che la tabella è vuota e va riempita. Il modo più semplice è passare con il mouse dentro il riquadro della tabella: vedremo il cursore - che generalmente ha la punta rivolta a destra - assumere una diversa posizione, con la punta diretta verso sinistra. Ebbene quando il cursore punta a sinistra vuol dire che possiamo agganciare la curva dei valori interni della tabella e posizionarci a piacere all'interno del riquadro. Invece con la punta rivolta a destra questo non è possibile. Possiamo così caricare di valori la nostra tabella: nel nostro caso solo 5 (a causa del secondo argomento impostato).

6.2 Lavorare con una tabella

Facciamo subito una esercitazione. Creiamo una tabella con *Put* $\rightarrow$ *Array*, la chiamiamo *tabella* e la dimensioniamo a 10. Sempre tramite il pulsante destro avremo attivato una finestra di nome *canvas properties*: qui è possibile settare il range del grafico: poichè sull'asse verticale avremo valori da 0 a 10, impostiamo le due caselle *Y range: from* e *to* rispettivamente a 10 e 0. In PD le coordinate di un grafico partono sempre dall'angolo sinistro e in alto. Scrivendo da 10 a 0 imposto dunque che lo 0 sia in basso e il 10 in alto.

Quindi copiamo la patch come riportata in fig. 6.2.

In basso abbiamo il grafico della tabella, pronta ad essere riempita di dati (numerici).

A destra riconosciamo l'algoritmo di un contatore in cui è aggiunto l'oggetto *moses*, con argomento 10. Ciò significa che i primi 9 numeri passeranno a sinistra; a partire dal decimo numero si apre la porta destra di *moses* che attiva un bang connesso al *toggle* di controllo dell'oggetto *metro*.

6.2.1 Scrivere con tabwrite

Ipotizziamo che, a sinistra, *toggle* era stato posto sullo stato 1, ossia acceso, da una precedente spunta con il mouse. Ora il nuovo stato, dopo l'intervento di *moses*, sarà 0, quanto basta per interrompere *metro*. Per conseguenza anche il contatore si arresterà. Sempre sulla sinistra osserviamo l'oggetto **random** che emette una serie di numeri (pseudo)casuali; in basso vediamo infine l'oggetto **tabwrite** che, come dice il nome, scrive sulla tabella dal nome 'tabella' (il suo argomento), esattamente quella che abbiamo appena creato.

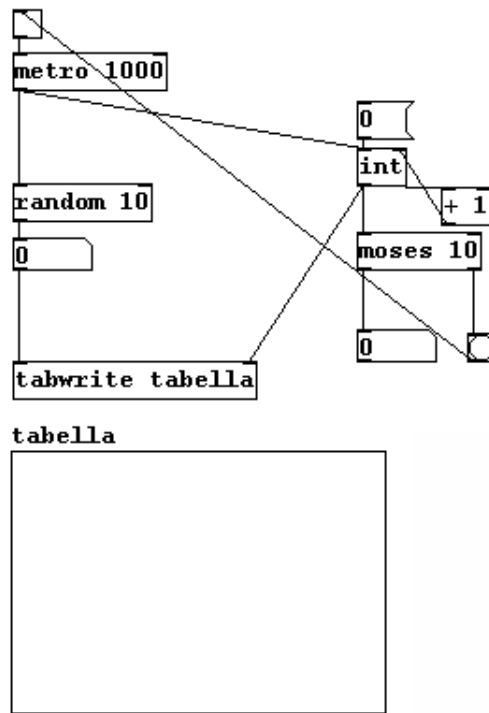


Figura 6.2:

Come funziona *tabwrite*? Si notano due inlet: quello a destra riceve gli indici, quello a sinistra i valori per ciascun indice. Nella patch gli indici sono i numeri da 0 a 9 prodotti dalla sezione contatore. I dati sono invece prodotti dall'oggetto *random*. Se i dati sono, nell'ordine, 2 - 4 - 6 si avranno le coppie, rispettivamente [0 - 2], [1, 4], [2, 6].

6.2.2 Leggere con *tabread*

Scrivere serve a poco se non predisponiamo, nel contempo, un sistema di lettura dei dati inseriti nella nostra tabella. All'upò ci serviamo dell'oggetto **tabread** il cui funzionamento è semplicissimo. Per prima cosa è necessario dichiarare la tabella che andrà a leggere. Ciò può essere fatto sia all'atto della creazione della tabella con un argomento che, successivamente, in modo dinamico tramite un messaggio *set* da inviare all'inlet dell'oggetto. In questo unico inlet *tabread* si aspetta di ricevere, di norma, i numeri indice. In uscita riceveremo i valori associati a quell'indice.

E' un'occasione per introdurre con una nuova patch gli oggetti **radio** (fig. 6.3), presenti nel menù a discesa *Put*, in posizione semicentrale. Si tratta di oggetti grafici con cui si può facilmente selezionare un valore cliccando su quadretti 'attivi'. Possono essere orizzontali o verticali (rispettivamente **Hradio** e **Vradio**. Nel nostro esempio abbiamo usato *Hradio*). La dimensione dell'oggetto sarà impostata accedendo all'area *Proprietà*, cliccando col pulsante destro sull'oggetto.

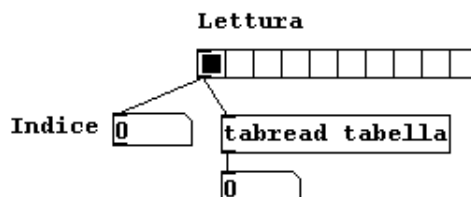


Figura 6.3:

6.2.3 Azzeramento della tabella

Per rendere più pratica la nostra patch aggiungiamo un ultimo pezzo: una routine che provvede ad azzerare la tabella predisponendola per un ulteriore utilizzo. E' quanto si vede (lato sinistro) in fig. 6.4 che ripropone una parte della patch completa.

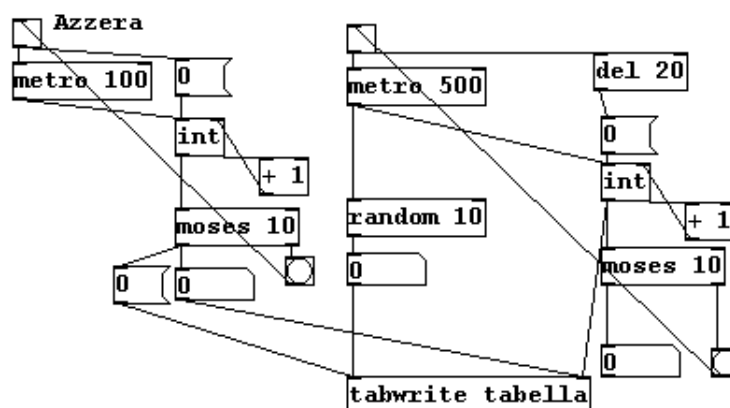


Figura 6.4:

Riconosciamo, innanzitutto, un altro contatore anche questa volta connesso a *moses* (di argomento 10). Attivando il processo (tramite *toggle*, in alto a sinistra) l'oggetto *metro* entra in funzione sparando 1 bang ogni 0.1 secondi. In uscita da *moses*, a sinistra, si attiva un box messaggio che manda in sequenza altrettanti zero inviati all'inlet sinistro di *tabwrite*; nel contempo un oggetto numerico incrementa il valore in uscita da *moses* inviando all'inlet di destra di *tabwrite* numeri da 0 a 9. In tal modo si avranno tante coppie [0, 0], [1, 0], [2, 0], ecc¹. per tutte le posizioni della tabella. L'oggetto ora è 'vuoto' e pronto ad un successivo utilizzo.

¹Ricordiamo che gli indici sono il *secondo* numero della coppia, mentre il valore corrispondente a quell'indice è il *primo*.

6.3 Per maggiori precisioni: `tabread4`

Come si è visto, *tabread* si limita a leggere i valori discreti memorizzati nell'array, *discreti* nel senso che vi sarà un salto dall'uno all'altro (ad esempio: 4, 7, 11, 5, ecc.). Un oggetto più sofisticato è **`tabread4`** che non si limita a leggere i valori corrispondenti agli indici inviati al suo inlet sinistro, ma effettua un'interpolazione tra ciascuna coppia di valori contigui a seguito di un incremento 'frazionale' dell'indice. Cosa si intende per incremento frazionale? Innanzitutto va ricordato che ponendoci sul box numerico e tenendo premuto il tasto *shift* i numeri si muoveranno non più per interi, ma per parti decimali. Dunque se, per fare un esempio, all'indice 0 corrisponde 2 e all'indice 1 corrisponde 7, passando da 0 ad 1 in modo 'frazionale' (ad es. 0.52, 0.53, 0.54, ecc.) si avrà un'opportuna interpolazione tra 2 e 7, transitando per valori intermedi.

6.4 Selezionare fra varie tabelle

Consideriamo di avere più di una tabella nella nostra patch. In questo caso sarebbe una inutile ridondanza prevedere un oggetto di lettura (e scrittura) per ciascuna tabella. Possiamo invece usare un solo oggetto semplicemente informandolo che deve riferirsi ad una certa tabella. A tale scopo si può inviare all'oggetto un messaggio con la parola *select* seguita dal *nome della tabella* (che, ovviamente, deve già essere stata creata). Nell'esempio (fig. 6.5) abbiamo lasciato solo le parti che ci interessano con i due box messaggio che inviano a *tabread* - rispettivamente - il nome di *tabella* e *tabella2*. Non importa, naturalmente, che l'oggetto *tabread* contenga già un argomento (*tabella*). Il messaggio lo sovrascriverà.

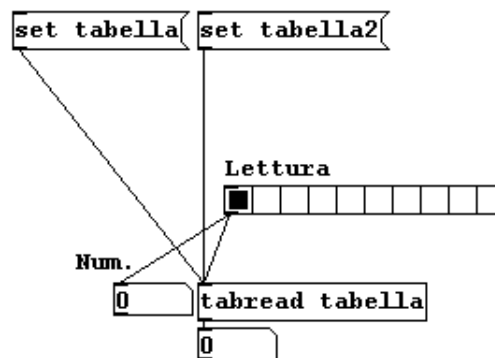


Figura 6.5:

6.5 Inviare messaggi alle tabelle

Il sistema prima usato per caricare la tabella può non risultare sempre comodo. Immaginiamo di sapere quali valori dovrà assumere la nostra tabella. Come glieli trasmettiamo? In PD le comunicazioni si fanno tramite i box messaggio. Per impostare un mittente occorre per prima cosa che il box inizi con un 'punto e virgola': ciò dice a PD che si sta per indirizzare un ordine ad un certo destinatario. Al secondo posto dobbiamo dunque inserire il nome del destinatario. Quindi gli argomenti.

6.5.1 Carichiamo una tabella

Per caricare una tabella (dopo averla creata e dimensionata) basta inviare un messaggio come in fig. 6.6.

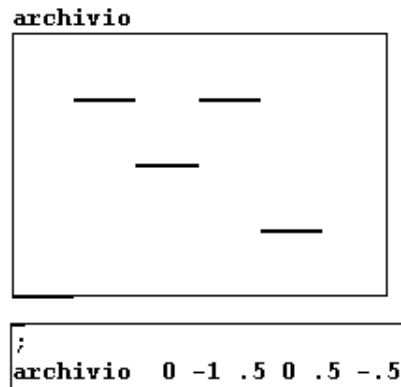


Figura 6.6:

Ossia: alla tabella *archivio* inviamo 5 valori per altrettanti 5 indici. Come si legge il messaggio? Lo 0 iniziale significa che il caricamento avverrà a partire dal primo indice (si parte sempre da 0, quindi il primo indice è 0). A tale indice corrisponde dunque il valore -1, all'indice 1 avremo 0.5, a 2 si avrà 0 e così via con 0.5 e -0.5.

6.5.2 Ridimensioniamo la tabella: bounds

Immaginiamo ora che si voglia, dopo averla già creata e impostata, ridimensionare la nostra tabella, ad esempio per renderla più capiente e capace di contenere valori di un più ampio range. Invece di rifare tutto posso inviare alla tabella un messaggio con la parola chiave *bounds*. Vedi fig. 6.7.

Cliccando sul box più in basso ridimensioniamo la tabella *archivio* ai seguenti valori:

la X va da 0 a 20; la Y va da 4 a -4. Come si vede l'asse X si è molto ampliato (prima era 10) e così pure l'asse verticale (con una scala totale di 9 punti).

6.5.3 Inserire una gradazione

Contenendo dei valori, la tabelle devono anche risultare facili alla lettura ad un primo sguardo. Così risultano utili dei sistemi di gradazione che orientino l'occhio indicandogli una scala di riferimento. PD presenta dei comandi che, inviati ad una tabella, ne fioriscono la visuale con un sistema di tacche spaziate. Anche in questo caso l'ordine andrà inviato alla tabella tramite un messaggio (fig. 6.8).

Le parole riservate sono in questo caso `<xticks>` e `<yticks>` seguite da 3 argomenti. Vediamoli nell'ordine:

1. a partire da quale punto inizia la graduazione. Nel nostro esempio dal punto 0;
2. quanto sarà larga la spaziatura tra le tacche. Nel nostro esempio, sull'asse X la distanza è di 1 punto, sull'asse Y di 0.2 punti;

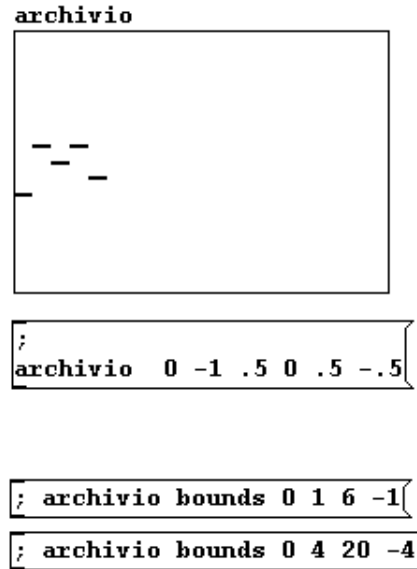


Figura 6.7:

3. si possono inserire tacche più grandi inframmezzate da tacche più piccole. Il terzo argomento indica appunto quante suddivisioni piccole vi saranno per ogni tacca grande. Nel nostro esempio: per la Y ogni 5 suddivisioni piccole vi sarà una tacca grande.

6.5.4 Numerare una tabella

Anche la numerazione di una tabella è molto utile e PD, come nel caso esaminato in precedenza, mette a disposizione due comandi, `<xlabel>` e `<ylabel>` seguiti da una serie di argomenti numerici.

In fig. 6.9 alla parola riservata *xlabel* seguono:

- -1.1: indica la localizzazione della numerazione rispetto alla tabella. In questo caso il numero negativo sta ad indicare che si trova in basso all'oggetto. Ciò va messo in relazione con l'impostazione delle coordinate del riquadro che - come si ricorderà - sull'asse Y vanno da 0 a 20.
- 0 1 2 3 ecc.. sono invece i numeri che compariranno (equispaziati) sull'asse X

Per quanto riguarda invece l'altro asse leggiamo l'impostazione di *ylabel*:

- -0.5 sta ad indicare che la numerazione si troverà a sinistra dell'asse Y;
- -1 0 1 indica invece i numeri che compariranno

6.6 Abbiamo studiato

Oggetti: **take**, **tabwrite**, **tabread**, **tabread4**

Messaggi: **set**, **select**, **bounds**, **xticks**, **yticks**, **xlabel**, **ylabel**

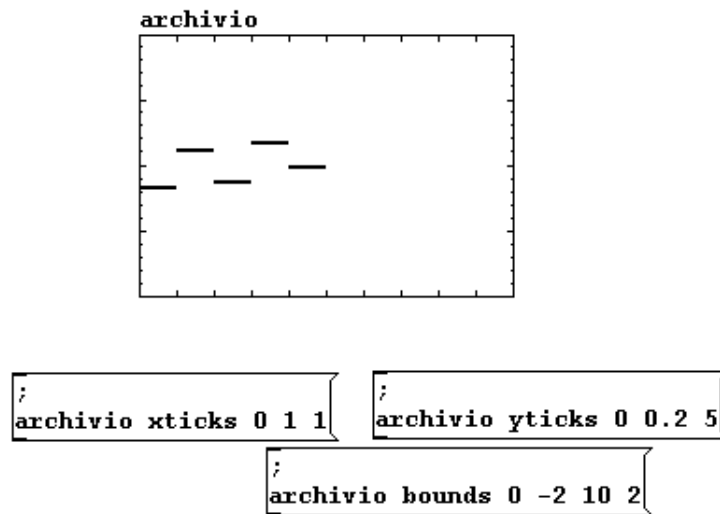


Figura 6.8:

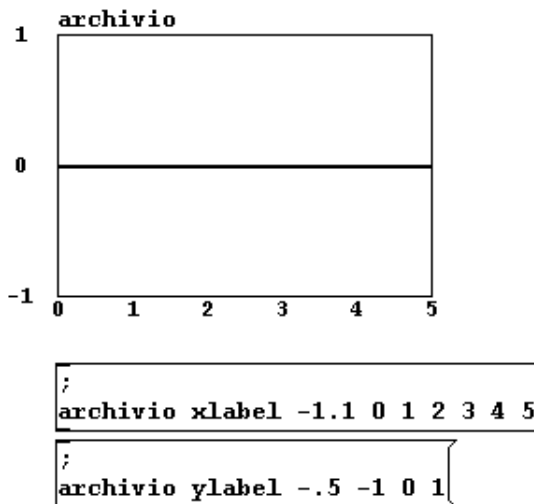


Figura 6.9:

Capitolo 7

MIDI

A dispetto di quanto qualcuno aveva preannunciato circa una precoce morte del MIDI si osserva che tutti i maggiori softwares musicali lo tengono tuttora in forte considerazione. E non è da meno PD che dispone di un buon numero di oggetti per la gestione dei comandi principali di questo fortunato protocollo. In verità chi ne volesse una gestione più approfondita dovrà ricorrere ad alcune librerie appositamente pensate, la più celebre delle quali è forse *Maxlib* di cui si parlerà più avanti nello spazio dedicato proprio alle librerie esterne.

Utile risulta, del MIDI, la possibilità di inviare messaggi di controllo per la gestione di macchine remote. Inoltre il caratteristico formato delle grandezze MIDI - che in prevalenza si muovono nel range 0 - 127 e in scala logartimica - finisce spesso col risultare assai più diretto e accessibile per il musicista che non la grandezza fisica vera e propria.

7.1 Un semplice strumento: uso di *makenote* e *noteout*

Proponiamo subito un esempio pratico. Usiamo PD per creare un semplice strumento MIDI cui potremo inviare il comando di suonare una certa nota, di una certa intensità, di una certa durata ed, eventualmente, con un certo timbro. Immaginiamo di avere un tastiera MIDI: secondo il protocollo, abbassando un tasto si invia il comando *Note-On* provvisto di numero di nota (per convenzione 60, equivale al Do centrale) e di un valore di *velocity* (in pratica la velocità con cui si affonda il tasto). Inoltre, per assegnare un timbro alla nota dovremmo dire alla scheda MIDI quale 'strumento' suonare ossia dirle quale *programma* usare, dove per 'programma' si intende una configurazione di parametri che definisce una certa identità sonora. Il comando all'uopo definito dal protocollo MIDI è *program change*. Infine, quando il tasto viene rilasciato, l'evento si traduce in un ulteriore messaggio, questa volta di *Note-Off*, ossia di fine attivazione del tasto. A quel punto il suono si fermerà.

In PD è l'oggetto **makenote** ad occuparsi di creare una coppia di dati MIDI del tipo *Note On* e *Note Off*. Qui occorre fare attenzione: *makenote* non si occupa semplicemente di 'far suonare' una nota MIDI, per la qual cosa potremmo usare più facilmente l'oggetto *noteout*. *Makenote* oltre a comandare un *Note On*, permette di programmare un *Note Off* dopo un certo lasso di tempo. Vediamo come funziona più da vicino.

Questo oggetto possiede 3 inlet per altrettante informazioni: 2 per i parametri minimi che il sistema si aspetta di ricevere (*numero nota* e *velocity*) più un terzo che riguarda la durata del suono (questo dato, usando una tastiera MIDI, si avrebbe con il rilascio del tasto). Dunque:

1. numero di nota (da 0 a 127, dove 60 è il Do centrale)

2. *velocity* (da 0 a 127)
3. la durata, in millisecondi

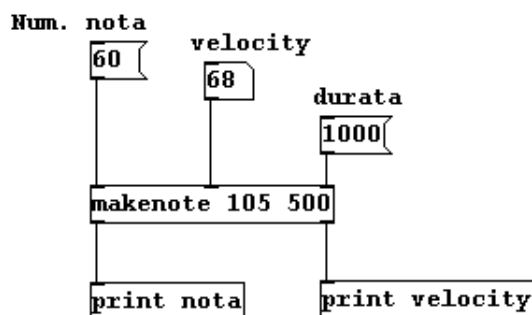


Figura 7.1:

Nel nostro esempio (fig. 7.1) *makenote* riceve 3 input ma, come si vede, l'oggetto può ospitare 2 argomenti, rispettivamente per *velocity* (a sinistra) e *durata* (a destra). Non avrebbe invece molto senso inserire come argomento anche la nota da suonare. Alle uscite due oggetti *print* ci permettono di monitorare il funzionamento di *makenote*. Non resta dunque che provarlo. Come?

Cliccando sul box messaggio più a sinistra, relativo al *Numero di nota*. Se non abbiamo modificato gli argomenti di *makefile* (ossia se non abbiamo attivato i box degli inlet 2 e 3) vedremo comparire sul terminale le seguenti liste:

velocity: 68

nota: 60

e, dopo mezzo secondo,

velocity: 0

nota: 60

PD ha inviato una coppia di messaggi di *velocity* e di *numero nota*, poi ha aspettato mezzo secondo (500 msec.) - la durata impostata della nota - dopodichè ha inviato ancora un messaggio di nota 60, ma questa volta con la velocity 0. Quest'ultima coppia di dati sintetizza (e sostituisce di fatto) il comando *Note Off*.

Agendo su *velocity* e *durata*, ossia modificando gli argomenti dell'oggetto *makefile*, non succede niente. Ciò vuol dire che abbiamo semplicemente impostato diversamente i parametri di una futura nota che verrà attivata solo agendo tramite l'inlet di sinistra (Num. nota).

A questo punto siamo pronti per produrre dei suoni MIDI. Basterà semplicemente indirizzare le uscite di *makenote* verso un altro oggetto, **noteout**.

Come risulta dalla fig. 7.2, *noteout* ha 3 inlet, rispettivamente per il *numero* di nota, *velocity* e per il *numero di canale* MIDI cui viene inviato il messaggio. In assenza vale il numero di default che è 1. *Noteout* può avere quest'ultimo dato come argomento, ma è facoltativo, valendo sempre il principio che in assenza di informazioni il valore di default è 1.

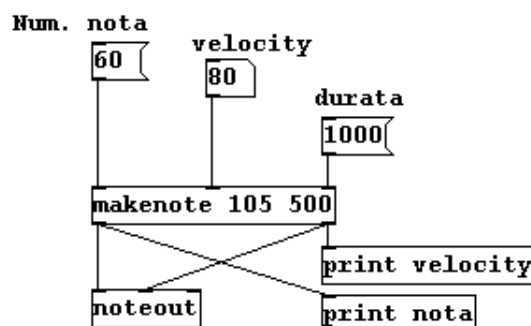


Figura 7.2:

Riprendendo un concetto espresso prima, *noteout* non deve necessariamente ricevere i suoi dati da *makenote*, come nel nostro esempio. Si può benissimo inserire dati con messaggi o box numerici ai suoi inlet. In sostanza il compito di *noteout* è scrivere un messaggio MIDI di *Note On* sulla porta di uscita. Ma nulla dice su quanto deve durare quella nota.

Va fatta un'ultima precisazione sull'inlet di sinistra che si aspetta di ricevere o un singolo *numero* o una *lista* (di numeri). Nel primo caso (*un* numero) si inteso come *numero di nota*; nel secondo caso (una lista di numeri) sarà letta come una lista dei tre valori, di cui si è detto, nell'ordine: *numero nota*, *velocity*, *numero di canale* (quest'ultimo è facoltativo).

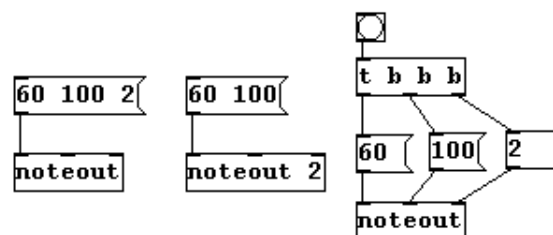


Figura 7.3:

In tal senso i tre casi indicati in figura sono del tutto equivalenti.

7.1.1 Digressione sulla notifica MIDI

Può essere utile una breve digressione sulla codifica MIDI per essere più consapevoli dei dati intorno ai quali PD lavora. Il linguaggio MIDI stabilisce che l'invio di informazioni avvenga tramite blocchi di bit (8 bit = 1 byte). Il tutto avviene attraverso un rigido protocollo: prima viene inviato un *byte di stato* che ha il compito di avvisare il sistema del comando che sta per essere inoltrato (e infatti si chiama anche byte di *comando*), quindi seguono, a seconda dei casi, i dati relativi a quel comando, con altri pacchetti di bit.

La specifica stabilisce che i *messaggi di stato* inizino con 1, il che vuol dire che, in un complesso di 8 bit, si parte dal numero decimale 128 (in binario 1000 0000) fino a 255 (in binario 1111 1111). Invece i byte di dati devono mandare numeri compresi tra 0 (0000 0000) e 127

(0111 1111). In tal modo si comprende come una macchina MIDI riconosca immediatamente la tipologia dei byte che la raggiungono.

Vediamo ora cosa succede quando si suona una nota su una tastiera MIDI. Abbiamo l'invio di 3 byte, dei quali il primo, come si è detto, è quello di stato (o di comando), gli altri 2 di dati. Ecco il modello:

1xxx nnnn 0ttt tttt 0vvv vvvv

Esaminiamo ciascun byte:

1xxx nnnn Riconosciamo l'1 iniziale del *byte di stato*. Il semibyte di sinistra continua con tre numeri binari che stabiliscono il **comando**. Nel caso di Nota On è il numero 9, quindi in binario 1001. Il semibyte di destra (nnnn) chiarisce qual è il **canale MIDI** che dovrà rispondere al messaggio. Se, ad esempio, vogliamo il canale MIDI 5, sarà: 0011. Dunque il messaggio relativo al primo byte diventa: 1001 0011

0ttt tttt Il secondo byte, come pure il terzo, riguarda i dati. Nel messaggio che stiamo studiando il secondo byte si riferisce al **numero di tasto** abbassato, ossia alla Nota che si vuole eseguire. Consideriamo di aver suonato il Mi4 (sopra il Do centrale): espresso numericamente esso corrisponde a 64, ossia (in binario) 0010 0000.

0vvv vvvv Infine il terzo byte, che nel messaggio del nostro esempio si riferisce alla **velocità** (*velocity*) con cui si è abbassato il tasto. Si va ovviamente da 0 che è il minimo di forza, il che vuol dire 'niente nota', a 127 che è il massimo di intensità. Consideriamo una *velocity* di 100, in binario: 0011 0010.

In definitiva il Messaggio di *Nota On* relativo ad una nota Mi4 di intensità medio-alta inviato al canale 5 sarà:

1001 0011 0010 0000 0011 0010

Il messaggio di **Note Off** è simile al precedente con la sola differenza che il primo semibyte a sinistra invece di essere 9 è 8. Per il resto è tutto uguale: sia il secondo byte (numero tasto) che il terzo (*velocity*). In quest'ultimo caso la *velocity* indica con quale velocità è stato rilasciato il tasto. Tuttavia, come si è detto, è invalsa l'abitudine di sostituire il messaggio di *Note Off* con uno di *Note On* in cui la *velocity* è impostata a 0. L'utilità consiste in un risparmio nel flusso di informazioni perchè vi è la convenzione che se ad un messaggio ne segue un altro dello stesso tipo (come è il caso di 2 o più *Note On* di seguito) allora non è necessario trasmettere il byte di Stato, ma solo i dati. Nel nostro caso, non trasmettere il byte di stato su pacchetti di tre, comporta un risparmio di un terzo del traffico totale. Il che non è poco.

7.2 Suonare da tastiera: *notein* e *stripnote*

Se abbiamo configurato in modo corretto il MIDI nel nostro sistema possiamo provare a suonare da tastiera esterna collegata all'interfaccia MIDI e fare leggere l'operazione a PD. Quindi, per poter ascoltare, mandiamo i dati captati da PD alla porta MIDI in uscita, tramite *noteout*. Per fare questo ci serviamo dell'oggetto **notein** che fa proprio questo: legge le note MIDI in ingresso indicando nell'ordine (da sinistra a destra)

- numero di nota,
- velocity e
- numero di canale MIDI.

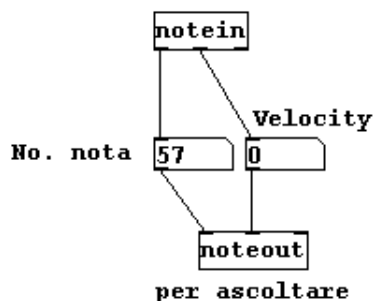


Figura 7.4:

Ha dunque 3 uscite e nessun ingresso, perchè direttamente collegato al sistema MIDI.

Realizziamo una semplice patch che utilizza *notein* (fig. 7.4): il box numerico di destra, che indica la *Velocity*, riporta uno zero quando è rilasciato. E' la riprova di quanto si è detto prima circa la sostituzione del messaggio di *Note Off* con *Note On* a *velocity* 0. Nell'esempio non abbiamo collegato nulla all'outlet più a destra di *notein* (che riportebbe il canale MIDI di trasmissione). Va precisato che l'oggetto *notein* può assumere un argomento al suo interno per dichiarare il canale dal quale legge il flusso MIDI. In assenza, si mette in ascolto di *tutti* i canali.

Talvolta - ad esempio per monitorare le note suonate, come si vedrà nel capitolo seguente - può essere utile fare uscire da *notein* solo il dato *Note On* filtrando il successivo *Note Off*. Di questo si occupa l'oggetto **stripnote** che, come dice il prefisso inglese *strip*, indica che la nota viene 'spogliata' e 'alleggerita' di qualcosa, ossia di una componente della sua codifica completa. Proviamo l'oggetto con la patch precedente (fig. 7.5).

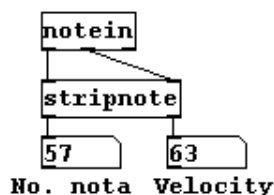


Figura 7.5:

Abbiamo interposto *stripnote* tra *notein* e i box numerici che rivelano i tasti utilizzati nonché la rispettiva *velocity*. Se ora rilasciamo il tasto non accade più, come prima, che la *velocity* volge a 0, o meglio, questo dato non viene più rilevato, appunto perchè filtrato da *stripnote*.

7.3 Cambiamo suono: *pgmout*

Altri comandi MIDI possono essere inviati tramite PD, ad esempio il classico **Program Change**. Ricordiamo che nel linguaggio MIDI per 'programma' si intende un particolare timbro, ovvero un intero set di configurazione che produce un particolare suono. Impostata

una patch possiamo divertirci a cambiare timbro usando l'oggetto **pgmout**. Esso consta di 2 inlet e nessuna uscita (in quanto manda direttamente alla porta MIDI). E più precisamente:

- l'inlet di sinistra riceve un intero che definisce il numero di *Program Change*
- l'inlet di destra (sempre come intero) serve a definire il canale MIDI destinatario del messaggio

Quest'ultimo dato può anche essere incluso nell'oggetto come suo proprio argomento.

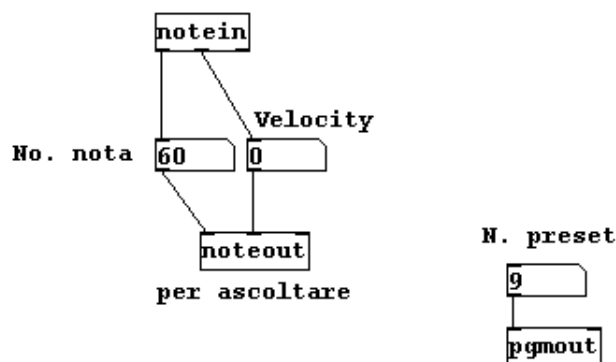


Figura 7.6:

Nella nostra semplice applicazione (fig. 7.6) vediamo che l'oggetto *pgmout* non è connesso al corpo principale della patch, quello che invia il comando di suonare. E' messo a lato, con un semplice box numerico per variare il preset che offrirà il timbro alla nota generata.

Il corrispettivo di *pgmout* è **pgmin** che invece legge i messaggi di *Program Change* inviati alla porta MIDI (ad esempio dalla nostra tastiera). Proprio per questo consta di soli outlet, 2 per la precisione: quello di sinistra riporta il numero di *Program Change*, quello di destra il numero del canale MIDI. Anche in questo caso l'argomento, che è opzionale, può includere il numero di canale. Va detto che qualora l'oggetto venga creato con un argomento, automaticamente vi sarà un solo outlet piuttosto che 2 (essendo il secondo inutile).

7.4 Il *Control Change*

Per *Control Change* intendiamo quei messaggi MIDI che servono a comunicare cambiamenti in alcuni parametri dei dispositivi musicali MIDI, come il Pan (ossia l'immagine stereo), il volume, l'eco, ecc. Va detto, peraltro, che rientrano in questa categoria di messaggi anche quelli che dispongono i dispositivi nei cosiddetti 4 modi, *OmniOn/Poly*, *OmniOn/Mono*, *OmniOff/Poly*, *OmniOff/Mono*.

Può essere utile ricordare, in breve, a cosa corrispondono questi 4 modi. Una certa voce di un dispositivo MIDI è in modo *Omni* (ossia *OmniOn*) se risponde a tutti i messaggi in arrivo, indipendentemente dal canale di trasmissione. Viceversa, ossia se è in *OmniOff*, risponderà solo ad un particolare canale MIDI e sarà insensibile agli altri. Invece il modo *Poly* significa che un certo canale MIDI potrà essere destinatario di più voci. Ad esempio al canale 4 posso mandare un timbro di clarinetto, di pianoforte, di vibrafono, contemporaneamente (usando un sequencer, ad esempio). Se è in modo *Mono* questo non è possibile: ogni canale riceverà solo una voce e non di più.

Per riprendere un esempio di messaggio MIDI visto prima relativamente al *Note On*, il messaggio di *Control Change* si avvale di un byte di Stato nella forma di 1001, seguito da due byte di dati, il primo dei quali si riferisce al numero di controllo MIDI, il secondo al valore di quel controllo.

Sebbene il protocollo assegni 128 possibili controller, di fatto solo alcuni hanno una codifica standard. Altri non sono usati o possono essere assegnati liberamente. Il più noto è forse quello che risponde al numero 7 e riguarda il volume. Faremo più avanti un esempio usando questo controller. Un altro numero standard è l'1 cui si riferisce la rotella del Mod Wheel.

In PD esiste una coppia di oggetti - *ctlin* e *ctlout* - per la gestione di questa tipologia di messaggi.

ctlin l'oggetto **ctlin** legge i messaggi di *control change* in arrivo. Lo fa direttamente dalla porta MIDI, quindi non dispone di alcun ingresso. Se è usato senza argomenti *ctlin* verrà istanziato con tre uscite, rispettivamente da sinistra a destra:

- valore di controllo
- numero di controller
- numero di canale

Ma l'oggetto può essere creato anche con uno o due argomenti. Se è solo uno, indica il numero di controllo da processare. In tal caso l'oggetto disporrà di 2 sole uscite. Se invece si ha un secondo argomento - e allora l'oggetto disporrà di un solo outlet - esso indica il canale MIDI sul quale verrà processato il controller. In assenza di argomenti (o anche di uno solo di essi) vale il principio che l'oggetto legge su tutti i canali.

ctlout il corrispettivo di *ctlin* è **ctlout**: il suo compito è inviare messaggi di *control change* alla porta MIDI. Se è senza argomenti presenta 3 ingressi, rispettivamente (da sinistra a destra):

- valore di controllo
- numero di controller
- numero di canale

Medesima a *ctlin* è la situazione degli argomenti. Tuttavia, tattandosi di ingressi, l'oggetto accetta anche una lista di numeri (2 o anche 3), da intendersi sempre nell'ordine su riportato. I numeri della lista *non* vanno separati da virgole, ma da un semplice spazio.

Proponiamo un semplice esempio (fig. 7.7).

La parte sinistra della patch invia un bang ogni mezzo secondo tramite l'oggetto *metro*. Viene, per conseguenza, spedita a *noteout* una lista di 2 numeri indicanti un numero di nota MIDI (il Do centrale) con una particolare *velocity* (100). Nella sezione destra, invece, possiamo agire tramite l'oggetto *ctlout* per inviare al dispositivo MIDI cambiamenti che interessano il *Pitch Bend* e il Volume. Clicchiamo sul box messaggio '1' per poi passare a scorrere i numeri sul box numerico 'Valore': sentiremo la nota acquistare (o perdere) un progressivo vibrato. Cliccando invece sul box 'Volume', e scorrendo i numeri del box 'Valore', sentiremo modificarsi il volume del suono (più forte / più piano) in corrispondenza dei valori impostati. Ricordiamo comunque che i valori devono oscillare tra 0 e 127, almeno secondo la codifica MIDI.

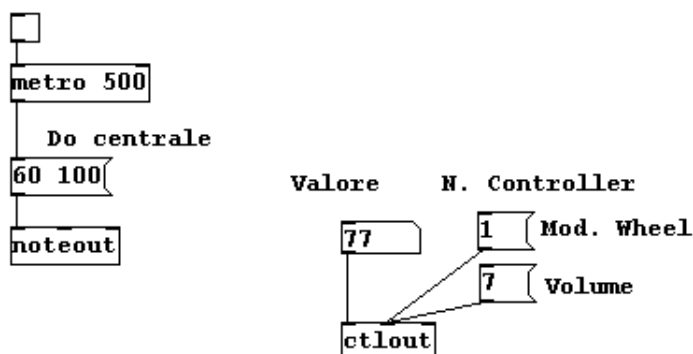


Figura 7.7:

7.5 Piccoli incrementi: *bendin* e *bendout*

Tra i messaggi di controllo, nel linguaggio MIDI, un posto particolare spetta ai messaggi che inviano messaggi - per così dire - *continui*. Dire 'continuo' non è molto corretto (si rimane infatti pur sempre nell'ambito di una variazione 'a gradini'), ma è tuttavia un'espressione pratica per specificare che le variazioni avvengono con una risoluzione assai maggiore di quanto non sia l'ambito da 0 a 127 cui si è accennato per i precedenti messaggi. Il *Pitch Bend* è appunto uno di questi messaggi. Esso serve a variare in modo (per quanto possibile) continuo l'intonazione delle note. Per ottenere questo, entrambi i 2 byte di dati sono assegnati alla definizione delle variazioni. Pertanto, considerato che lo zero rimane il numero fisso iniziale di ciascun byte, si avranno 7 più 7 bit, per un totale di 14 bit; il che porta ad avere un range di oscillazione nell'ordine di 2 elevato a 14, vale a dire 16384 possibilità.

Vano fatte due osservazioni. La prima è che queste oscillazioni variano in base all'impostazione del dispositivo ricevente. Ad esempio possiamo impostare che queste oltre sedicimila combinazioni si riferiscano ad un ambito di +3 e -3, ossia tra una terza minore sotto, e una terza minore sopra, oppure si riferiscano ad un ambito di +8 e -8. La risoluzione sarà ben diversa.

La seconda osservazione attiene al canale MIDI di riferimento che riguarda tutte le note. Non si può cioè assegnare una variazione di *Pitch Bend* ad una nota ed un'altra variazione ad un'altra nota. L'unica via d'uscita è assegnare a note differenti altrettanto differenti canali MIDI riceventi.

In PD esiste la coppia *bendin* e *bendout*.

bendin legge i valori di *pitch bend* in arrivo e consta di 2 uscite. In quella di sinistra possiamo leggere i valori di *pitch bend*, in quella di destra il canale MIDI dal quale legge. Può avere un argomento che riguarda proprio il canale MIDI. In assenza di argomento, in uscita sull'outlet destro compare il canale in quel momento ricevente. In presenza di argomento l'oggetto leggerà unicamente i dati da quel canale.

bendout Viceversa *bendout* manda le variazioni di *pitch bend* alla porta MIDI, quindi la nota in quel momento attiva verrà effettuata da questo controllo.

Come si vede in fig. 7.8, *bendout* presenta due ingressi. Quello di sinistra permette di inserire il valore di *pitch bend*, quello di destra il canale MIDI su cui andrà ad agire. In assenza di argomento vale di default il canale 1.

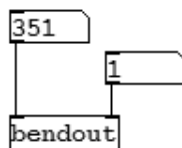


Figura 7.8:

Proviamo una patch in cui si possa modificare a piacere il *pitch bend*, usando il mouse, esattamente come si potrebbe fare tramite una controller MIDI esterno (ad es. la rotella di una tastiera).

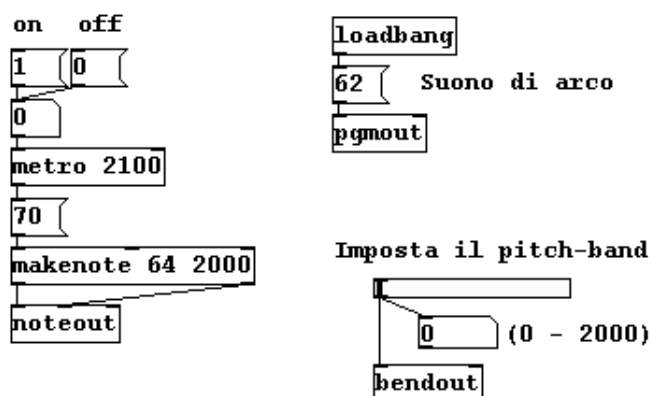


Figura 7.9:

In fig. 7.9 la parte sinistra produce, ad intervalli di 2,1 secondi, una nota della durata di 2 secondi. Sulla destra abbiamo due sezioni. In alto un oggetto **loadbang** provvede a mandare un bang al momento del caricamento della patch in modo che venga impostato il preset MIDI numero 62, che corrisponde ad uno strumento ad arco. In basso possiamo sperimentare l'oggetto *bendout* spostando a destra e sinistra il cursore dell'oggetto grafico *Hslider*. Ascolteremo la nota variare in altezza lungo un range da 0 a 2000. L'inlet di destra non ha alcun valore, quindi si intende il canale MIDI numero 1.

7.6 Altri oggetti MIDI

Per ragioni di spazio non tratteremo tutti gli oggetti MIDI disponibili in PD, ma il lettore potrà facilmente capirne il funzionamento sulla base di quanto detto nelle precedenti sezioni di questo capitolo. Un discorso a parte meritano gli oggetti **midiin**, **midiout** e **sysexin** in quanto attualmente funzionano solo sotto Linux. I primi due, rispettivamente, ricevono in ingresso e mandano in uscita dati MIDI (porta a sinistra) e, nell'altra porta, il numero della porta MIDI.

In ogni caso gli oggetti implementati nella versione base di PD, per quanto numerosi, sono ancora insufficienti per le effettive esigenze di lavoro in ambiente MIDI. Si rivelano dunque

assai comode le librerie scritte da altri programmatori (dette *externals*) con un gran numero di altri oggetti. Ci soffermeremo, nel prossimo capitolo, ad illustrare due importanti librerie, una chiamata *maxlib* e l'altra *xeq*. La prima è assai ricca di oggetti MIDI dalle più svariate funzioni, la seconda, scritta da Krzysztof Czaja, presenta alcuni interessanti oggetti che permettono di implementare in PD le proprietà di un sequencer.

7.7 Abbiamo studiato

makenote, notein, noteout, stripnote, pgmin, pgmout, ctlin, ctlout, bendin, bendout, midiin, midiout, sysexin.

Capitolo 8

Caricare librerie esterne

Per librerie esterne si intendono parti di programma, sorta di *plug-in*, che espandono di nuovi oggetti la struttura base di PD. In ciò risiede anche la forza di questa applicazione che è poi quella tipica del mondo Open Source. Tutti possono contribuire a scrivere *externals* da 'agganciare' al programma madre e in rete si trova anche un comodo tutorial che guida e aiuta i programmatori disposti a farlo. Per chi è interessato l'indirizzo è:

<http://iem.at/pd/externals-HOWTO/>

Poichè vogliamo continuare ad occuparci di MIDI, tra i numerosi externals presentiamo *maxlib*, scritto da Olaf Matthes, un pacchetto ricco di svariati e comodissimi oggetti per il mondo MIDI e *xeq*, di Krzysztof Czaja, che contiene pochi ma utilissimi oggetti per la gestione dei flussi MIDI.

Infine faremo un breve accenno (breve rispetto alla vastità degli oggetti che offre) ad una libreria che si occupa invece di audio, dal nome *iemlib*, scritta da Thomas Musil e giunta, al momento in cui scriviamo, alla Release 1.17. Contiene un enorme numero di filtri, vari oggetti audio nonché altri di natura grafica che migliorano possibilità già presenti nella versione base di PD.

Ma andiamo con ordine.

8.1 Istallare *maxlib*

Possiamo scaricare la libreria al seguente indirizzo:

<http://pure-data.sourceforge.net/download.php>

Nella sezione contrassegnata da *pd-externals* troviamo un applicativo solo per Windows (con estensione .zip) ed un altro generico, ma è quello che ci serve per instrallarlo in ambiente Linux, con estensione .tar.gz. Ci occuperemo brevemente della procedura di installazione sotto Linux.

Dopo aver scaricato e scompattato il pacchetto degli externals entrare nella cartella dove sono i files. Nel mio sistema si chiama:

pd-externals-20030311 (ma può avere anche un altro nome).

Dentro la cartella si trovano altre sottocartelle con i vari externals. Entrare in quella chiamata 'maxlib'. Aprire a questo livello una shell e compilare i files sorgenti con il comando:

`"make pd_linux"`

La cartella verrà riempita da altri files. Quando il processo sarà finito digitare, sempre da shell:

`"make install"`.

Prendere nota, alla fine del breve listato che compare nella shell, dell'esatto percorso dove sono stati copiati i files. In particolare quelli dell'help. Infatti, nelle varie distribuzioni di Linux, i files base di PD non vengono scritti sempre nella medesima posizione del filesystem, sicchè i files dei vari *externals* possono non risultare presenti nelle locazioni dove il programma va a cercarli. Se vi è discordanza, ossia se una volta lanciato correttamente il programma, gli help non vengono caricati, (ci stiamo infatti riferendo alla documentazione degli help) è sufficiente copiare la cartella 'maxlib' entro quella principale di PD. Nel mio sistema è:

```
/usr/bin/pd/doc/5.reference
```

A questo punto maxlib è stato installato. Basterà lanciare PD con l'aggiunta del seguente script:

```
-lib maxlib
```

8.2 Alcuni oggetti di maxlib: *borax*, *pitch* e *chord*

Dei tanti oggetti di *maxlib* faremo, per ragioni di spazio, un breve riferimento solo ad alcuni. Molti di questi oggetti sono simili (talvolta identici anche nel nome) a quelli per Max/MSP. In particolare prenderemo in considerazione queglii oggetti che si rivelano utili a fini di analisi degli eventi MIDI in ingresso.

8.2.1 Borax

Uno di questi è **borax**, definito un 'coltellino svizzero' per la sua versatilità nel dividere in tante informazioni ciò che, esteriormente, può sembrare una semplice nota MIDI. Ma vediamo come funziona.

Creando in una patch vuota un oggetto *borax* ci si accorge subito che la parte inferiore è tutta in grassetto e non sono visibili gli outlet.

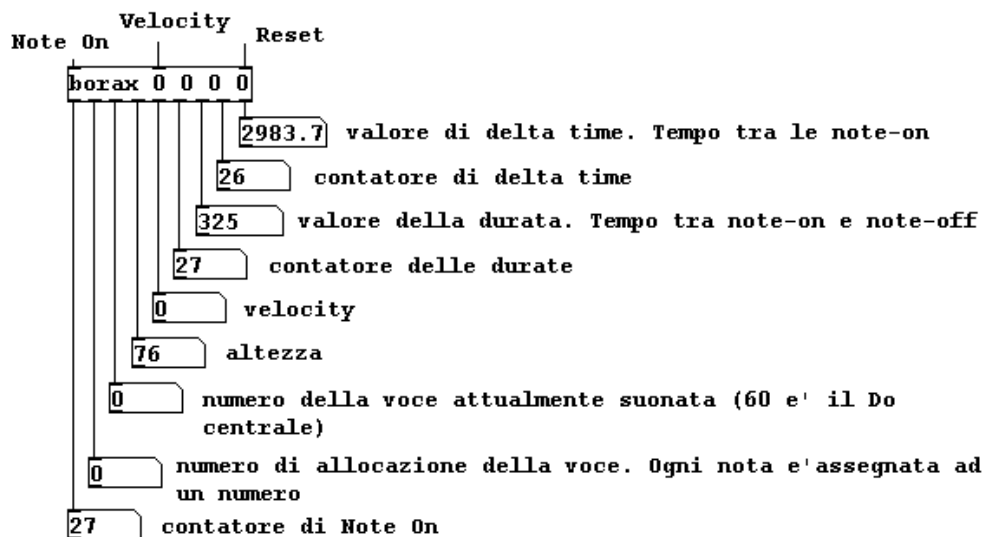


Figura 8.1:

Curiosamente, per allargare la dimensione grafica dell'oggetto si può ricorrere ad un semplice ma efficace stratagemma: aggiungere degli 'zero' come argomento (questi 'zero' non hanno alcun significato se non questo). Conviene mettere almeno 4 o, meglio ancora, 5 zero. Ora vediamo distintamente i numerosi outlet, 9 in tutto. Gli inlet sono invece 3. Più in dettaglio, da sinistra a destra:

Inlet

- messaggio di Note On,
- velocity,
- riceve un bang con funzione di reset mandando tutto a zero e inviando un messaggio di Note Off alle note attualmente in play.

Outlet Come detto, sono in tutto 9, come indica la figura 8.1. E' evidente che questo strumento si rivela utilissimo quando si vogliono effettuare rilievi analitici su brani di cui disponiamo di una trascrizione in formato MIDI. Ad esempio, prendiamo un brano in formato MIDI file, lo facciamo leggere a PD e, contestualmente, inviamo a *borax* i dati in lettura in modo che li analizzi.

8.2.2 Pitch

Ma vediamo qualche altro oggetto di interesse, ad esempio l'oggetto **pitch** che analizza le note MIDI in ingresso fornendone 5 livelli di dettaglio. Più in particolare *pitch* manda in uscita (da sinistra verso destra):

- numero di nota MIDI
- nome simbolico (alfanumerico) della nota. Ad es.: C4
- *pitch class* della nota, ossia il numero che identifica la classe di appartenenza di una nota in una scala da 0 ad 11, indipendentemente dalla sua ottava reale. Ad esempio il Do è una nota di classe 0, il Si di classe 11, il Sol di classe 7 e così via.
- l'intervallo rispetto all'ultima nota. Nel caso di intervalli discendenti i valori saranno negativi
- il registro, considerando il 4 come ottava centrale.

L'oggetto *pitch* dovrebbe ricevere un argomento, se non altro per poter consentire il calcolo richiesto dal quarto outlet (intervallo rispetto all'ultima nota). Si può mettere, ad esempio, 60. In ingresso, invece, *pitch* riceve il messaggio di nota (a sinistra) e la relativa velocity (a destra).

8.2.3 Chord

Come dice il suo nome, **chord** manda in uscita la sigla (dato *symbol*) di un intero accordo riconoscendolo tra le note che riceve in ingresso. Vediamo in figura 8.2 come lavora:

L'argomento serve ad indicare la soglia oltre la quale il programma non prende in considerazione le note in ingresso. Ciò può essere utile nel caso tipico in cui gli accordi sono la parte più grave di una testura di melodia con accompagnamento. Inoltre sappiamo che la nota

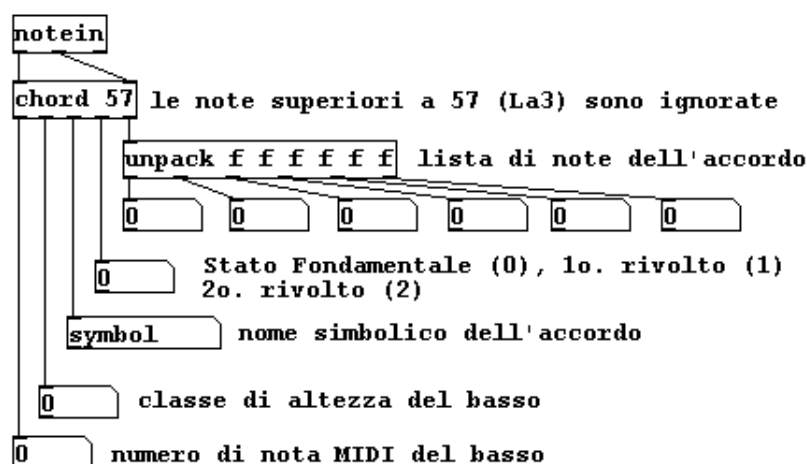


Figura 8.2:

più grave di un accordo ha un'importanza analitica maggiore, per questo il riconoscimento di questa altezza viene messo in primo piano (dai primi due outlet). Si noterà che l'oggetto è programmato ad emettere dati di analisi in funzione degli aspetti più tipici dell'armonia tradizionale come la nota del basso e stato dell'accordo (quarto outlet).

Va infine detto che l'outlet più a destra emette una lista di dati che è da scompattare usando l'oggetto *unpack*. E' ovvio che metteremo come argomento ad *unpack* tante *f* (ricordiamo che *f* sta per *float*) quante sono, presumibilmente, le note di cui si compongono i nostri accordi.

A questo punto prima di procedere con una esercitazione è il caso di presentare un'altra libreria di oggetti. Ciò è dovuto al fatto che il pacchetto standard di PD non dispone di un lettore di files in formato MIDI e nemmeno l'external *maxlib* ci è d'aiuto in tal senso. Invece questa possibilità è un'opzione davvero indispensabile per rendere davvero utili le numerose risorse di *maxlib*. Per fortuna esiste all'uopo un altro pacchetto di externals, *xeq*, scritto da Krzysztof Czaja.

8.3 Installare *xeq*

Xeq è una libreria di *externals* per la manipolazione di sequenze temporali. Ai nostri fini ne metteremo in evidenza l'utilità sul piano del controllo del flusso di dati MIDI in lettura. In pratica lo useremo come un sequencer. L'autore ha prodotto un interessante documento teorico che ne illustra le caratteristiche, scaricabile all'URL:

http://puredata.org/community/projects/convention04/lectures/tk_czaja/

Vediamo per prima cosa come installare il pacchetto. Connettersi all'indirizzo:

<http://suita.chopin.edu.pl/~czaja/miXed/externs/xeq.html>

e, dopo una breve descrizione, troviamo in fondo alla pagina due links: uno permette il download per una installazione su Linux, l'altro su Windows (riconoscibile perchè il file ha l'estensione .zip). Come sempre rivolgerò una maggiore attenzione alla prima opzione, se non altro perchè potrebbe dare qualche problema in più all'utente che non ha confidenza con l'universo Unix.

Dopo il download, il file è in formato *.tar.gz*, quindi occorre scompattarlo. Per fare questo, oltre al classico script da linea di comando, è possibile oggi usare le funzioni grafiche offerte sia da Gnome che da KDE. Basta cliccare col pulsante destro del mouse e scegliere l'opzione

Estrarre qui o simili. Verrà creata una cartella con il nome della versione di *xeq* che abbiamo scaricato. Per comodità la chiameremo *libXeq* (fare attenzione alle maiuscole). Dentro vi troviamo altre sottocartelle e alcuni files tra cui uno che ha nome *xeq.pd_linux*. Si tratta di un file precompilato verso cui va indirizzato lo script da usare al lancio di PD. Conviene dunque spostare questa cartella dentro quella generale di PD. In casi normali, come sul mio sistema, si avrà: */usr/lib/pd/libXeq*.

A questo punto tutto è pronto: basta ricordarsi di lanciare PD con la seguente aggiunta:

-lib /usr/lib/pd/libXeq/xeq

dove lo *xeq* finale è proprio il nome dell'*external*.

8.4 Esercitazione. Ipotesi di analisi.

Nell'illustrare alcuni dei tanti oggetti presenti negli externals *maxlib* e *xeq* ci siamo riferiti essenzialmente all'ipotesi di un possibile impiego in ambito di analisi musicale. Ciò che rientra sotto il nome di 'analisi assistita dall'elaboratore' riguarda sostanzialmente due grandi settori.

Uno è rivolto per lo più alla computazione di dati di macrolivello della partitura, come possono essere le note, gli accordi, le dinamiche, i segni di articolazione, il decorso temporale (ciò che musicalmente si chiama *agogica*), e così via. Sotto tale prospettiva la codifica MIDI si rivela di grande utilità. Ovviamente il computer si limita al rilevamento di dati grezzi richiesti dall'analista, e spetterà a quest'ultimo non solo impostare nella maniera migliore il tipo di dati su cui effettuare il rilevamento, ma anche, a posteriori, farne un uso intelligente, che porti a risultati analiticamente rilevanti. Per fare qualche esempio si potrebbe studiare in un brano tonale i criteri con cui, sotto il profilo stilistico, un compositore introduce le note estranee alla tonalità, sia rimanendo nel tono che modulando; oppure si potrebbe studiare l'uso di certe forme accordali, ad esempio l'incidenza dei rivolti, in luoghi particolari della forma musicale da parte di un compositore¹. Interessante è poi l'impiego di analisi computazionali che rilevino le trasformazioni testurali in funzione di particolari momenti della forma musicale. Ciò può essere fatto calcolando il grado di densità, e i relativi processi di crescita e rarefazione, di eventi sonori nell'unità di tempo, confrontando - ad esempio in un tempo di Sonata classica - la Transizione con un gruppo tematico o con lo Svolgimento². Non ultimo vanno menzionati gli approcci cosiddetti performativi all'analisi, laddove si confrontano più esecuzioni di un medesimo brano. I dati raccolti dal computer possono dunque essere messi a confronto per poi studiare, sotto il profilo musicologico, i criteri e le ragioni di particolari scelte esecutive.

Un secondo settore di analisi assistita dall'elaboratore riguarda invece un approccio ai dati di microlivello degli eventi sonori. Mentre nel caso precedente si rivelava utile la notazione dei dati nella codifica MIDI, qui si entra nell'ambito vero e proprio del suono, con rappresentazioni nel dominio del tempo o della frequenza (spettrogrammi, sonogrammi, ecc.). Anche in questo caso occorrono metodologie efficienti che permettano all'analista di saper trarre profitto dalla mole a volte notevole di dati che le macchine possono fornire.

In questa esercitazione utilizzeremo gli oggetti di PD studiati per impostare una patch che effettui alcune semplici operazioni, il conteggio di alcune particolari note in un brano tonale (un frammento di una *Invenzione a 2 voci* di Bach) e ne dia una rappresentazione grafica. Le note che intendiamo far rilevare sono la Tonica, la Dominante e la Sensibile.

¹Per un importante esempio che rientra in questa tipologia analitica si rimanda il lettore al volume di Baroni, M., Dalmonte, R., Jacoboni, C., *Le regole della musica. Indagine sui meccanismi della comunicazione*, EDT, Torino 1999.

²Per un riferimento teorico a tale metodologia di analisi si rimanda a Wallace Berry, *Structural Functions in Music*, Dover, 1987. Da segnalare, sullo stesso tema, ma da una prospettiva essenzialmente computazionale, Alessandro Perini, *Strumenti per l'analisi dei parametri musicali in ambiente di rappresentazione multistrato*, 2006, Tesi di Laurea in Scienze e Tecnologie della Comunicazione Musicale, Università Statale di Milano.

Iniziamo per prima cosa a far leggere a PD un file MIDI. Propongo una Invenzione di Bach, la prima, che è possibile scaricare liberamente dalla rete. Mettiamolo in una cartella di cui conosciamo l'indirizzo. Per rendere molto semplice l'esercitazione, potremo tagliare il file con un editor MIDI in modo da analizzare solo un frammento della parte per la mano destra.

Abbiamo bisogno dell'oggetto *xeq*, quindi ricordiamoci di aprire una sessione di PD caricando la libreria *xeq* e, ovviamente, anche *maxlib*, per gli oggetti che dovremo usare fra poco.

Fase 1 Iniziamo ad impostare una sezione con cui aprire il nostro file MIDI. Vediamo il relativo frammento di patch in fig. 8.3:

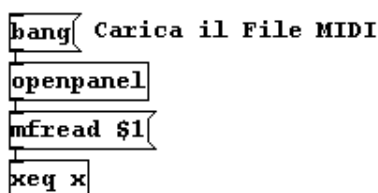


Figura 8.3:

Un bang viene spedito all'oggetto *openpanel* che apre una finestra del sistema operativo da cui è possibile scegliere il file interessato. Questo file, o meglio il suo nome, viene passato come argomento al box messaggio più in basso con la scritta **mfread**, che sta per Midi File Reader, sicché all'ultimo oggetto in basso *xeq* giunge il comando *leggi il MIDI File nome_file*. La *x* che segue a *xeq* è il nome della sequenza, ossia il nome a cui puntare ogni qualvolta vogliamo leggerla e utilizzarla.

All'indirizzo.

<http://suita.chopin.edu.pl/~czaja/miXed/externs/xeq-plain.html>

è possibile trovare tutta la sintassi dei possibili comandi da mandare a *xeq*. Poiché è molto lunga ci limiteremo a considerare solo ciò che a noi interessa. Uno di questi, l'abbiamo visto, è *mfread*. In uscita *xeq* presenta 3 outlet che qui non utilizziamo. Possiamo citare che quello più a destra manda un bang quando la sequenza è terminata durante il playback e può essere utile per impostare dei loop.

Fase 2 Ora occupiamoci di far leggere la sequenza MIDI e di mandare i relativi dati non solo alla porta di uscita ma anche all'analizzatore che ci fornirà le informazioni richieste.

Lo strumento di lettura del file MIDI è **xeq_parse** che, come si vede, raccoglie l'alias *x* riferito a 'cosa' deve leggere, creato come argomento nel precedente oggetto *xeq*. Per far partire la sequenza basta mandare un bang all'oggetto che, a sua volta, è connesso in uscita con un oggetto *noteout* che invia il tutto alla scheda MIDI. Ma il nostro intento non è quello di ascoltare la sequenza bensì di effettuare un'analisi delle note in essa presenti. Per fare questo usiamo l'oggetto *pitch* (della libreria *maxlib*). La patch mostra anche (fig. 8.4) che abbiamo interposto tra *pitch* e *xeq_parse* un altro oggetto, *stripnote* che, come si ricorderà, effettua un filtro sulle note MIDI eliminando quelle che hanno *velocity* 0 (in pratica il *Note Off*). In realtà a noi interessa solo l'inlet di sinistra che è quello che riceve il messaggio di *Note On* (nella patch risulta una connessione anche con l'inlet di destra, concernente la *velocity*). In uscita abbiamo solo la porta sinistra, quella che lascia passare il messaggio di nota MIDI premuta.

A questo punto tutto è pronto per entrare nell'oggetto *pitch*. L'uso di *stripnote* ci ha permesso di evitare un'inutile ridondanza di dati, perché ogni nota premuta avrebbe avuto il suo doppio al momento del proprio rilascio. Delle varie uscite di *pitch* usiamo soltanto

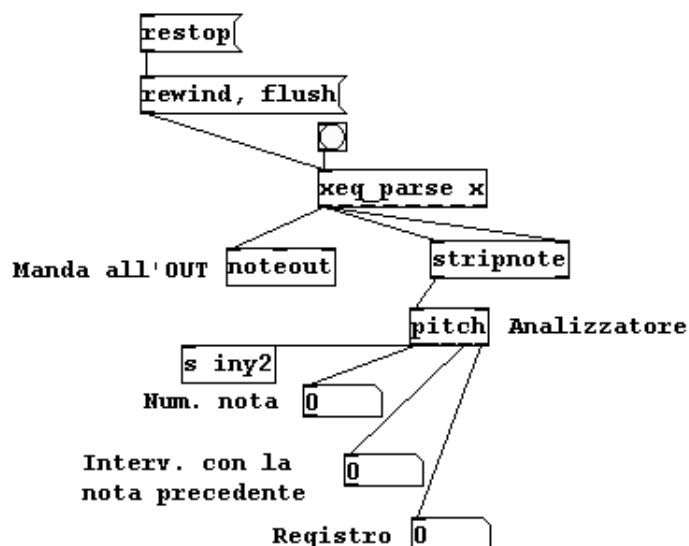


Figura 8.4:

quella più a sinistra, che rileva il numero di nota (ricordiamo che 60 è il Do centrale), e le ultime due a destra, rispettivamente indicanti l'intervallo con la nota precedente e il registro. Teniamo presente che l'oggetto dovrebbe contenere un argomento proprio per consentire il calcolo dell'intervallo con la nota precedente. In assenza verrà prodotto un numero imprevisto.

Si vede inoltre che l'uscita più a sinistra è connessa con un oggetto *send* (abbreviato con *s*), seguito dall'argomento *iny2*. A cosa serve? Per comprenderlo dobbiamo considerare che vogliamo trasmettere i dati ad una tabella, in modo da poter monitorare l'andamento degli eventi nota e averne anche una rappresentazione grafica. La variabile *iny2* è appunto il valore di ogni nota: in senso stretto è l'ingresso (*in*) dei valori delle ordinate (*y*) laddove, sul lato delle ascisse, le *x*, vi sarà un incremento di 1 per ogni valore letto. Come fare tutto ciò? Vediamo la fig. 8.5 che corrisponde ad un altro pezzo della nostra patch.

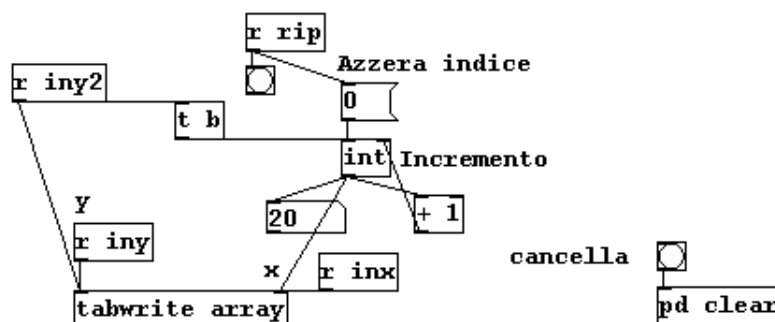
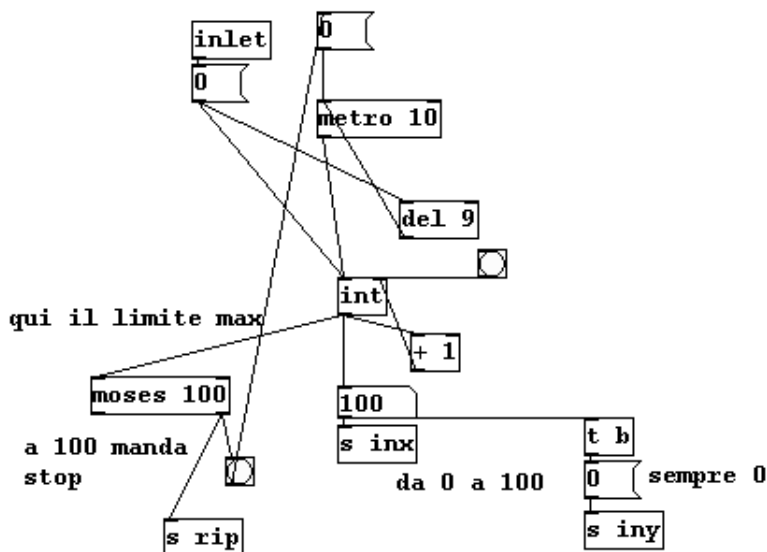


Figura 8.5:

In basso l'oggetto *tabwrite*, con argomento 'array' riceve in ingresso i dati per le *x* e per le *y*. Per le *x* si nota un piccolo contatore, simile a quello che abbiamo realizzato in una nostra precedente esercitazione (in questo momento è arrivato a 20), che incrementa di 1 ad ogni step.

Mancano ancora da chiarire a cosa servono altri 3 oggetti: *r inx*, *r rip* ed *r iny*. Per introdurla va detto che la patch può essere di qualche utilità se, una volta usata, possiamo cancellare i dati contenuti nell'array (anche a livello grafico) ed utilizzarla una successiva volta. Ecco dunque che è presente, sulla destra in figura 8.5, una sezione con l'etichetta 'cancella' dotata di una subpatch. Per vederne il contenuto, in modo Utente, è sufficiente cliccarvi sopra con il mouse. Si vedrà quanto illustrato nella prossima fig. 8.6:



Dall'inlet entra il segnale di attivazione. Il concetto su cui si basa questo processo è semplice: occorre associare ad ogni valore di x (che per noi rimanda ai numeri del contatore) uno 0. In tal modo alla fine del percorso (100 incrementi) tutto l'array è stato svuotato. Quindi il contatore, contrassegnato dall'oggetto *int*, incrementa il valore numerico contenuto nel box sottostante (ora con il numero 100): questo valore viene mandato a *inx* tramite *send*, ma anche, tramutato in bang tramite il *trigger* alla sua destra, al box messaggio che contiene uno 0: per ogni incremento dunque si avrà uno 0, mandato ad *iny*. Più in dettaglio, per far iniziare da 0 il contatore, il bang di attivazione (ossia il primo step del processo) agisce al di fuori dell'oggetto metro. Dopo pochi millisecondi di ritardo (9 per l'esattezza) l'oggetto *del* attiva metro.

A questo punto è chiaro da dove vengono inviati (send) *inx*, *iny*, e *rip*. Quest'ultimo, in sostanza, serve a riportare a zero l'indice del contatore incrementale che si occupa della cancellazione dell'array.

Fase 3 L'ultima fase della patch è quella in cui, per comodità, inseriamo ulteriori supporti grafici per facilitare le analisi di *pitch*.

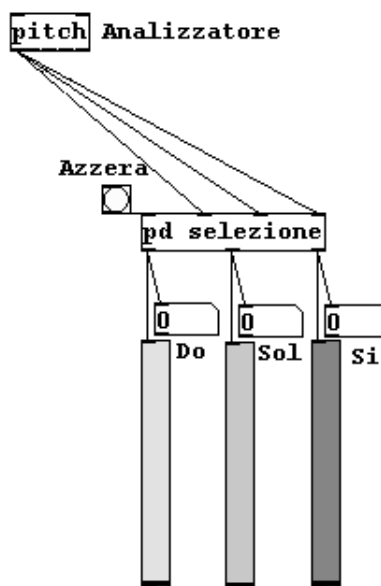


Figura 8.7:

In fig. 8.7 è riportata solo la sezione nuova. I valori emessi da *pitch* vengono inviati in una subpatch che li filtra in base ad alcuni criteri di nostra impostazione. Vogliamo che vengano rilevate le note più significative del nostro brano, che è in Do maggiore: quindi scegliamo la tonica (Do), la dominante (Sol) e la sensibile (Si). Per fare questo abbiamo bisogno dell'oggetto *select* che lascia passare proprio i valori da noi impostati. Come si vede in figura la subpatch contiene a sua volta un'altra subpatch chiamata *incr*. Quest'ultima contiene un semplice contatore, di quelli che ormai conosciamo. Appena si clicca su 'azzerare' questo contatore viene azzerato. Gli oggetti *select* sono in tutto tre, uno per ciascuna nota. Infatti sono predisposti a rispondere ai numeri di nota MIDI corrispondenti alle note che vogliamo siano riconosciute: solo quelle passeranno. Ovviamente nell'esempio ci si è limitati ad inserire solo note di alcuni registri: il lettore può liberamente aumentare i valori che saranno liberi di passare dal filtro di *select*. L'uscita di *select* entra sua volta, nel contatore *incr* che lascerà uscire il numero di occorrenze in cui quella nota si è verificata. Questo numero andrà ad incrementare gli oggetti GUI *Vslider*, opportunamente predisposti (vedi fig. 8.8).

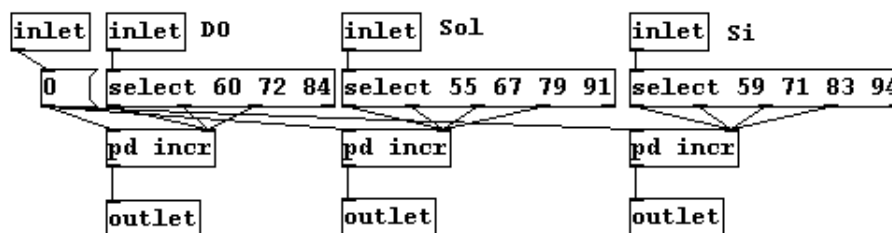


Figura 8.8:

Con questa esercitazione può dirsi conclusa la prima parte del libro dedicata alla presentazione di PD e alle sue più generali modalità d'uso. Nel prossimo capitolo, dedicato all'audio, il ruolo di PD verrà ad assumere anche un ruolo di divulgazione. Non soltanto, infatti, studieremo da vicino alcuni dei più importanti oggetti per la gestione dell'audio, ma ciò sarà anche l'occasione per affrontare alcuni classici argomenti del trattamento digitale del suono.

8.5 Installare *iemlib*

La libreria *iemlib*, scritta da Thomas Musil (Graz, Austria), è una collezione assai ricca di oggetti, per la maggior parte filtri, ma più in generale oggetti audio di vario tipo, tra cui alcuni assai utili anche per un uso non troppo specialistico di PD. I fanatici dell'*mp3*, ad esempio - che potrebbero soffrire del fatto che PD non ne supporta il codec - saranno contenti di sapere che, caricando questa libreria, c'è uno specifico oggetto che fa da lettore (*mp3play*). Inoltre vi sono diversi oggetti GUI che migliorano l'aspetto di alcune funzionalità già presenti in PD, ma graficamente più spartane.

Poiché tuttavia l'intento di questo libro è quello di inquadrare l'uso e la filosofia di PD nella sua forma base, non faremo particolare menzione degli oggetti di questa preziosa libreria. Ci limiteremo ad utilizzarne uno (*f2note*), in un prossimo capitolo, per la sua proprietà di mostrarci il nome simbolico delle note a partire da una qualunque frequenza. Per questo motivo è doveroso illustrarne brevemente l'installazione.

Scaricare il file nel sito <http://iem.at/pd/iemlib/>. Qui si trova sia la versione per Windows che quella per Linux.

Non c'è sostanziale differenza di installazione tra due sistemi operativi. In entrambi è sufficiente scompattare il rispettivo file, quindi copiare le cartelle *iemlib1*, *iemlib2*, *iem_mp3*, *iem_t3_lib* e *iemabs* nella sottocartella di PD denominata *extra*. Ad esempio in Windows potrebbe essere *c:\Programmi\pd\extra*, mentre in Linux (nel mio sistema) */usr/lib/pd/extra*. Ovviamente, in quest'ultimo caso, si faccia attenzione ad avere gli opportuni permessi di scrittura nelle cartelle in oggetto.

A questo punto lanciare PD e, dalla finestra principale, cliccare sul menù *File*, quindi *Startup...* Qui scrivere (una stringa per rigo) i nomi delle varie librerie: *iemlib1*, *iemlib2*, *iem_mp3*, *iem_t3_lib*. Non è necessario indicare una path perché sono già presenti in una sottocartella dove PD va a cercare.

Non resta che chiudere il programma e rilanciarlo. Vedremo apparire, nella finestra principale, l'indicazione del caricamento delle 4 librerie.

8.5.1 Installazione sotto Linux

8.6 Abbiamo studiato

borax, pitch, chord, xeq, xeq_parse

Messaggi: **mread**

Parte II

Gestione dell'audio

Capitolo 9

PD e l'audio. Per cominciare

9.1 Oggetti *adc~* e *dac~*

Una prima esperienza con il suono ci farà conoscere gli oggetti essenziali di PD che gestiscono il flusso audio. Iniziamo con aprire il programma al mondo esterno, creando una porta di ingresso per la nostra voce e successivamente a scrivere una breve traccia su un file.

Diamo per scontato che la configurazione audio sia a posto, agendo - come si è detto - sul menù a tendina *Media*. Per prima cosa dunque introduciamo l'oggetto **adc~**, il cui nome, come si intuisce, è dato dalle iniziali di **A**nalogic to **D**igital **C**onverter.

Se poniamo l'oggetto sul nostro piano di lavoro non ascolteremo ancora nulla perchè è necessario attivare la scheda audio, indirizzargli il flusso digitale per la conversione e avere quindi l'uscita finale.

Per la prima operazione è possibile agire in due modi:

1. sempre dal menù Media, cliccare su *Audio On* (o servendoci della sequenza da tastiera di Ctrl+*/*). Oppure

2. importare entro la nostra patch un dispositivo di controllo che attivi o spenga l'Audio. Per fare questo occorre ricordare che si può inviare un messaggio ad un destinatario facendo precedere un punto e virgola al nome simbolico che lo rappresenta. Nel nostro caso vogliamo comunicare a PD qualcosa, dunque: **<; pd>**. Successivamente, il comando per attivare la scheda audio è **dsp** seguito da 1 (viceversa, per la disattivazione, useremo lo 0). Pertanto la stringa completa da porre nel box messaggio è:

```
; pd dsp 1
```

Per evitare di mettere due box uno per l'apertura e uno per la chiusura del dsp, possiamo ricorrere ad una piccola astuzia. Invece di scrivere 1 o 0 poniamo il simbolo di una variabile, **\$1**. Quindi, in alto al box, inseriamo un trigger 0/1 che, come si ricorderà, si può ottenere anche usando un box numerico di cui abbiamo impostato (nelle *Proprietà*) l'ampiezza ad 1. Di volta in volta un 1 o uno 0 verranno passati come valore della variabile che attiverà il conseguente comportamento al nostro messaggio.

A questo punto, con il *dsp* aperto, non ci resta che mandare il flusso all'oggetto che si comporta in modo inverso di *adc~*, che è **dac~**. Qui l'acronimo sta ovviamente per **D**igital to **A**nalogic **C**onverter. Ora finalmente cominciamo a sentire i rumori del mondo.

Come si vede in fig. 9.1, i tratti di connessione tra gli oggetti quando trasportano (idealmente) segnali audio sono più spessi rispetto a quelli che portano solo segnali di controllo numerici. Ciò si rivela utile anche perchè di norma, tranne casi particolari che vedremo, un oggetto pensato per ricevere segnali audio rifiuta connessioni con segnali di controllo, e vicever-

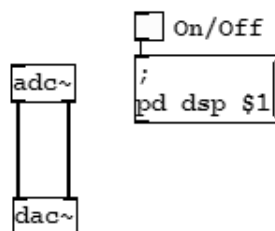


Figura 9.1:

sa. E soprattutto, a rendere ben chiara la differenza tra le due categorie di segnale, gli oggetti si differenziano per la presenza o meno di una *tilde*, $\sim$. Solo quelli che gestiscono l'audio hanno la *tilde*. Ecco dunque che i nostri oggetti convertitori ne sono provvisti.

Ma la nostra patch è ancora troppo rudimentale. Inseriamo allora un controllo di volume. Per convenzione in PD l'uscita dalla computazione audio ha valori normalizzati ad 1, dunque è necessario moltiplicare il flusso digitale per un valore compreso tra 0 ed 1 ed ottenere il valore dinamico che ci soddisfa. Per fare questo aggiungiamo un oggetto moltiplicatore, $\ast\sim$, dove la presenza della *tilde* indica che effettua i suoi calcoli su dati di tipo audio.

Si osserverà in fig. 9.2 che nell'inlet di destra dell'oggetto moltiplicatore entra un segnale numerico (nel nostro caso 0.26). Vale infatti questa regola: se l'oggetto matematico audio, e questa è una delle eccezioni a cui in precedenza abbiamo accennato, ha un argomento, allora PD sa che al suo inlet destro entrerà un numero. Se invece *non* ha l'argomento allora si aspetta un segnale audio. Questa è la regola. Tuttavia PD riesce a riconoscere il segnale di controllo (cioè numerico) e lo accetta anche se non mettiamo alcun argomento. Viceversa se l'oggetto ha un argomento e proviamo di collegare a destra un segnale audio la connessione verrà respinta e comparirà un messaggio di errore sul terminale.

Va precisato che anche $adc\sim$ e $dac\sim$ possono accettare argomenti, ossia numeri che indicano quali canali audio saranno usati da PD. In assenza si intende che l'oggetto è stereo: $adc\sim$ avrà dunque due uscite e $dac\sim$ due ingressi. Ma è possibile avere anche altre configurazioni.

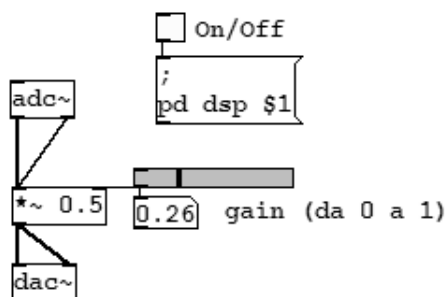


Figura 9.2:

9.2 Un semplice oscillatore: `osc~`

Invece di 'prendere' un suono dal mondo esterno possiamo dire a PD di sintetizzarne uno e mandarlo in uscita. Per fare questo ci serviamo di un oggetto classico nel mondo dell'audio digitale, l'oscillatore, che in PD si chiama `osc~`. Esso produce un'onda sinusoidale (per l'esattezza è una cosinusoide, ma la differenza è solo nella fase) di una frequenza che può essere programmata. Come? La maniera più semplice è passare il valore di frequenza come argomento, entro lo stesso oggetto. Ma è molto più versatile predisporre un sistema che accetti una frequenza a piacere.

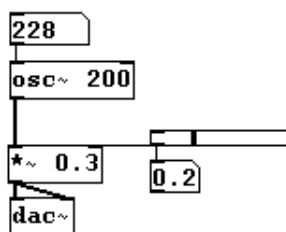


Figura 9.3:

Osservando la fig. 9.3 procediamo più in dettaglio:

Ingresso L'inlet *sinistro* accetta un box numerico tramite il quale è possibile scegliere la frequenza che vogliamo. Ad essere precisi, quando l'oggetto possiede (come in questo caso) un argomento PD assume che i messaggi di ordine numerico in ingresso siano cambi di frequenza. Se invece non ha argomento assume che in ingresso vi sia un segnale audio.

L'inlet *destro* riguarda invece la fase. Inviandovi un bang essa viene resettata a 0. Ciò può rivelarsi utile quando con la forma d'onda si vuole ripartire da alcuni valori precisi.

Argomento Riguarda la frequenza di oscillazione. Richiamando quanto detto nella sezione precedente, anche qui la presenza di un argomento fa sì che l'oggetto si aspetti, nel suo inlet sinistro, un valore numerico indicante la frequenza. In assenza di argomento accetterà solo dati audio.

Uscita Il segnale audio da mandare al convertitore `dac~`. Prestare ancora una volta attenzione al fatto che in ingresso la linea di connessione è sottile, in uscita è spessa a significare che nel primo caso si tratta di un segnale di controllo, nel secondo di un segnale audio.

9.3 Registrare files audio su Hard Disk

Poichè siamo orientati a scopi pratici prendiamo subito in considerazione alcune piccole necessità in cui ci si imbatte quando si tratta con l'audio. La prima è certamente la possibilità di registrare su hard disk i nostri esperimenti. Come è possibile indirizzare il segnale verso un dispositivo di memorizzazione su file? Con l'oggetto `writesf~` che significa proprio: "scrivi su file di suono".

Ingresso E' necessario innanzi tutto inviargli un messaggio che contenga il nome del file da aprire (che sarà quindi riempito di dati): nel nostro caso un box messaggio con intestazione **open** seguito dal nome del file che intendiamo scrivere (eventualmente con la relativa path). Quindi, per attivare o stoppare il nostro registratore virtuale, dovremo inviargli rispettivamente un messaggio di **start** e **stop**. Resta inteso che qualora clicchiamo su **start** prima di aver aperto il file da scrivere, riceveremo sul terminale un messaggio di errore. Va inoltre tenuto presente che tra un comando di **stop** ed uno successivo di apertura file va lasciato il tempo necessario alle operazioni di scrittura. L'oggetto può ricevere un argomento che indica il numero di canali: ad esempio, *writesf~ 4* per 4 canali.

Argomenti Sono i seguenti (fare attenzione al trattino iniziale):

- -wave, -aiff, -nextstep
- -big, -little (solo per nextstep)
- -bytes, che possono essere 2, 3 o 4
- -rate (la freq. di campionamento)

Esempi:

- open *miosuono.wav*
- open *-byte 3*
- open *miosuono.wav -rate22050*

Ecco dunque, in fig. 9.4, una patch completa che utilizza *writesf~*. Si noterà che abbiamo inserito un semplice controllo che serve a ricordarci di chiudere il file in scrittura. In questo esempio l'oggetto *del* invierà un messaggio *stop* all'oggetto dopo 3 secondi.

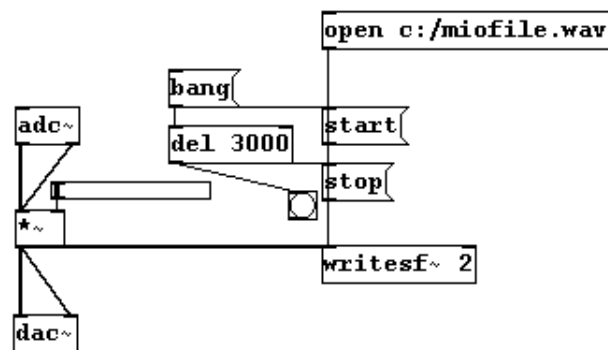


Figura 9.4:

Dopo aver aperto l'audio (potete arricchire la patch inserendo un pulsante che si occupi di aprire e chiudere il *dsp*) cliccare prima sul messaggio *open*, quindi invece di agire separatamente sui pulsanti *start* e *stop*, cliccare direttamente sul *bang* sopra l'oggetto *del*. Lo *stop* verrà infatti attivato subito e, senza dover usare una seconda volta il mouse, dopo 3 secondi (3000 msec.) *del* lascerà passare un *bang* per il messaggio *stop*. E il gioco è fatto. A questo punto non resta che andare a cercare su *C:/* il file appena creato e verificare che tutto funzioni.

9.4 Leggere files audio da Hard Disk

La via più semplice per far leggere a PD un file audio è usare il corrispettivo (in lettura) di *writesf~* che è *readsf~*.

Ingresso Come è logico, occorre prima dire all'oggetto quale file deve aprire, con il solito messaggio *open*. Una volta aperto il file si può attivare la lettura con un messaggio di *start* e, successivamente, fermarla con *stop*. In alternativa si possono usare i messaggi di 1 (vai) e 0 (ferma).

Oltre al nome del file si possono inviare, se necessario, altre richieste a *readsf~*. Ad esempio, invece di far partire la lettura dall'inizio, possiamo voler saltare un certo numero di campioni, o indicare un certo numero di canali, o il numero di bytes per canale, e anche l'ordine dei bytes (*endianness*). Si entra dunque in una tipologia di informazioni di basso livello.

Argomenti Con gli argomenti (opzionali) possiamo comunicare a *readsf~* il numero dei canali (da mettere in relazione con l'informazione in *open*) e l'ampiezza di buffer per canale (in bytes). Riprendendo un esempio dall'help di PD possiamo scrivere `<readsf~ 4 1e+006>`: il numero 4 indica che PD predispone 4 canali in uscita, mentre il numero `1e+006`, che è solo un modo compatto per scrivere 1000000, indica un'ampiezza di un milione di bytes per canale.

Uscita Avremo tante uscite quanti sono i canali dichiarati (nell'oggetto) più 1. Questo, l'ultimo a destra, è il solito outlet di controllo che manda un bang quando la lettura è terminata. Può essere utile per impostare un'attività in tale evenienza, nel caso più semplice far iniziare la lettura di un altro file o ripetere la lettura dello stesso.

Ecco allora, in fig. 9.5, una breve patch che legge il file prodotto nella sezione precedente

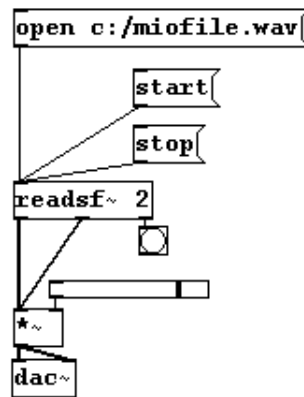


Figura 9.5:

9.5 Abbiamo studiato

`adc~`, `dac~`, `osc~`, `writesf~`, `readsf~`

Messaggi: `open`, `start`, `stop`

Capitolo 10

Lettura e scrittura da tabelle

Ci siamo occupati nel capitolo precedente della lettura di files audio da Hard Disk. Se questa opzione può rivelarsi spesso assai comoda, certamente non rappresenta il modo migliore per un qualunque intervento sull'audio. In questi casi, infatti, occorre un requisito di velocità che può essere soddisfatto solo se la descrizione del suono viene caricata, tramite l'uso di idonee tabelle, direttamente nella Ram. E' dunque necessaria prima una fase di caricamento in una tabella e poi di lettura della stessa. Il criterio di lettura, poi, farà la differenza. Una lettura che mantiene la medesima velocità di campionamento lascerà il suono all'altezza originale. Se la lettura è più lenta o più veloce verrà trasposto di conseguenza.

Iniziamo dunque, per prima cosa, ad occuparci di come PD assolve al problema di caricare dell'audio in una tabella. I casi sono di tre tipi: potremmo voler caricare in una tabella

1. l'audio già memorizzato in Hard Disk, come file
2. un suono che entra nel microfono del nostro sistema
3. un suono che PD sta sintetizzando all'interno della patch

Esaminiamo queste 3 tipologie studiando gli oggetti *soundfiler*~, *tabwrite*~ e *tabplay*~.

10.0.1 Caricare un file da Hard Disk: *soundfiler*

Ipotizziamo di avere in HD un file audio di nome *1.wav* e vogliamo caricarlo in una tabella che denominiamo *tab1*. Per prima cosa va creata la tabella. Andiamo in *Put*, quindi *Array*: Comparirà una finestra (vedi la fig. 10.1) con le proprietà dell'*array* (che vuol dire 'tabella'). Nello spazio del nome scriviamo *tab1*, in *size* dobbiamo mettere la sua estensione in numero di punti. Per indicare questo dato dobbiamo comprendere di che si tratta. Un file audio viene digitalizzato e quindi memorizzato in campioni (*samples*). Ad una frequenza di campionamento standard (44100 Hz) si avranno 44.100 campioni per ogni secondo di suono. Se noi sappiamo che il nostro suono è lungo circa due secondi dovremo creare una tabella di $44.100 \times 2 = 88.200$ punti. Ciò che eccede questa dimensione verrà ovviamente tagliato. Più in basso, nella finestra, si può mettere la spunta a *save contents* il che significa che, quando salveremo la nostra patch, verrà salvato anche il contenuto della tabella *tab1*. In questo caso, però, non ci interessa, quindi lasciamo in bianco. Cliccando infine su OK compare sull'area di lavoro della nostra patch un riquadro vuoto: è la nostra tabella.

Per riempirla useremo l'oggetto **soundfiler** che ha una duplice funzione: non solo legge da un file e carica una tabella, ma può fare anche il contrario, ossia scrivere un file leggendo da una tabella data. Studiamolo più da vicino.

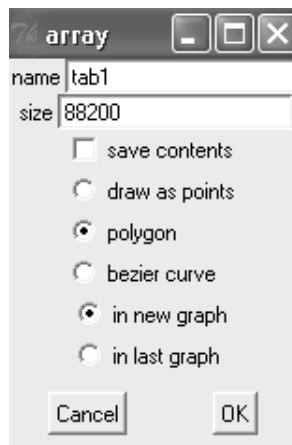


Figura 10.1:

Ingresso Vanno comunicati il nome del file audio da leggere e il nome della tabella su cui appoggiarsi. La sintassi prevede che si tratti di un messaggio che inizi con la parola *read* (leggi). Dunque, nel nostro caso più semplice:

```
<read c:/1.wav tab1>
```

I files (sia in lettura che in scrittura) possono essere a 2, 3 e anche 4 bytes. Nel caso più tipico si hanno 2 bytes. Va tenuto presente che un file normalmente possiede una header, ossia una specie di carta di identità con alcune importanti informazioni. A volte queste possono essere passate semplicemente comunicando il tipo di estensione del file, come ad esempio *.aiff*. In ogni caso è possibile riscrivere queste informazioni tramite il flag **-raw**. Ad esso seguiranno nell'ordine:

- *ampiezza dell'header*,
- *numero di canali*,
- *bytes per sampler* ed
- *endianness* (riguarda la modalità di memorizzazione dei bytes e può essere *l* - per *little*, come nei sistemi Intel - e *b* - per *big*, nei sistemi Macintosh).

Un altro flag è **-skip** che, come dice il termine, serve a saltare un certo numero di campioni. Deve essere seguito da un numero indicante appunto quei campioni.

Inoltre **-nframes** seguito da un numero che indica il massimo numero di campioni da leggere.

Ma sicuramente più importante e utile è il flag **-resize**. Con esso diciamo a PD che dovrà ridimensionare la tabella in base al file che si andrà a leggere, senza essere noi a doverglielo comunicare. La tabella risulterà così pronta per contenere tutto il suono del file in oggetto. Simile a questo flag è **-maxsize** che invece stabilisce qual è la dimensione massima che dovrà assumere la tabella.

Possiamo quindi impostare una patch come in fig. 10.2:

L'uscita dall'oggetto *soundfiler* ci informa della quantità dei campioni importati nella tabella. Poichè ci siamo serviti del flag **-resize**, il numero corrisponde a quello totale del nostro file d'esempio.

Il file è caricato in una tabella. Per ascoltare il suono dobbiamo far leggere la tabella *tab1* con un opportuno lettore.

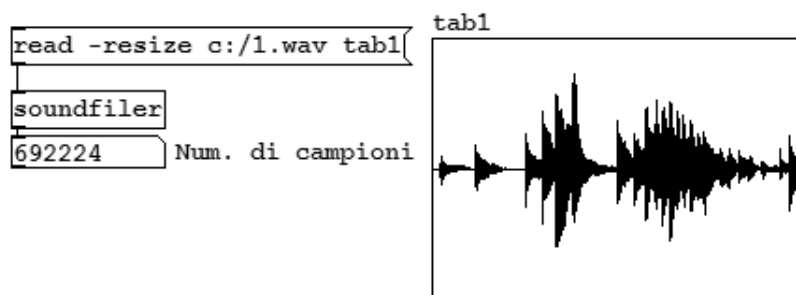


Figura 10.2:

10.0.2 Acquisizione da microfono con l'oggetto *tabwrite~*

Questa volta il procedimento consiste nel 'far entrare' un suono dal mondo esterno tramite un microfono e permettere a PD di caricarlo scrivendolo su un'apposita tabella. Potrò poi rileggere il suono nei vari modi possibili che studieremo più avanti. Per disporre la scheda audio a digitalizzare un suono occorre usare, come già sappiamo, l'oggetto *adc~* (ossia effettuare un conversione Analogico/Digitale). Possiamo poi voler aggiustare a nostro piacere il guadagno del segnale. Infine carichiamo il segnale in una tabella con l'oggetto ***tabwrite~*** che, come dice il nome, scrive su una tabella preventivamente creata e con relativo nome. Il nome della tabella deve figurare come argomento dell'oggetto. Nell'esempio di fig. 10.3 il nome è *mio*.

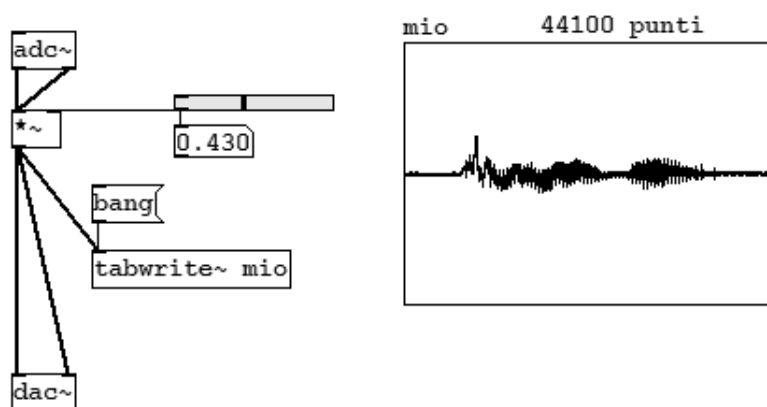


Figura 10.3:

Ingresso Nell'oggetto *tabwrite~* facciamo entrare, con apposita connessione, un segnale audio. Qui proviene da *adc~* opportunamente tarato in guadagno. Ma l'oggetto non inizia da solo a scrivere, occorre inviargli un ordine di attivazione tramite un classico *bang*. Il tutto nell'inlet sinistro. Analogamente, per interrompere la lettura si può inviare a *tabwrite~* un messaggio di *stop*. Ciò può essere utile se di un'intera sequenza audio ci interessa digitalizzare solo una certa porzione, saltando campioni iniziali e/o finali. La sequenza dei campioni verrà troncata secondo blocchi prestabiliti che, di default, è di 64 alla volta.

Può capitare di dover far leggere a PD tabelle differenti. In tal caso invece di creare un oggetto di lettura per ciascuna tabella è comodo inviare ad un unico *tabwrite~* un messaggio indicante il nome della tabella da caricare. Ciò è fatto tramite il messaggio *set nome_tabella*, opportunamente inviato a *tabwrite~*.

10.0.3 Caricare un suono in uscita da PD e rileggerlo con *tabplay~*

Questo caso non è dissimile dal precedente perchè non fa alcuna differenza se il suono proviene dall'esterno di PD o da se stessa. Per capire meglio studiamo la patch di fig. 10.4.

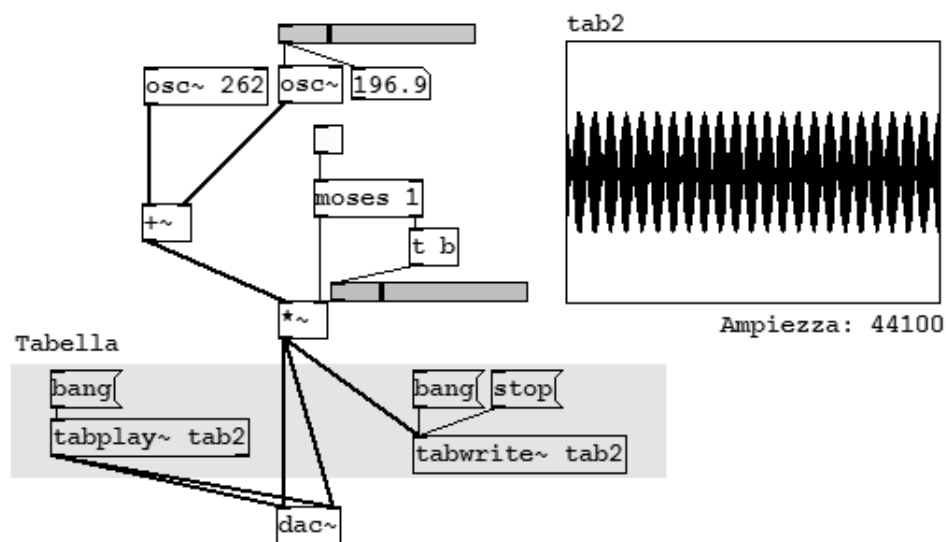


Figura 10.4:

La parte superiore crea un suono risultante dalla somma di 2 segnali elementari (prodotti da *osc~*): uno di frequenza fissa (262 Hz) l'altro variabile (qui 196.9 Hz). Entrambe le uscite vengono infatti convogliate prima nell'oggetto somma (*+~*), opportunamente tarate in guadagno con un fader, poi inviate a *dac~*.

La terna di oggetti sopra il fader del guadagno merita una spiegazione. Innanzi tutto chiediamoci: cosa vogliamo ottenere? Che PD

1. crei, in prima istanza, un suono che possiamo modificare agendo sul fader in alto (relativo al secondo oscillatore)
2. possa registrare ciò che abbiamo ottenuto caricando in apposita tabella il suono così prodotto
3. possa farci riascoltare il suono registrato.

La terna in questione serve proprio a smistare l'audio da mandare a *dac~*. Agendo sul *toggle* (che si dispone alternativamente a 0 o a 1) apparirà la spunta e con ciò inviamo in uscita un 1 a *moses* che, come sappiamo, agisce da filtro: avendo 1 per argomento manderà a sinistra i numeri in entrata inferiori a 1 e a destra quelli uguali o superiori. In particolare:

- con 1 il numero trasmesso andrà a destra dove viene convertito in bang da *toggle b* e il bang così ottenuto andrà ad attivare il fader che contiene (da prima) un certo numero (il guadagno da noi tarato). Il segnale audio viene ora moltiplicato per questo numero e inviato a *dac~*;
- con 0, ossia cliccando su *toggle* e togliendo la spunta, *moses* lascia passare lo 0 a sinistra e ciò pone a 0 il moltiplicatore che renderà 0 tutto il segnale inviato a *dac~*: il risultato è che non sentiamo più nulla del suono che PD sta producendo con i due oscillatori.

Questo silenzio ci è utile nel momento in cui vorremo sperimentare l'audio caricato in tabella. Passiamo dunque a caricare il suono nella tabella: una volta trovato il suono che ci piace clicchiamo sul bang connesso a *tabwrite~*. La tabella, preventivamente creata, a nome *tab2* e di 44100 campioni, viene così riempita. A questo punto ci torna utile spegnere l'audio che proviene dai 2 oscillatori disattivando la spunta. Subito dopo clicchiamo sul bang connesso a *tabplay~* un oggetto che serve proprio a leggere il contenuto di una tabella, nel nostro caso *tab2*, inviando l'uscita a *dac~*.

Il funzionamento di **tabplay~** è intuitivo una volta studiato il suo parallelo *tabwrite~*.

Ingresso Per attivarlo dobbiamo inviargli un bang (è equivalente anche un 1) mentre per fermare la lettura occorre uno stop (oppure 0).

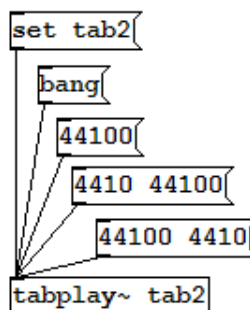


Figura 10.5:

Inoltre è possibile indicare all'oggetto quale tabella deve andare a leggere tramite un box messaggio con lo script `<set nome_tabella>`.

Assai utili sono poi i messaggi che è possibile inviare a *tabplay~* per dirgli da quale campione iniziare la lettura. Ad esempio, in fig. 10.5, il messaggio `<44100>` significa: “salta un secondo di suono e inizia a leggere dal campione 44100”. E' possibile anche indicare un range di campioni entro cui deve avvenire la lettura o, ancora, forzare una lettura in senso contrario. Ad esempio, lo script `<44100 4410>` di fig. 10.5 indica che *tabplay~* leggerà all'indietro, nel range specificato.

Uscite Dall'uscita di sinistra esce il segnale memorizzato in tabella. L'uscita destra, invece, emette un bang quando la lettura ha fine. Ciò può rivelarsi utile come strumento di controllo, ad esempio per reiterare la lettura o attivare qualche altro dispositivo.

Va detto che *tabplay~* è uno strumento assai semplice in quanto si limita a leggere una tabella relativa ad un segnale senza possibilità di fare trasposizioni o altri interventi più in profondità sui campioni. Più avanti presenteremo oggetti più potenti in grado di assimilare PD ai più costosi campionatori commerciali (e addirittura superandoli).

Nella gestione dell'audio - ad esempio attivando/disattivando la lettura di una tabella o mandando un segnale di *stop* alla lettura di un file - possono verificarsi dei click dovuti al repentino cambiamento dello stato di acceso/spento della scheda audio. E' quindi buona norma inserire nella patch un dispositivo che azzeri dolcemente il segnale prima della disattivazione, oppure impostare un tempo di attacco meno brusco nella mandata in play di un suono. Per queste operazioni è utile l'oggetto *line* e i suoi affini sul versante audio: *line~* e *vline~*.

L'oggetto *line* è stato già trattato (sez. 4.5), quanto a **line**~ la presenza della *tilde* ci fa capire che tratta un segnale audio e dunque garantisce una definizione della rampa assai maggiore. Più precisamente *line* compie il percorso inviando un messaggio di controllo (per la definizione della rampa) con una frequenza che, per default, è di 20 msec., ossia 50 aggiornamenti al secondo: possiamo dunque immaginarci più una scala che una rampa lineare. Viceversa *line*~ aggiorna la rampa ogni campione, con la conseguenza che il profilo è praticamente lineare. Ovviamente questo si sconta con un maggior onere di calcolo per la CPU. In fig. 10.6 abbiamo un semplice esempio di impiego di *line*~.

Figura 10.6:

Essa ci mostra un impiego dell'oggetto che consiste nel produrre un *fade out* o un *fade in* moltiplicando la forma d'onda prodotta da *osc*~ campione per campione: in un caso producendo una rampa discendente fino a 0 della durata di mezzo secondo (500 msec.), nell'altro salendo a 0.6 del valore massimo sempre in mezzo secondo. Va detto comunque che potremmo benissimo riscrivere la patch usando *line* al posto di *line*~. In tal caso, peraltro, PD provvederà a convertire in segnale audio l'ingresso in *~: ciò avviene automaticamente ogniqualvolta un segnale numerico (di controllo) entra in un ingresso previsto per l'audio.

Anche in questo caso i messaggi inviati all'oggetto possono essere un singolo numero o una coppia di numeri (come nell'esempio). Se si invia un solo numero line~ va direttamente a quel valore. Va detto che, una volta attivato, line~ raggiunge il target e lì rimane. Ciò significa che l'oggetto manterrà in memoria quel valore raggiunto con il precedente messaggio. Occorre fare attenzione a questo aspetto perchè spesso è causa di funzionamenti inaspettati.

Vline $\sim$ non è altro che un ulteriore perfezionamento della performance di *line* $\sim$, capace di produrre rampe di altissima precisione. In pratica, mentre *line* $\sim$ allinea i segmenti prescelti ai confini del blocco di computazione, *vline* $\sim$ scende più in dettaglio, allineando gli estremi del segnale agli esatti punti di tempo desiderati. Ciò significa che le rampe possono iniziare o

finire anche in mezzo ai blocchi senza che l'uscita venga compromessa da eventuali imperfezioni: l'oggetto effettuerà infatti le opportune interpolazioni.

Ingressi Più in dettaglio *vline~* presenta 3 ingressi. I primi due, da sinistra a destra, sono come in *line~*: il valore di target e il tempo per raggiungerlo (in msec.). Il terzo inlet accetta invece un tempo di ritardo prima di attivare la rampa. Ad esempio, inviando a *vline~* un messaggio come il seguente `< 1 2000 4000 >`, e supponendo che l'oggetto abbia in memoria un valore diverso da 1, si avrà un'attesa di 4 secondi, quindi si attiverà una rampa che porta al valore di 1 in 2 secondi.

10.1.3 Produrre un suono tramite rampe

Usando *line~* o *vline~* è possibile produrre una rudimentale forma d'onda seghettata che, portata ad un'adeguata frequenza, produce un suono. Sullo spunto di una patch presente nel manuale di Puckette, proponiamo l'esempio di fig. 10.7. Oltre a mostrare un procedimento di produzione dell'audio usando delle semplici rampe, la patch mette a confronto i due oggetti *line~* e *vline~* mostrandone le caratteristiche e le ricadute percettive del loro uso.

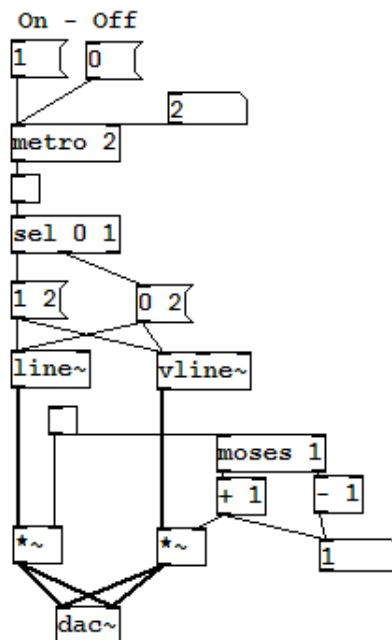


Figura 10.7:

Dall'alto: *metro* produce una rapidissima raffica di bang (500 al secondo) che accendono/spengono un *toggle* connesso ad un oggetto *select*. Da qui parte, con una frequenza di 500 al secondo, una sequenza di 0 e 1 che attiva di volta in volta 2 box messaggio: il primo porta ad 1 il valore della rampa in 500 msec., il secondo lo riporta a 0 con la medesima velocità. Si avrà dunque una scala seghettata 0, 1, 0, 1 ecc. Entrambi questi messaggi entrano negli oggetti *line~* e *vline~* ed entrambi produrrebbero contemporaneamente un suono se non vi fosse una sorta di deviatore in grado di aprire l'uno e mettere a zero l'altro (si veda l'uso di *moses*). E viceversa. In tal modo possiamo isolare il comportamento singolo di ciascun

oggetto. Qual è il risultato? Il segnale in uscita da *vline~* è assolutamente più pulito di quello prodotto dal cugino *line~* a motivo della maggiore precisione nella definizione della rampa.

10.2 Lettura di campioni: *tabosc4~* e *tabread4~*

Abbiamo già visto come fare a leggere una tabella opportunamente caricata con *tabplay~*. Di quell'oggetto abbiamo anche messo in luce la sostanziale semplicità in quanto si limita ad una lettura in modo standard, impedendo ad esempio una trasposizione. Presentiamo ora due oggetti il cui compito è ugualmente quello di leggere tabelle, ma in modo più sofisticato.

10.2.1 Uso di *tabosc4~*

Si tratta di un oscillatore *wavetable* che legge cioè in modo reiterato una tabella di campioni producendo un suono continuo. Possiamo creare una tabella di campioni sia importandola dal mondo esterno sia producendola matematicamente. Per lavorare bene l'oggetto necessita di una tabella la cui dimensione abbia, come numero di punti, una potenza di 2 con l'aggiunta di altri 3. In simboli: $2^m + 3$. Ad esempio, se ci serve una tabella con un migliaio di punti si può scegliere $2^{10} = 1024$, poi si aggiunge 3 e si ottiene 1027 punti.

L'oggetto **tabosc4~** usa un'interpolazione a 4 punti e i tre punti *in più* vengono messi uno all'inizio e 2 alla fine della tabella proprio per ottimizzare la reiterazione della lettura. Se vogliamo produrre un oscillatore sinusoidale basta caricare una funzione *seno* nella tabella e quindi lanciare *tabosc4~*. All'uopo PD mette a disposizione uno script da inserire in un messaggio da inviarsi ad una tabella (o meglio al suo nome identificativo).

Supponiamo di voler caricare una tabella di nome *sinusoide* con una senoide di 1024 punti. Creiamo la tabella non importa con quale dimensione (possiamo lasciare anche quella di default). Quindi inseriamo in un box messaggio il seguente script:

```
<;sinusoide sinesum 1024 1>
```

dove il primo numero, 1024, indica i punti della curva e il secondo (qui 1) la sua ampiezza massima.

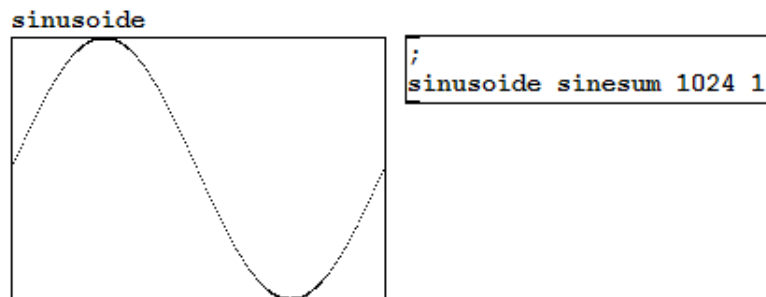


Figura 10.8:

Cliccando sul messaggio comparirà la senoide (fig. 10.8). Ora se andiamo a controllare la dimensione della tabella ci accorgiamo che è stata ridimensionata a 1027 punti. PD ha infatti aggiunto ai 1024 punti dichiarati i 3 in più che ottimizzano la tabella per una successiva

lettura. A questo punto basta integrare la patch con quanto occorre per mandare in uscita l'audio riferito alla sinusoide così creata (fig. 10.9).

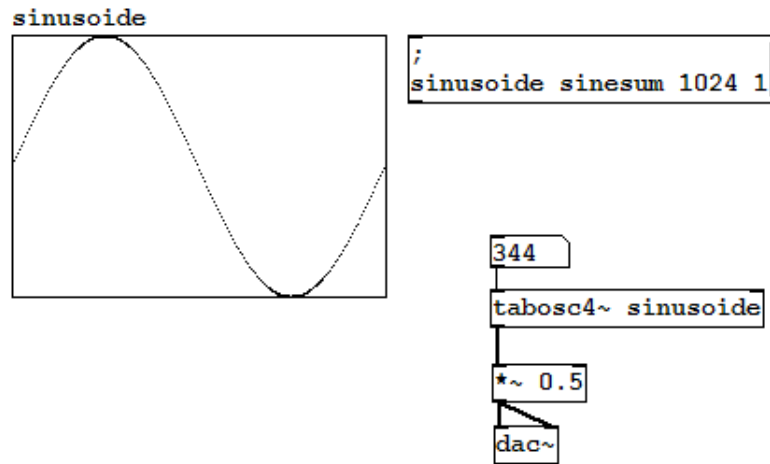


Figura 10.9:

L'oggetto *tabosc4~* possiede 2 inlet:

- il primo a *sinistra*, che in fig. 10.9 appare utilizzato, serve a comunicargli a quale frequenza dovrà leggere il campione. Nel nostro caso la lettura produrrà un suono di 344 Hz. Si può anche far scorrere il numero via mouse e ascoltare le varie oscillazioni di altezza
- L'inlet di *destra* serve invece a resettare la fase: ciò vuol dire che, inviandogli un bang, si forza la lettura a riprendere da 0.

Ovviamente possiamo divertirci a costruire onde anche più complesse grazie al comando *sinesum* che vuol dire appunto 'somma di sinusoidi'. Ad esempio inseriamo:

```
; sinusoide sinesum 1024 1 0.33 0.11
```

Dopo il dimensionamento (1024) ora i numeri sono 3: la curva sarà il risultato di 3 sinusoidi semplici di ampiezza, rispettivamente, 1, 0.33 e 0.11. Detto altrimenti: l'onda successiva ha un'ampiezza inferiore di 1/3 rispetto alla precedente.

Noteremo che la sinusoide ha trabordato i confini del riquadro che le è assegnato. All'uopo è comodo un altro comando che *normalizza* la curva, da mandare con il solito messaggio indirizzato all'oggetto tabella. Si scriverà dunque:

```
; sinusoide normalize N
```

dove N sta per il *tipo* di normalizzazione. A noi interessa *ad 1*, per forzare il contenimento esatto nel riquadro. In tal caso possiamo porre N ad 1 o anche, per default, non scrivere niente. Ma potremmo inviare anche 0.5 o altro ancora.

Si avrà dunque la patch di fig. 10.10.

Se attiviamo la scheda audio ascolteremo la differenza rispetto al suono puro dell'esempio precedente. Abbiamo creato infatti una curva (approssimativamente) a 'dente di sega'.

10.2.2 Uso di *tabread4~* con *phasor~*

La differenza sostanziale di *tabread4~* rispetto a *tabosc4~* è che il suo inlet (ne ha 1 solo) si aspetta di ricevere non più una frequenza di lettura, ma un indice, comportandosi cioè come

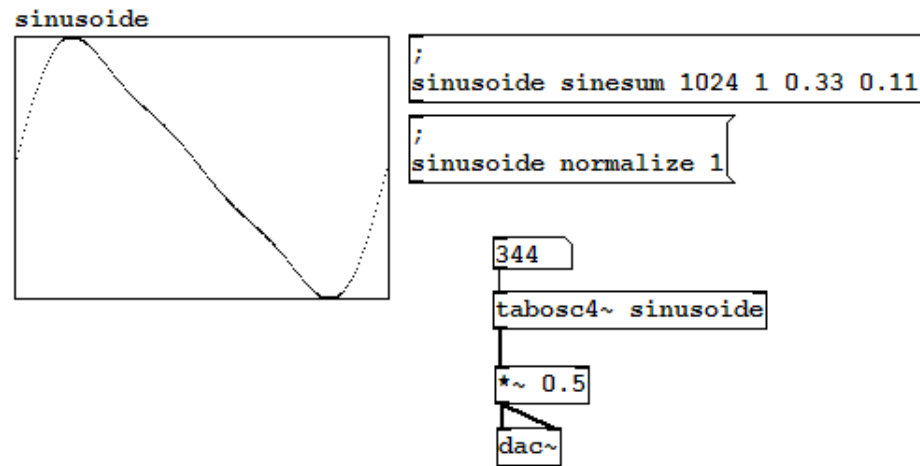


Figura 10.10:

un vero e proprio campionatore. La conseguenza è una gestione assai più precisa dei dati in tabella. Ad esempio possiamo leggerla a velocità variabile, in su, in giù, a partire da un certo punto, e così via. In termini più tecnici questo oggetto ci permette di controllare la fase del suono. Questa maggiore versatilità si risolve d'altra parte in un suo uso un po' più complesso. Dovremo infatti provvedere all'invio di una rampa di valori crescenti che copra il numero dei punti della tabella per poi ricominciare da capo e così di seguito, in un loop continuo, fino a nuovo ordine. Come fare?

Una prima soluzione del problema è creare una forma d'onda ripetitiva e l'esempio più semplice ci viene da **phasor~** un generatore di onde a dente di sega con i valori che oscillano da 0 ad 1. L'uscita da **phasor~** può essere descritta come in fig. 10.11.

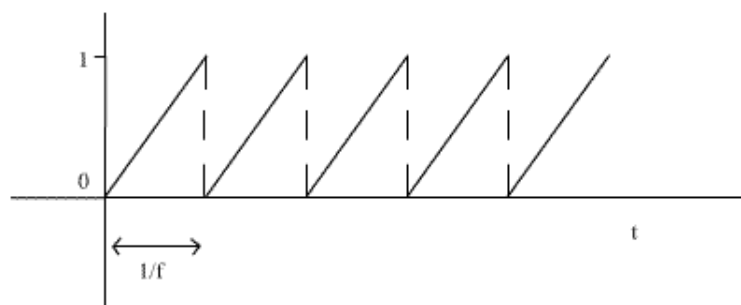


Figura 10.11:

La pendenza delle rampe sarà funzione della frequenza che noi gli inviamo (come numero) nel suo inlet sinistro. Se inviamo 400, produrrà 400 rampe - da 0 ad 1 - per ogni secondo. Il tempo per ogni rampa sarà di $1/f$, ovvero 2 millisecondi e mezzo. Ma se il nostro pacchetto di campioni è ad esempio di 1000 punti, leggere da 0 ad 1 non sarà di grande utilità perchè useremo solo 1 millesimo del campione totale. L'uscita di **phasor~** andrà dunque tarata per far leggere al meglio l'oggetto **tabread4~**. E come lavora questo oggetto? Legge valori da 1 alla dimensione totale della tabella *meno 2* (*size - 2*). Solo questa è la parte 'buona'.

Facciamo un esempio:

Creiamo una tabella di nome *sinusoide* di 1000 punti: in questo caso non siamo vincolati ad usare un'ampiezza particolare perchè il vincolo, come si è visto, si sposta sul lato della lettura. Occorre dunque leggere *tabread4~* da 1 a 998 (cioè: $1000 - 2$). Per il momento possiamo disegnare dentro l'area della tabella con il mouse. Ad esempio abbozziamo una specie di senoide, eventualmente da riscalarlo ad 1, usando il messaggio già visto, *normalize*. L'uscita di *phasor~* andrà moltiplicata per 997 e successivamente sommata ad 1, così da creare rampe (sempre con la medesima frequenza) nel range da 1 a 998, in ripetizione continua.

Cosa succede se forziamo il range oltre i limiti della parte 'buona' della tabella? (Consigliamo, a tal proposito, di studiare la patch *B03.tabread4.pd* contenuta negli esempi audio della documentazione in linea di PD). Semplicemente la forma d'onda viene 'compressa' nel tempo con conseguente alterazione della natura timbrica del suono. Infatti, se l'onda originale si dispiega su 100 punti e io ne faccio leggere 150, l'onda verrà compressa in $2/3$ del range di lettura e per $1/3$ vi saranno tanti 0.

La patch può apparire quindi come in fig. 10.12.

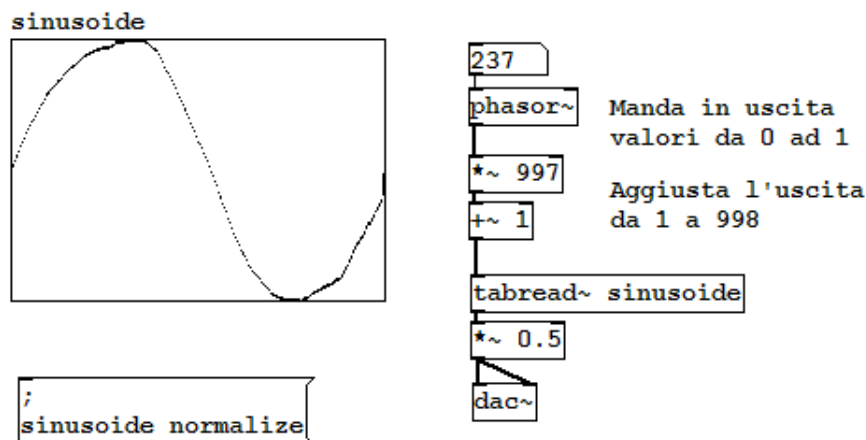


Figura 10.12:

La prima moltiplicazione serve a portare i valori primitivi di uscita (da 0 ad 1) ai nuovi che vanno da 0 a 997. Aggiungendo poi 1 ad ogni valore si avrà una oscillazione finale da 1 a 998. E il gioco è fatto.

10.2.3 Uso di *tabread4~* con *cos~*

Studieremo in questa patch l'uso di *tabread~* accoppiato con un nuovo oggetto, *cos~*, che ci è utile per impostare un involuppo di lettura. L'oggetto *cos~* manda in uscita il coseno del segnale inviato al suo ingresso, formando un'onda cosinusoidale continua. Più in dettaglio il suo ingresso è in *cicli*, dove ogni ciclo equivale a 2π radianti. Quindi mezzo ciclo (π) va a 0 a 0.5, un quarto di ciclo da 0 a 0.25 e così via, come mostra la fig. 10.13:

Se noi leggiamo solo la parte positiva della curva, da -0.25 a +0.25, otteniamo valori che oscillano da 0 a 1 a 0. Ciò può fungere come modello di involuppo per la lettura reiterata di un campione in modo da smussare gli estremi e non avere dei clicks.

Proponiamo la patch di fig. 10.14, riadattata da un esempio di M. Puckette: acquisiamo in una tabella (di nome *array1*) il suono di 1 secondo della nostra voce, sulle vocali "ah ... eh", meglio ancora se a distanza di un certo intervallo, poniamo la 5a. Impostiamo quindi un

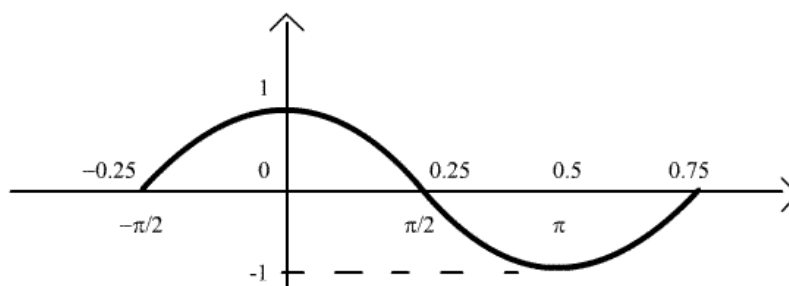


Figura 10.13:

sistema di rilettura del campione che sfrutti la proprietà di *tabread4~* di usufruire di indici per accedere alla tabella da qualunque suo punto e per il range che ci interessa. Utilizziamo infine l'oggetto *cos~* per creare l'involuppo.

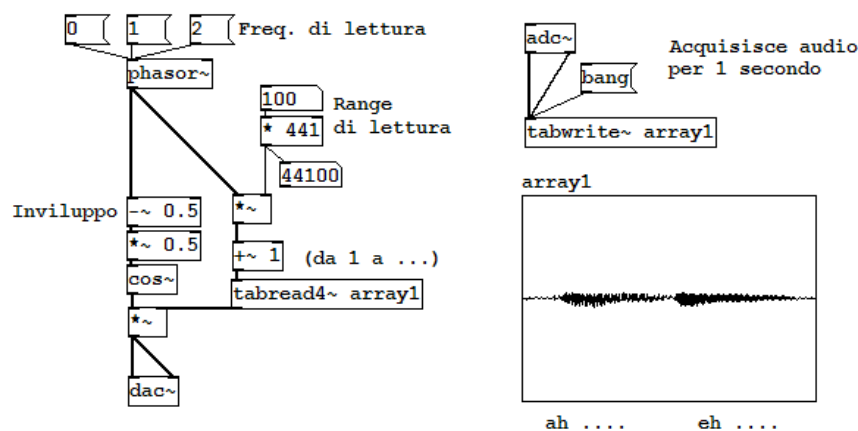


Figura 10.14:

Sull'acquisizione audio non torniamo più perchè ne abbiamo già parlato. Esaminiamo invece la parte sinistra della patch. Da *phasor~* partono rampe di segnale, da 0 ad 1, a frequenze che possiamo scegliere e che abbiamo impostato a 0, 1 o 2. Da qui partono due connessioni una che fa capo a *tabread4~* e un'altra a *cos~*. Studiamole singolarmente:

- **lato *tabread~***. I valori da 0 ad 1 in uscita da *phasor~* vengono moltiplicati per un numero che si ottiene agendo su un box numerico settato da 0 a 100. Questo valore viene moltiplicato per 441. Perché? A pensarci bene, 441 campioni, con una frequenza di campionamento di 44100 Hz. equivalgono ad 1/100 di suono. Pertanto quando il box numerico è impostato, come in figura, a 100, il moltiplicatore porterà le rampe in uscita da *phasor~* non più ad 1 ma a 44100 che è un intero secondo di suono. Ma dato che *tabread~* legge solo a partire da 1 occorre aggiungere 1 al segnale e solo a questo punto gli indici sono pronti per entrare nel lettore di tabelle. In sostanza con questo dispositivo possiamo dire quanta parte del campione andremo a leggere: tutta se poniamo il valore del box numerico a 100, metà se poniamo lo poniamo a 50, e così via per tutte le possibilità intermedie.
- **lato *cos~***. I valori in uscita da *phasor~* vengono prima sottratti di 0.5 (e ora oscilleranno da -0.5 a +0.5) e poi moltiplicati per 0.5 (e ora andranno da -0.25 a +0.25). Usiamo

così, in modo reiterato (perchè *phasor~* continua a sfornare la sua sequenza di valori), la parte positiva della curva *coseno* mostrata precedentemente in una figura. Ogni lettura del campione, intera o parziale che sia, avrà gli estremi smussati a 0 con conseguente assenza di click.

Infine, se abbiamo scelto 1 (box messaggio in alto alla patch) come frequenza di *phasor~*, ogni secondo verrà letta una porzione di tabella o l'intera se abbiamo optato per 100. Se invece scegliamo 2, il procedimento si ripete 2 volte per secondo, col risultato che l'altezza del suono originale viene traspota di un'ottava in alto.

Ma possiamo trasporre anche agendo sul range di lettura del segnale. Più in generale, se la durata temporale di una rampa equivale a tutto il campione, il suono rimarrà inalterato, ma se il tempo di lettura è maggiore della durata della porzione di campione allora avverrà una trasposizione verso il basso (verrà letto con più lentezza). Ad esempio con un valore 50 nel range di lettura il suono verrà abbassato di un'ottava. In tal caso, ovviamente, se prima avremo registrato la nostra sequenza "ah... eh..." ascolteremo solo "ah..." che è proprio metà dell'intero segnale.

10.2.4 Uso di *samphold~*

Nella patch precedente potevamo scorrere a piacere il box numerico relativo al range di lettura modificando, di conseguenza, l'altezza di intonazione del nostro suono. Tuttavia ciò comportava anche una serie di impurità al suono per tutto il tempo della modifica di tale range. Vediamo ora che si può ovviare a questo inconveniente usando un oggetto, **samphold~**: accoppiato a *phasor~* ci assicura che ogni modifica sulla quantità di campione da leggere in tabella diviene operativa solo nel momento in cui ricomincia un nuovo ciclo di lettura. Il che nella nostra patch significa quando l'involuppo è a 0. Ciò assicura l'assenza totale di discontinuità e sporcizia nel suono.

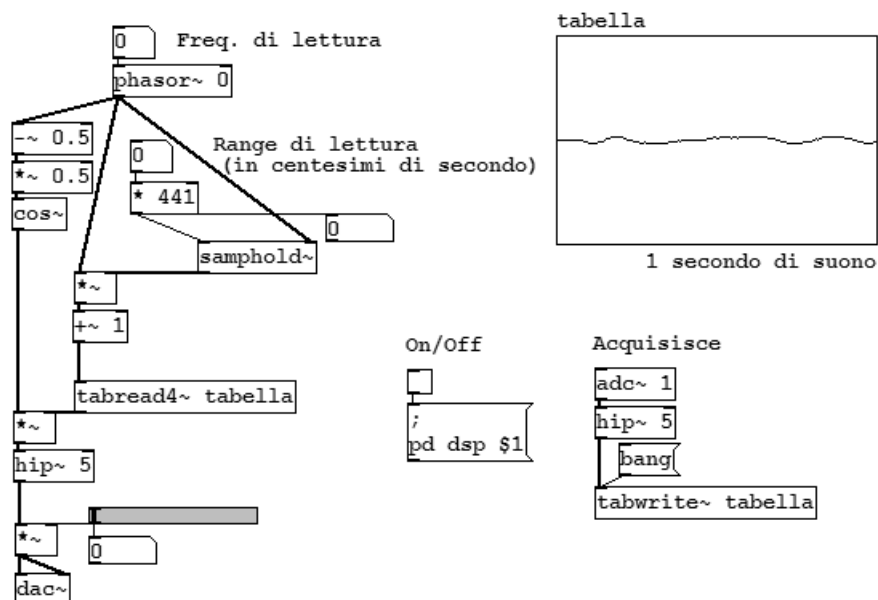


Figura 10.15:

Consideriamo della fig. 10.15 soltanto la parte aggiunta rispetto alla patch precedente, riguardante le due connessioni più a destra in uscita da *phasor~*. L'oggetto *samphold~*, come dice il suo nome, si comporta come un dispositivo 'sample & hold' in quanto acquisisce (cioè 'campiona', *sample*) un valore in entrata nel suo inlet *sinistro* e lo mantiene (*hold*) fino a quando non succede qualcosa. Cosa esattamente? Il suo inlet *destro* decresce in valore. Detto altrimenti: se nell'inlet destro entrano valori crescenti (ad es. 2, 3 5, 8, ecc.) allora rimane in memoria il valore acquisito in precedenza. Quando invece si inverte la tendenza e giunge un valore *più basso*, allora l'oggetto si attiva, acquisendo un altro campione.

Si comprende bene come tale oggetto sia pensato proprio per lavorare in coppia con *phasor~* perchè è proprio *phasor~* che cresce, in una rampa continua, fino al valore 1 per poi ricadere bruscamente a 0. Quando *samphold~* rileva (a destra) questo azzeramento allora acquisisce il valore che in quel momento è nel suo inlet sinistro. E questo valore sarà tenuto fino al prossimo ciclo (sempre che, ovviamente, nel frattempo sia cambiato l'input di controllo a destra).

L'input di *samphold~* dovrebbe essere per entrambi gli inlet un segnale audio, ma come si vede nell'esempio (fig. 10.15), a sinistra può entrare anche un numero, che verrà opportunamente convertito. L'oggetto può essere gestito anche con alcuni messaggi. In particolare:

set N Imposta un particolare valore di controllo, N. Ad es.: *set 45*.

reset N Dice all'oggetto di comportarsi come se il valore del suo argomento (N) fosse l'ultimo, di un ipotetico flusso, ricevuto nel suo input di controllo (il destro). Ad esempio, con *reset 30* solo un valore inferiore a 30 (nell'input destro) farà scattare il nuovo campionamento. *Reset* senza alcun argomento imposta invece il valore ad un numero elevatissimo (prossimo ad infinito) il che costringe l'oggetto ad una successiva attivazione (campionamento) perchè qualunque numero di controllo sarà sicuramente più basso di questo valore.

10.3 Abbiamo studiato

soundfiler~, *tabwrite*, *tabplay~*, *line~*, *vline~*, *tabosc4~*, *tabread4~*, *phasor~*, *cos~*, *samphold~*

Messaggio: **read**

Capitolo 11

Filtri

Un filtro è dispositivo basato su una *rete di ritardi* capace di produrre, a partire da un suono su cui agisce, una trasformazione in funzione della frequenza nel profilo d'ampiezza del segnale e della sua fase.

Ma cosa si intende per *rete di ritardi*? Nel caso più semplice una configurazione nel flusso dei campioni tale per cui ciascun campione del suono originale interagisce con uno o più campioni precedenti ad esso. Ad esempio, se chiamiamo $x[n]$ un certo n -esimo campione di suono, possiamo disporre che esso venga sommato ad una copia del campione precedente, $x[n-1]$. Ipotizziamo, inoltre, che ciascuno di questi valori sia moltiplicato per un certo coefficiente, a_0 e a_1 . Ciò produrrà, in uscita dal dispositivo, $y[n]$. La rete qui rappresentata viene chiamata *non ricorsiva*. In termini simbolici:

$$y[n] = a_0x[n] + a_1x[n-1] \quad (11.1)$$

Ma possiamo immaginare che l'interazione con il nostro campione $x[n]$ non avvenga con campioni precedenti non processati (ossia non modificati dal medesimo dispositivo), ma al contrario includa il campione in uscita dal dispositivo stesso, ossia $y[n]$, opportunamente moltiplicato per un certo fattore. In sostanza si potrà avere:

$$y[n] = x[n] + y[n-1] \quad (11.2)$$

In questo caso si parla di rete *ricorsiva*. E' ovvio che le reti possono assumere configurazioni anche molto complesse a partire da questi due esempi elementari.

Quanto sopra è sufficiente per comprendere il significato di due termini assai diffusi nella letteratura scientifica, ossia filtri FIR e filtri IIR.

- FIR sta per *Finite-Impulse-Response* ossia Risposta ad Impulso Finito e riguarda i filtri non ricorsivi, come alla 11.1. La ragione nel nome è nel fatto che il dispositivo risponde ad una 'storia' di segnali precedenti (impulsi) che è breve e finita. Viceversa
- IIR sta per *Infinite-Impulse-Response*, ossia Risposta ad Impulso Infinito, una caratteristica dei filtri ricorsivi esemplificata dalla 11.2. In questo caso la rete è continuamente in relazione alla sua 'storia' precedente, impressa nella sua stessa uscita in quanto reimpressa nel sistema. Dunque la 'storia' non è limitata solo ad 1 o pochi più campioni, ma torna indietro 'infinitamente'.

Il comportamento di un filtro, ossia la modalità con cui un segnale in ingresso si presenta in uscita, è la *risposta in frequenza* del filtro. In termini matematici la funzione che la esprime è

chiamata *funzione di trasferimento*. Progettare un filtro significa operare affinché esso risponda ai nostri propositi. Se vogliamo, ad esempio, eliminare o ridurre l'energia di certe aree di frequenza si dovrà ottenere un dispositivo che sia capace di farlo.

11.1 Un po' di matematica

Si tende a rappresentare un filtro tramite il cosiddetto *piano complesso* Z , termine derivante dal fatto che, per ragioni di comodità, si impiegano i numeri complessi in notazione polare. In musica elettronica si tende ad usare i numeri complessi in quanto, per le loro proprietà, permettono una più comoda gestione delle somme di angoli (detto altrimenti, le trasformazioni di fase) in termini di manipolazione algebrica. Ricordiamo che il prodotto di due numeri complessi Z_1 e Z_2 ha per ampiezza il prodotto delle rispettive ampiezze e per argomento (fase) la somma degli argomenti. Vediamo perchè.

Siano dati due numeri complessi:

$$Z_1 = r_1[\cos(\alpha) + i\sin(\alpha)]$$

$$Z_2 = r_2[\cos(\beta) + i\sin(\beta)]$$

il loro prodotto, svolgendo algebricamente, sarà

$$r_1 r_2 [\cos(\alpha) + i\sin(\alpha)][\cos(\beta) + i\sin(\beta)]$$

Ricordando ora che $i * i = -1$, concentriamoci sul prodotto trigonometrico

$$\cos(\alpha)\cos(\beta) + i\sin(\beta)\cos(\alpha) + i\sin(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta)$$

che, mettendo in evidenza la i , diventa

$$[\cos(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta)] + i[\sin(\beta)\cos(\alpha) + \sin(\alpha)\cos(\beta)]$$

Con una semplice manipolazione algebrica¹ il tutto si riduce a

$$\cos(\alpha + \beta) + i\sin(\alpha + \beta)$$

Pertanto siamo ora in grado di dire che moltiplicando 2 numeri complessi Z_1 e Z_2 si avrà:

$$r_1 r_2 [\cos(\alpha + \beta) + i\sin(\alpha + \beta)] \quad (11.3)$$

cioè significa *moltiplicare* i loro argomenti e *sommare* le loro fasi. In che modo questa formula può esserci utile? Basta ricordare l'equazione di un'onda semplice

$$x[n] = a\cos(\omega n + \phi)$$

¹Ricordiamo infatti le formule goniometriche di addizione

$$\sin(a + b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

e

$$\cos(a + b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

Si vede bene che si tratta di una sequenza di coseni di angoli (fasi) che si incrementano col crescere di n . E' semplice verificarlo: per comodità poniamo l'ampiezza $a=1$ e la fase $\phi=0$. Si avrà

$$x[n] = \cos(\omega n) \quad (11.4)$$

dove, al crescere di n , sarà come avere: per $n=1$ il coseno di ω , per $n=2$ il coseno di $\omega+\omega$, per $n=3$ il coseno di $\omega+\omega+\omega$, e così via. Ma la 11.4 altro non è che la componente reale di una successione di numeri complessi dei quali ora conosciamo la proprietà per cui la somma di n angoli equivale al prodotto di n numeri (complessi) di ampiezza unitaria, dunque

$$Z^n = \cos(\omega n) + i \sin(\omega n) \quad (11.5)$$

Possiamo rendere graficamente la 11.5 tramite un cerchio unitario (1 è infatti, come abbiamo posto, l'argomento dei nostri numeri complessi)².

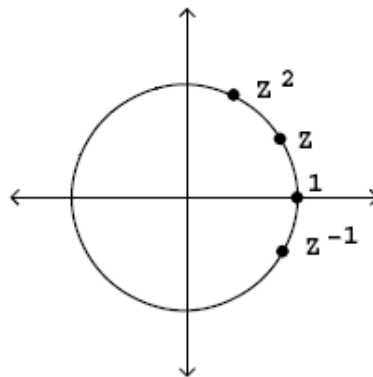


Figura 11.1:

Le potenze di Z , per $n = 0, 1, 2$, ecc. sono degli slittamenti in senso anti-orario sulla circonferenza, da z^0, z^1, z^2 , ecc. Ciò vale anche per i valori negativi di n , ma ovviamente in senso inverso (cioè orario).

A questo punto, senza addentrarci ulteriormente in una descrizione matematica della materia, siamo in grado di capire ciò che talvolta troviamo scritto nei testi scientifici (e anche nell'help di PD), cioè il significato di z^{-1} . Si tratta, detto semplicemente, di *un* campione di ritardo.

11.2 Poli e zeri

Nella letteratura scientifica i filtri vengono descritti tramite il cosiddetto *piano complesso* z . Si tratta di un grafico che rappresenta in modo bidimensionale gli effetti del filtro nel dominio della frequenza. In esso troviamo i **poli** e gli **zeri**: i *poli* rappresentano i picchi della risposta ossia la risonanza del filtro, mentre gli *zeri* rappresentano i luoghi in cui la risposta si annulla. In tal senso dire che un filtro ha *un polo* significa dire che presenta un'area di risonanza, un filtro a *due poli* che il dispositivo presenta *due* picchi nella sua risposta. Analogamente un filtro

²La fig. 11.1 è ripresa dal libro di Miller Puckette, *The Theory and the Technique of Electronic Music* (cfr. cap. 7), liberamente scaricabile dal suo sito.

ad *uno zero* presenta *un* unico ambito di frequenze in cui la risposta tende ad annullarsi, e così via. Tale piano è la rappresentazione di un'equazione complessa detta *trasformata z* che descrive in modo formale il comportamento del filtro. In essa gli *zeri* sono i valori che annullano il numeratore (ossia le sue radici), mentre i *poli* quelli che annullano il denominatore.

La materia è molto astratta ed esula dagli scopi di questo libro. Per fortuna, però, i musicisti elettronici trovano già predisposti dei dispositivi di filtraggio su cui agire in modo più o meno flessibile. Nel nostro caso PD offre un set di oggetti filtro di base il cui uso richiede solo l'introduzione di alcuni dati di configurazione che ora andremo a studiare. Per esigenze più sottili si può invece far ricorso a librerie esterne di cui, tra le più ricche, possiamo citare *iem-lib*.

11.3 Teoria essenziale sui filtri

Un filtro deve servire a ridurre o addirittura eliminare (caso più teorico) una certa regione di frequenze presente nel segnale. Ciò può avere differenti motivazioni. In un caso assai pratico si vorranno eliminare frequenze indesiderate che introducono 'sporco' nel segnale. Ciò può avvenire, ad esempio, nella regione delle parziali acute. Altre volte le frequenze indesiderate non tanto interferiscono sulla qualità del suono, perchè non si sentono, quanto sull'energia complessiva del segnale mandato alle casse. Il sistema audio è cioè oberato da un peso di informazioni che non ha alcuna rilevanza finale sul suono e può esserne quindi alleggerito per una maggiore efficienza. E' il caso, ad esempio, della componente DC (*direct current*). In senso più musicale, tuttavia, i filtri vengono impiegati per la composizione vera e propria di suoni a partire da spettri assai complessi, ciò che avviene nella cosiddetta *sintesi sottrattiva*. In tali casi un dispositivo di filtraggio dovrà non tanto eliminare semplicemente una certa area frequenziale, ma essere in grado di gestire - con riduzioni ma anche con enfattizzazioni - tutta la gamma spettrale e in modo dinamico.

Ma vi è un uso ancora più originale dei filtri, un uso che ci fa capire come essi costituiscano un *corpus* davvero essenziale nella teoria della musica digitale. I filtri possono essere impiegati per trasformazioni del suono che non attengono propriamente al suo timbro, ma ad altre categorie percettive, come l'altezza. Vedremo infatti, con un'apposita esercitazione, come si possa trasporre di un'ottava l'intonazione di un suono usando un filtro appropriato³.

Passiamo ora in rassegna alcune tipologie classiche: il filtro passa-basso, passa-alto, passa-banda e passa-tutto e, infine, i filtri equalizzatori

11.3.1 Passa alto, passa basso

I filtri più comuni sono i cosiddetti *passa basso* e *passa alto*. Come indica il termine, essi servono ad eliminare un'intera area di frequenze che si trova *sopra* (passa-basso) o *sotto* una certa soglia (passa-alto). Nel caso ideale, nell'area prescelta la risposta sarà 1, mentre sarà 0 all'esterno di tale soglia. Nella realtà, tuttavia, ciò non si verifica mai e la bontà di un filtro si misura proprio nella sua capacità di avvicinarsi il più possibile, e in modo stabile, a quei valori.

In fig. 11.2 vediamo un filtro *passa basso* in una rappresentazione che riporta le frequenze in ascissa. Si notano, propriamente, 3 aree di frequenza:

- un'area passa-banda, dove cioè il segnale passa interamente
- una centrale di transizione dove si attua la brusca caduta di energia. In un filtro ideale invece di un'area sarebbe una linea corrispondente alla *frequenza di taglio*⁴, la frequenza

³Si veda la sezione 12.4.2.

⁴La soglia tra ciò che è *aldiqua* e ciò che è al di là di questo 'taglio' è data dal valore di -3 dB del segnale.

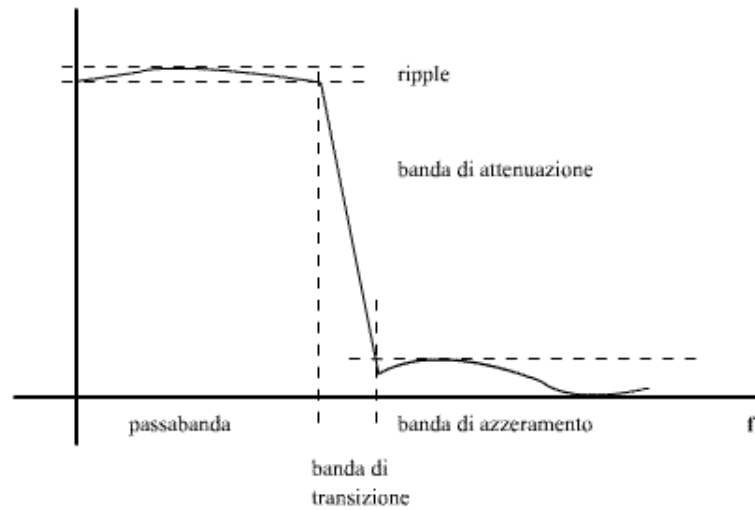


Figura 11.2:

che costituisce il limite a partire dal quale il filtro agisce abbattendo l'energia del segnale. E

- una terza, più a destra, dove la banda è più o meno stabilizzata su valori minimi.

Ma, come si vede, siamo lontani da un filtro ideale dove si avrebbe una banda di attenuazione assolutamente verticale e l'assenza totale di *ripple*, ossia di quella oscillazione intorno al valore di risposta nella banda passante. Infine l'attenuazione dovrebbe essere totale, ossia a valori di ordinata zero.

11.3.2 Passa banda

Un altro tipo di filtro è il *passa-banda* (fig. 11.3) dove l'energia passa *al di sopra* di una certa soglia *a* (di frequenza) e *al di sotto* di un'altra soglia *b*. In questo caso abbiamo 2 parametri che caratterizzano il filtro rispetto ai casi precedenti, l'*ampiezza di banda* e la *frequenza centrale*.

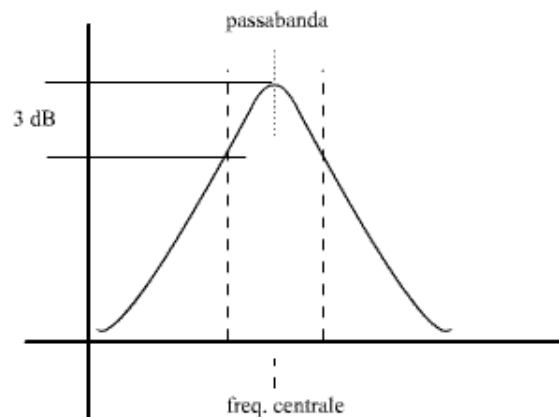


Figura 11.3:

Ampiezza di banda E' appunto l'ampiezza, in termini di frequenza, entro cui il segnale ha una potenza di non meno di 3 dB dall'ampiezza di picco.

Frequenza centrale E' la frequenza in cui la risposta del filtro è unitaria (dunque massima). Si trova di norma a metà dell'area passa-banda.

Ma, come si è detto, un uso interessante dei filtri permette di creare un sistema di *equalizzazioni* tramite il controllo di delimitate aree di frequenza, non solo in attenuazione di energia, ma anche in rinforzo. In questi filtri - che vengono disposti in serie per coprire, eventualmente, l'intero spettro - la frequenza centrale e l'ampiezza di banda sono configurabili in modo indipendente.

11.3.3 Filtri passa tutto

Può sembrare strana la dizione 'passa-tutto' perchè vien da chiedersi: a che servono questi filtri? In effetti tutte le frequenze 'passano' senza soffrire alcuna modifica. Tuttavia questo filtro produce uno slittamento (*shift*) di fase sul segnale in ingresso, detto *dispersione*, che è funzione della frequenza. Quando il ritardo dei campioni è sufficientemente esteso questo filtro genera una particolare coloratura del suono dovuto alla presenza di particolari echi in decadimento.

11.3.4 Filtri a pettine (comb filters)

I filtri a pettine (più comunemente detti *Comb filters* dove *comb* vuol dire appunto pettine) devono il loro nome al fatto che creano una risposta in frequenza ricca di picchi e relative flessioni equispaziate (in frequenza) che, soprattutto quando sono abbastanza fitti, ricordano proprio i denti di un pettine (fig. 11.4).

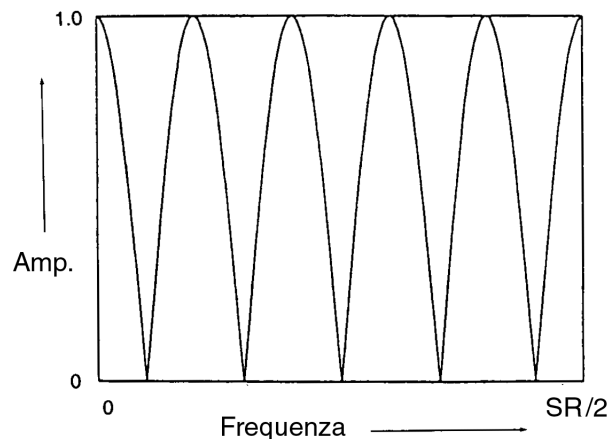


Figura 11.4:

Anche in questo caso vi è una tipologia FIR e una IIR. La caratteristica di un filtro a pettine è che il ritardo non è di un solo campione, ma di un certo numero di campioni preso interamente.

L'equazione di un un filtro a pettine FIR è la seguente:

$$y[n] = x[n] + x[n - D] \quad (11.6)$$

dove D rappresenta un ritardo di tempo di più campioni. Perchè diventi significativo il ritardo D deve essere di circa 0.1 ms^5 . In tali circostanze si vengono a creare i tipici avvallamenti dovuti all'azzeramento dell'ampiezza di alcune frequenze dello spettro, per via degli effetti delle cancellazioni di fase e conseguenti rinforzamenti all'interno di tali punti.

Nel caso descritto dalla 11.6 il primo picco si troverà a

$$f = \frac{1}{D} * SR$$

dove SR sta per frequenza di campionamento. I successivi si verificheranno a $2f$, $3f$, $4f$, ecc. Ne segue che questo filtro si rivela utilissimo sul piano musicale se vogliamo rinforzare, in un certo spettro, una frequenza f e le sue parziali superiori.

Facciamo un esempio. Se il ritardo D consiste di 16 campioni e la frequenza di campionamento (SR) è 44100 Hz si avrà il primo picco alla frequenza $(1/16)*44100 = 0.0625*44100 = 2756.25$ Hz e le successive frequenze enfatizzate saranno a 5512.5, 8268.75, 11025 Hz, ecc., fino alla frequenza di Nyquist.

Se invece di sommare si sottrae la componente di ritardo, e l'equazione diventa

$$y[n] = x[n] - x[n - D]$$

il comportamento del filtro è differente: a 0 Kz si ha un primo annullamento di energia e i successivi annullamenti si avranno a $2f$, $4f$, $6f$, ecc. Vengono cioè rimosse la frequenza fondamentale e tutte le sue parziali. Corrispettivamente i picchi si avranno a $\frac{f}{2}$, $\frac{3}{2}f$, $\frac{5}{2}f$ e così via.

Un filtro a pettine IIR, com'è della tipologia dei filtri ricorsivi, riprende in ingresso la sua stessa uscita. La sua equazione è

$$y[n] = a * x[n] + b * y[n - D]$$

In base al coefficiente b (del ritardo di campioni) questo filtro produce un certo effetto di risonanza che, per valori molto alti di b , può produrre *overflow* e conseguente distorsione.

11.3.5 Filtri equalizzatori

Questi filtri servono per 'aggiustare' la distribuzione dell'energia spettrale di un segnale, attenuando certe frequenze ed esaltandone altre. In particolare si hanno i filtri *Shelving* e i filtri *di picco*.

Filtri Shelving Servono a regolare (in più o in meno) il segnale alle estremità della banda di frequenza e cioè *al di sotto* di una certa soglia verso il basso (lo 0) e *al di sopra* di una certa soglia verso l'alto. Quella soglia è detta *crossover*. La versatilità di questi filtri risiede del fatto che si può regolare sia la localizzazione del *crossover* che il guadagno relativamente all'area di basse e alte frequenze su cui il filtro agisce.

Filtri di picco Questi filtri sono simili ai precedenti con la differenza che possono selezionare l'area di frequenze entro cui agiscono (ossia in qualsiasi ambito dello spazio frequenziale). Essi consentono di esercitare un controllo sull'ampiezza della banda di frequenze sui cui operano, sulla localizzazione del centro banda, nonché sul tipo di pendenza (ossia sul guadagno) del *fuori* banda e del *dentro* banda.

⁵Si tenga presente che il ritardo di un solo campione, alla frequenza di campionamento di 44100 Hz, equivale ad un ritardo temporale di circa 0.02 millesimi di secondo.

11.4 I filtri più semplici in PD

Le tipologie di filtri appena esaminate si trovano in PD con i nomi **hip~** (high pass), **lop~** (low pass) e **bp~** (band pass). Esaminiamoli brevemente.

11.4.0.1 *hip~*, *lop~* ed *env~*

Iniziamo con un filtro *passa-alto* considerando la patch di fig. 11.5 che commenteremo solo sommariamente lasciando al lettore il compito di studiarla punto per punto.

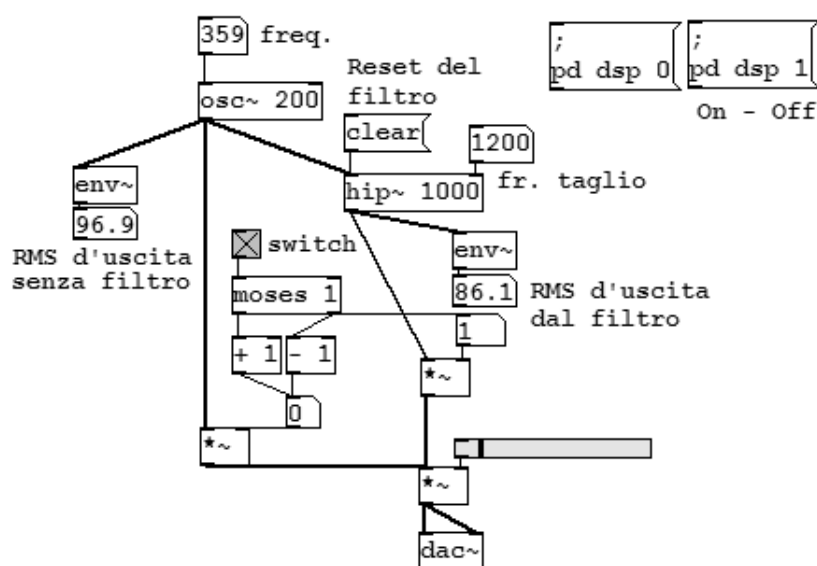


Figura 11.5:

Un semplice oscillatore *osc~* su una frequenza di 359 Hz produce un segnale che è mandato su due canali differenti. In quello a sinistra va direttamente verso il *dac~*, mentre a destra entra prima in un filtro **hip~** tarato, in figura, su una *frequenza di taglio* di 1200 Hz. Ricordiamo che la 'frequenza di taglio', riguardante i filtri *passa-basso* e *passa-alto*, sta a significare quella frequenza in cui l'attenuazione del filtro è pari a - 3dB.

Al centro c'è un dispositivo di switch con il relativo *toggle*. Con la spunta viene azzerato il flusso uscente dall'oscillatore e si ascolterà il segnale uscente dal filtro mentre senza la spunta la situazione si rovescia: si azzerà l'uscita dal filtro e si sentirà il suono non filtrato.

Perché una frequenza di taglio così lontana da quella del segnale originale? Per avere un risultato percettivo importante. Del resto basta sperimentare la patch: si noterà che il segnale filtrato sarà *più* debole di quello non filtrato e l'energia del segnale sarà tanto più attenuata quanto più ci allontaneremo dalla frequenza dell'oscillatore, impostando un valore a nostra scelta nell'apposito box numerico (in fig. 11.5 con la scritta *fr. di taglio*). Questo abbattimento di energia è proprio l'effetto del filtro. Il lettore può provare ad aggiungere in cascata uno o più filtri *hip~* dalle medesime caratteristiche: noterà che l'attenuazione aumenta in proporzione ai numeri dei filtri usati.

Più in dettaglio l'oggetto **hip~** è un filtro *passa-alto* ad 1 polo. In pratica ciò significa che presenta un solo picco nella risposta in frequenza. Consta di due ingressi: quello a sinistra è

dove entra il segnale da processare, quello a destra serve invece per aggiornare la frequenza di taglio (che già può comparire come argomento). E' possibile inoltre effettuare un reset dello stato interno del filtro mandando all'inlet sinistro il messaggio *clear*. Il filtro è normalizzato in modo da avere un guadagno pari a 1 alla frequenza di Nyquist (ossia $SR/2$). Un tipico utilizzo di questo filtro è per rimuovere la componente di DC (frequenza 0) da un segnale, cosa che porterebbe ad un aggravio nel flusso di informazioni.

La patch è completata, infine, con due oggetti di nome **env~** che mandano in uscita il controllo dell'ampiezza RMS⁶, ossia il *valore efficace*, espresso in decibel (il valore massimo del segnale, 1, è di default normalizzato a 100). Sebbene qui non compaiano, *env~* può avere due argomenti con cui impostare i criteri per il calcolo. Come primo argomento si può stabilire l'ampiezza in campioni della finestra di analisi. Di default il valore è 1024. Il secondo argomento indica invece il periodo in campioni per l'analisi. Di default vale il principio che sia metà del primo argomento (ossia della finestra).

In modo analogo, ma ovviamente inverso, lavora l'oggetto **lop~**, il filtro *passa-basso*. In questo caso il filtro è normalizzato per un guadagno di 1 alla frequenza 0.

11.4.1 *bp~* e *noise~*

L'oggetto **bp~**, che sta per filtro *passa banda*, si presenta con un inlet in più (ossia 3) rispetto ai precedenti. Da sinistra a destra i suoi inlet si riferiscono a:

- segnale da processare
- frequenza centrale
- Q

Poichè la frequenza centrale si colloca a metà dell'area della banda passante, con il controllo centrale possiamo impostare la frequenza, con tutto un suo *intorno*, in cui la risposta del filtro è massima. Quanto sia, poi, questo intorno - più stretto o più largo - dipende dal valore **Q** che esprime il rapporto tra la frequenza centrale e l'ampiezza di banda. Ad esempio, ponendo la frequenza centrale a 1000 Hz, ad una banda di 200 Hz corrisponde una **Q** di 5, per una di 50 Hz avremo una **Q** di 20. E così via.

Per valori maggiori di **Q** la campana che rappresenta il filtro si stringe, per valori minori si allarga. Inoltre anche in *bp~* si può inviare un messaggio *clear* all'inlet di sinistra per resettare lo stato interno del filtro. Va aggiunto che l'oggetto può presentare al suo nascere due argomenti, rispettivamente per la frequenza centrale e per **Q**.

Facciamo subito un esempio con la patch di fig. 11.6.

La parte sinistra ci mostra 4 oscillatori impostati a 200, 300, 400 e 500 Hz che inviano complessivamente i loro segnali all'oggetto *bp~* dotato di due argomenti (freq. centrale e **Q**). Ma è sicuramente più interessante impostare criteri dinamici per modificare quei valori. E a questo è dedicata la parte centrale della patch. A sua volta questa si perfeziona con la parte più a destra destinata a correggere, con opportuni inviluppi, i click che seguirebbero inevitabilmente a rapidi cambiamenti di impostazione. Ecco una breve spiegazione.

Quattro messaggi inviano i valori 2, 3, 4 e 5 ad un oggetto moltiplicatore che li moltiplica per 100 per impostare una frequenza di taglio coincidente con ciascuna frequenza degli oscillatori. L'obiettivo è quello di mostrare che un segnale contenente la somma di 4 frequenze può essere filtrato opportunamente facendo 'passare' solo (o quasi) una di esse.

⁶ Acronimo che sta per *Root Mean Square*, ossia 'Radice della media dei quadrati'. Si tratta appunto di un valore medio che si applica alle grandezze variabili nel tempo e, in quanto tale, è un po' inferiore al valore di picco.

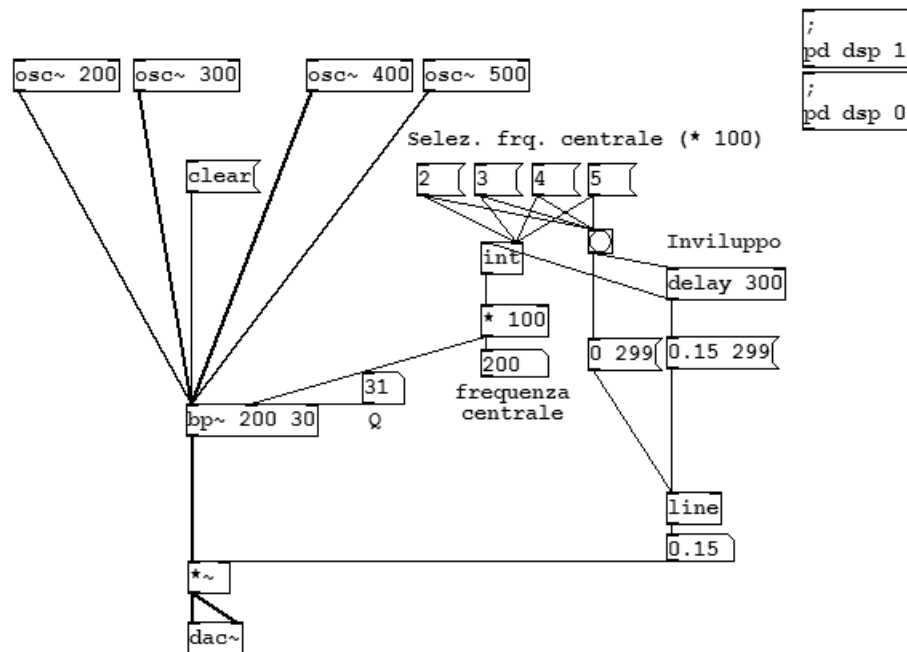


Figura 11.6:

Per evitare le brusche interruzioni nel flusso audio si è impostato il seguente criterio: cliccando su ciascuno dei 4 messaggi numerici il valore entra prima, da destra, in un oggetto *int* che, come si ricorderà, memorizza un valore senza mandarlo in uscita (a questo provvede l'inlet sinistro). Ma nel contempo viene mandato un bang a due target: ad un messaggio, per un oggetto *line* sottostante, e ad un oggetto *delay*. Dunque succede questo: *line* viene settato a creare una rampa di inviluppo negativa, che raggiunge 0 in 299 msec. Dopo 1 msec. l'oggetto *delay* sblocca il passaggio del segnale bang che viene indirizzato, a sua volta, ad altri due obbiettivi:

- un box messaggio con valori che rimandano *line* a 0.15 in altri 299 msec. Il valore 0.15, come prima lo 0, aggiornano l'oggetto moltiplicatore che entra in *dac~* per il controllo generale dell'inviluppo. Si è scelto un valore massimo piuttosto basso (0.15) da mandare a *dac~* perchè nella patch sono attivi ben 4 oscillatori
- un oggetto *int*, già contenente il valore selezionato. Raggiunto nell'inlet di sinistra, può mandare l'ordine di cambiare la frequenza centrale del filtro.

Va detto, infine, che agendo sul box numerico di Q oltre a modificare la selettività del filtro si interviene anche a modificare l'energia complessiva del segnale finale.

Mandando in esecuzione la patch, e mantenendo un valore di Q piuttosto alto, si può constatare effettivamente il restringimento della banda passante quasi esclusivamente alla sola frequenza da noi impostata.

Noise~ Le caratteristiche del filtro *bp~* ben si prestano per illustrare, sia pure sommariamente, i principi della sintesi sottrattiva, in cui si parte da una fonte grezza, assai ricca di frequenze, per poi procedere ad una riduzione o selezione solo di particolari frequenze. Per

darne un semplice esempio introduciamo l'oggetto **noise~** che produce *rumore bianco*, ossia un segnale complesso ricco di tutte le frequenze possibili statisticamente distribuite.

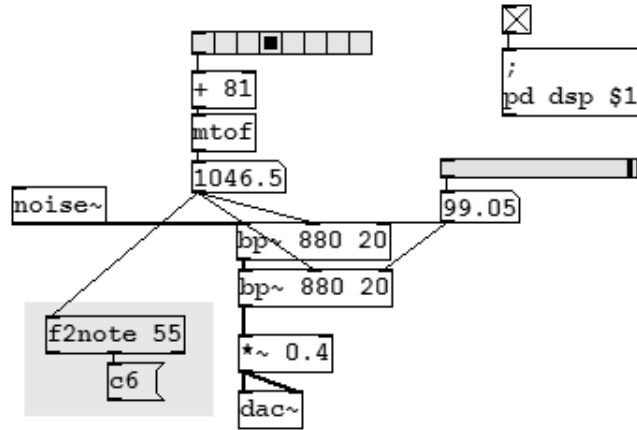


Figura 11.7:

In fig. 11.7 abbiamo, sulla sinistra, l'oggetto **noise~** e, in centro, due **bp~** in cascata. L'idea di metterne due in serie si giustifica per rinforzare il risultato del processo di filtraggio. Entrambi gli oggetti hanno due argomenti di riferimento, ma sia la frequenza centrale che il valore di Q sono programmabili. In particolare per la frequenza centrale troviamo un oggetto *Hradio* tramite cui è possibile selezionare agevolmente una scala di valori (qui da 0 a 7) che vengono sommati ad 81. Il motivo di tale somma è quella di produrre numeri intesi come note MIDI: in tal senso 81 corrisponde al La della quinta ottava. Grazie poi all'oggetto **mtof** si converte il numero di nota MIDI in valore di frequenza (Hz). L'uscita di questo numero entra poi nell'inlet centrale di **bp~**. Elevando il valore di Q al fondo scala del fader relativo (nell'esempio in figura è: 99.05) si assottiglia adeguatamente l'azione selettiva dei due filtri sicché possiamo distinguere dei suoni ben intonati secondo i valori nota che abbiamo selezionato. Spostando invece all'estrema sinistra il fader (ossia con Q basso) la strozzatura del dispositivo di filtraggio si allarga facendo passare gran parte dell'energia spettrale prodotta da **noise~**. In mezzo avremo tutte le gradazioni intermedie.

Per finire, sulla parte in basso a sinistra e nel riquadro, troviamo l'oggetto **f2note** che non fa parte della dotazione standard di PD, ma è preso dalla libreria *iemlib*⁷. La funzione di **f2note** è in parte l'opposto di **mtof**, in quanto prende valori di frequenza in ingresso e manda nell'uscita di sinistra (qui non usata) il corrispondente numero MIDI. Inoltre, ed è ciò che qui ci interessa, dall'outlet centrale fuoriesce il valore alfanumerico della nota (nell'esempio, *c6*): ciò può risultare comodo al musicista, certamente più abituato ad intendere le note tramite il loro nome letterale che non tramite frequenze o numeri MIDI. Va detto che **f2note** presenta un argomento con cui è possibile tarare la lettura della nota. Ad esempio il numero 55, presente nel nostro esempio, fa in modo che il nome dall'outlet centrale sia tarato con la 4a. ottava per il registro centrale. Infine l'uscita di destra (qui non usata) è in grado di farci visualizzare la quantità di *cents* per la quale il segnale in ingresso si discosta dal nota rilevata.

⁷In 8.5 abbiamo parlato dell'installazione di questa libreria.

11.4.2 $vcf\sim$

Assai simile a $bp\sim$ (filtro ad 1 polo), l'oggetto $vcf\sim$ presenta un'importante differenza: per impostare la frequenza centrale accetta un segnale audio al posto di uno di controllo. Ciò lo rende più potente di $bp\sim$ in quanto le modifiche di questo parametro possono ora arrivare (e cambiare) alla velocità dei segnali audio. Ovviamente ciò si sconta con un maggiore impiego di risorse del sistema. Dato l'impiego presunto per $vcf\sim$ non è previsto un valore di inizializzazione relativo al parametro 'frequenza centrale'. Pertanto l'unico argomento possibile è quello relativo a Q.

Un'ulteriore differenza è che $vcf\sim$ presenta due uscite separate (in riferimento a segnali complessi), rispettivamente per la parte reale (sinistra) e immaginaria (destra), laddove $bp\sim$ ha solo l'uscita reale.

Un esempio Come esempio riportiamo, in fig. 11.8, una patch presente nell'help di PD, parzialmente modificata per essere più leggibile.

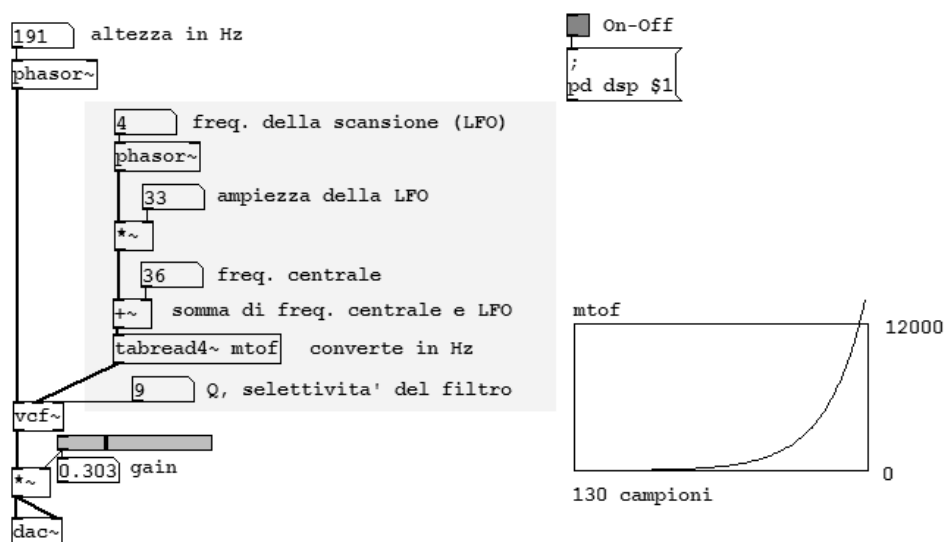


Figura 11.8:

Nell'oggetto $vcf\sim$ entrano (primi due inlet, da sinistra) due segnali audio. Nel primo (a sinistra), un segnale base prodotto da $phasor\sim$, qui impostato alla frequenza 191 Hz. Nel secondo, il segnale audio che controlla dinamicamente il filtro $vcf\sim$. Il controllo assume impostazioni configurabili, come risulta nella parte centrale della patch. Esaminiamola da vicino.

Un secondo oggetto $phasor\sim$ produce un segnale a bassa frequenza (qui indicata con LFO), in particolare un'onda a dente di sega con valori da 0 ad 1 alla frequenza f (qui 4). L'oggetto $\sim$ moltiplica l'ampiezza delle oscillazioni di 33 volte, dunque ora il segnale va da 0 a 33, alla frequenza di 4 volte al secondo. Questo range di oscillazioni viene quindi ricollocato su una frequenza di riferimento, espressa in unità MIDI (ad es. 60 sarà il Do centrale) tramite un semplice oggetto somma. Nella fattispecie si aggiunge 36 ai valori di oscillazione precedenti: ne deriva un nuovo range di oscillazione, da 36 a 69. Con questi valori di lettura, tramite $tabread4\sim$, si legge la tabella di nome *mtof* che si può vedere sulla destra. Si tratta di una tabella di 130 punti che rappresenta, con una curva evidentemente esponenziale, la conversione

tra i valori MIDI e le frequenze in Hz. L'oggetto *tabread4~* nel nostro caso non fa altro che leggere la tabella tra i punti 36 e 69 con una frequenza di 4 volte al secondo. Ovviamente si tratta di indici della tabella, che sono associati ai valori effettivi (di frequenze in Hz) della curva ritratta in figura. Essi verranno quindi trasmessi all'ingresso di *vcf~* producendo il tipico effetto di filtraggio dinamico.

Infine il terzo inlet di *vcf~* permette di impostare un valore al parametro Q del filtro. Più alto è questo valore più selettivo diventa il filtro.

11.5 Filtri grezzi

I filtri che abbiamo esaminato possono definirsi 'amichevoli' in quanto consentono all'utente di immettere immediatamente i dati per una risposta voluta. Detto altrimenti si tratta di dispositivi di alto livello il cui funzionamento può essere per lo più ignorato da parte del musicista che li impiega. Ma questa loro proprietà può talvolta scontarsi in termini di una minore flessibilità. Per questo motivo esiste in PD un gruppo di oggetti filtro di carattere, per così dire, grezzo, che sono meno facili da usare in quanto, senza una specifica conoscenza delle loro caratteristiche, possono avere comportamento imprevedibili. In essi l'utente può agire direttamente sui coefficienti dei filtri, così come essi appaiono nelle equazioni che li descrivono in modo formale.

Data la complessità dell'argomento faremo solo un breve accenno ad alcuni di essi. Tratteremo in particolare degli oggetti *rzero~*, *rzero_rev~*, *rpole~* e *biquad~* in quanto rappresentativi di questa categoria di filtri.

11.5.1 *rzero~*, *rzero_rev~*, *rpole~*

La lettera *r* che precede il nome di questi oggetti sta a significare che si tratta di filtri che operano su segnali a valore reale e accettano coefficienti reali.

rzero~ Come dice il suo nome, ***rzero~*** è un filtro ad uno zero, non ricorsivo che si comporta come un filtro passa-alto. Presenta 2 inlet, quello a sinistra riceve il segnale da filtrare e quello a destra serve per la sua configurazione ossia, in questo caso, per settarlo con un opportuno coefficiente (da intendere anche come segnale audio). Tale coefficiente, che può essere inserito anche come argomento, presiede al controllo della risposta in frequenza del filtro. In pratica imposta dove avverrà lo 'zero'.

La sua equazione è la seguente:

$$y[n] = x[n] - a[n] * x[n - 1]$$

Come si vede l'uscita è data dalla differenza tra un campione d'ingresso e il precedente campione, opportunamente scalato tramite il coefficiente $a[n]$. Il filtro è sempre stabile per qualunque valore di a (diversamente da quanto accade per i filtri ricorsivi).

La funzione di trasferimento è

$$H(Z) = 1 - aZ^{-1}$$

dove, come si è visto sopra, per Z^{-1} è da intendere il ritardo di un campione. Inoltre l'assenza di un denominatore ci fa capire che l'unica radice possibile riguarda uno zero.

rzero_rev~ E' analogo all'oggetto precedente se non per la posizione del coefficiente che in questo caso riguarda il campione in ingresso e non il campione ritardato. Sicchè il comportamento di questo filtro è 'rovesciato' nel tempo rispetto al precedente, da qui il suo nome. L'equazione che lo descrive è la seguente

$$y[n] = -a[n] * x[n] + x[n-1]$$

Anche in questo caso il filtro è sempre stabile. La funzione di trasferimento è

$$H(Z) = -a + Z^{-1}$$

rpole~ Per converso, **rpole**~ è un filtro ad un polo, dunque ricorsivo. Presenta anch'esso due ingressi, uno per il segnale ed uno per il coefficiente (anche come segnale audio) che serve a localizzare il polo. Il suo comportamento è descritto dalla seguente equazione

$$y[n] = y[n-1] + a[n] * x[n]$$

dove si vede la componente ricorsiva, data dal termine $y[n-1]$. L'ingresso viene poi scalato secondo il coefficiente $a[n]$ che in questo caso deve essere sempre maggiore di 1, altrimenti il filtro diventa instabile.

La sua funzione di trasferimento è

$$H(Z) = \frac{1}{1 - aZ^{-1}}$$

dove vediamo, peraltro, che la variabile complessa si trova al denominatore e dunque la radice possibile riguarderà un polo.

11.5.2 Il filtro *biquad*~

L'oggetto **biquad**~ di PD, più complesso e versatile dei precedenti, è una tipologia di filtro a 2 poli e 2 zeri, basato su una struttura IIR (*Infinite Impulse Response*) di secondo ordine. Il primo termine IIR, come abbiamo visto, denota i cosiddetti *filtri ricorsivi*, ossia quelli basati sul reindirizzamento dell'uscita del filtro verso l'ingresso. La seconda proprietà, invece, quella di essere di 'secondo ordine', denota ritardi di due campioni (laddove un filtro di 'primo ordine' riguarda ritardi di un solo campione). Si tratta di un filtro che può essere felicemente combinato con altri analoghi per creare sezioni di filtraggio anche complesse, agendo opportunamente sui parametri di configurazione di ciascun componente.

Una tra le più tipiche forme di questo filtro consiste nella seguente equazione⁸

$$y[n] = (a_0x[n]) + (a_1x[n-1]) + (a_2x[n-2]) - (b_1y[n-1]) - (b_2y[n-2]) \quad (11.7)$$

dove si vede la presenza di due ritardi di campioni di *ingresso* e due ritardi di campioni di *uscita* con i loro relativi coefficienti.

Le equazioni che troviamo nell'help di *biquad*~ possono apparire un po' diverse, ma sono di fatto riconducibili alla precedente. Le riportiamo qui per comodità

$$y[n] = ff_0w[n] + ff_1w[n-1] + ff_2w[n-2]$$

⁸Così, ad esempio, in Curtis Roads, *The Computer Music Tutorial*, MIT Press, 2000, pag. 412 che, a sua volta, cita i lavori di Rabiner e Gold, *Theory and Applications of Digital Signal Processing*, Prentice-Hall, 1975.

$$w[n] = x[n] + fb_1w[n-1] + fb_2w[n-2]$$

E' utile fornire al lettore qualche spiegazione sul tipo di notazione usata.

- I coefficienti *ff* e *fb* stanno rispettivamente per *feedforward* e *feedback*. La ragione di questo è dovuta al fatto che questo filtro presenta, ad un tempo, una componente *non ricorsiva* ed una *ricorsiva*. La prima ha la particolarità di essere un flusso di segnali impostati su un unico verso direzionale, da ingresso ad uscita, ossia i campioni di ritardo sono sempre degli 'ingressi' che vanno verso l'uscita. Per questo sono detti essere 'orientati in avanti', da cui il termine *feedforward*. Nel secondo caso, invece, i campioni di ritardo riguardano una o più uscite precedenti, attuandosi cioè un ritorno indietro nel tempo (ossia da un'uscita ad un ingresso). Da qui il termine *feedback*. Nella equazione 11.7 i coefficienti *ff* sono, più semplicemente, scritti a_n , mentre gli *fb* sono scritti b_n .
- La variabile $w[n]$ è essenzialmente fittizia in quanto possiamo rendere tutto secondo le consuete variabili y per l'uscita ed x per l'ingresso. Intuitivamente la prima equazione ci dice che una certa uscita è ottenuta tramite un ingresso *più* i due campioni immediatamente precedenti. E' la parte non ricorsiva del filtro. La seconda equazione ci dice che una certa uscita è ottenuta con un ingresso a cui si aggiungono due campioni ritardati di precedenti uscite, $w[n-1]$ e $w[n-2]$. Ed è la parte 'ricorsiva' del filtro.

L'oggetto *biquad~* possiede un unico ingresso cui va inviato, ovviamente, il segnale audio che intendiamo sottoporre a filtro. Inoltre si aspetta come argomenti (o tramite un messaggio da inviare all'unico inlet) il set dei cinque coefficienti corrispondenti a *fb1*, *fb2*, *ff1*, *ff2* ed *ff3* che, nella nostra 11.7, sono quelli relativi alle lettere *a* e *b*. Come per altri filtri possiamo inviare specifici messaggi quali: *set* per impostare un particolare stato interno e *clear* per azzerare una precedente configurazione.

11.6 Esempio di impiego di filtri grezzi

Prendiamo ora in esame e discutiamo una tipologia di filtro, presente nelle patch di corredo a PD, realizzata da Puckette.

11.6.1 Filtro Shelving

Per realizzare questo filtro (vedi fig. 11.9) sono stati messi in serie gli oggetti *rpole~* ed *rzero~* che, come si è detto, sono rispettivamente filtri ad un polo e ad uno zero. Tramite i rispettivi inlet di destra è possibile impostare adeguatamente i loro coefficienti che accettano valori da -1 a 1.

Come la fig. 11.9 fa intuire, i box numerici che gestiscono i coefficienti possono variare da -100 a 100, ma sono poi divisi per 100. Una sorgente di rumore bianco, come *noise~*, invia il suo segnale al banco di filtri disposti in serie: prima *rpole~* poi *rzero~*, il primo gestisce il polo e il secondo lo zero. E' utile predisporre un fader sul guadagno generale impostato a valori molto bassi per il controllo delle ampiezze generali della patch. Infatti, variando 'a caso' i coefficienti di poli e zeri si possono ottenere risposte con ampiezze anche molto grandi. Sempre in fig. 11.9 è riportato un grafico della risposta in frequenza del filtro con i coefficienti indicati (78 e 15). Per ottenerlo ci siamo serviti dell'utilissima patch dell'help di PD, *H11.shelving* che, pur non avendo un'uscita sull'audio, consente di monitorare il comportamento di un filtro Shelving al variare dei suoi coefficienti.

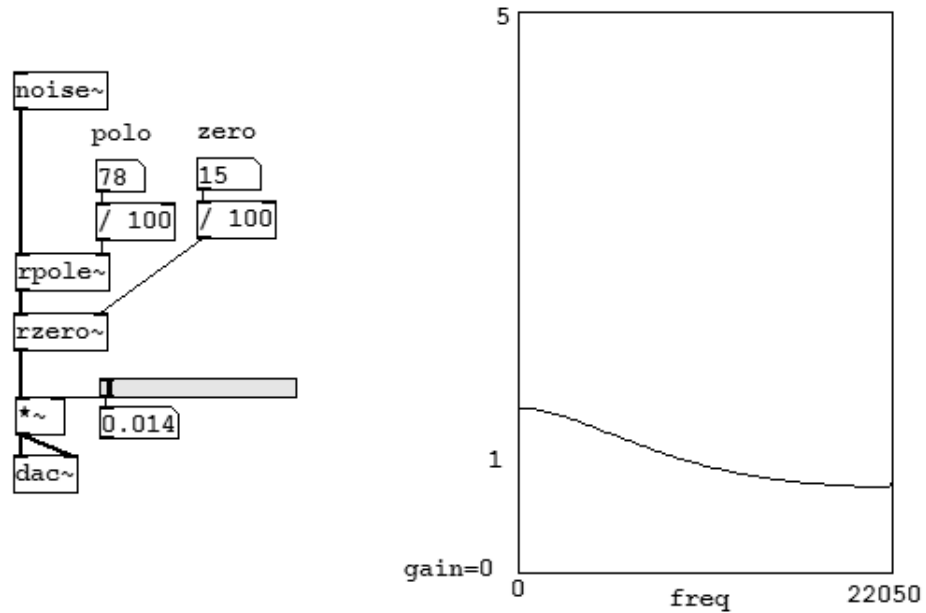


Figura 11.9:

In figura emerge la tipica curva di un filtro *Shelving* che opera all'estremità della banda di frequenza (qui è rappresentata solo la prima metà della banda totale, fino alla frequenza di Nyquist). In generale, in base al valore assunto dai coefficienti p e q , vale seguente relazione: il guadagno all'estremità sinistra della banda (DC) è dato da

$$\frac{1-p}{1-q}$$

mentre all'estremità destra (Nyquist) da

$$\frac{1+p}{1+q}$$

Più si sposta p verso valori alti (in senso assoluto), quindi verso 1 o -1, più si avranno alti guadagni, rispettivamente sul lato DC e sul lato Nyquist. Ma questa tendenza può essere contrastata da quanto q è vicino a p . Più è vicino e più la risposta tende ad essere piatta.

A titolo di esempio riportiamo alcune tarature (fig. 11.10):

- a: con $p = 3$ e $q = 84$ e
- b: con $p = 73$ e $q = 9$

dove si vede la differente risposta del filtro laddove p è basso (vicino allo 0) o alto.

In un'altra taratura (fig. 11.11) si ha

- c: con $p = 38$ e $q = 35$ e
- d: con $p = -61$ e $q = 27$

dove si vede l'effetto ottenuto con i coefficienti molto vicini tra loro, con la risposta appiattita. Se $p = q$ la risposta è completamente piatta.

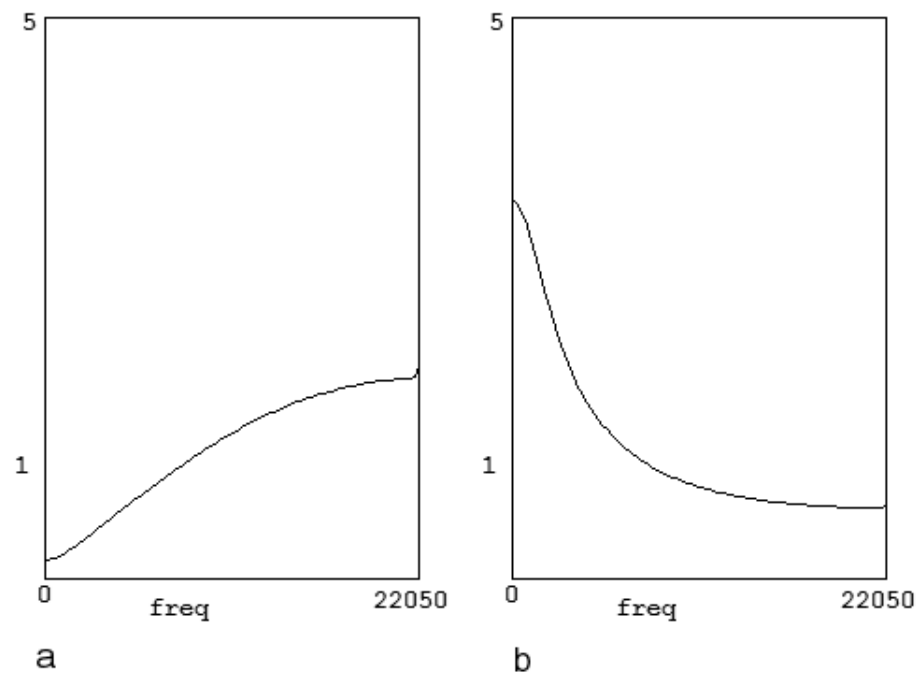


Figura 11.10:

11.7 Abbiamo studiato

`hip~`, `env~`, `lop~`, `bp~`, `noise~`, `vcf~`, `rzero~`, `rezero_rev~`, `rpole~`, `biquad~`, `f2note`
Messaggio: `clear`

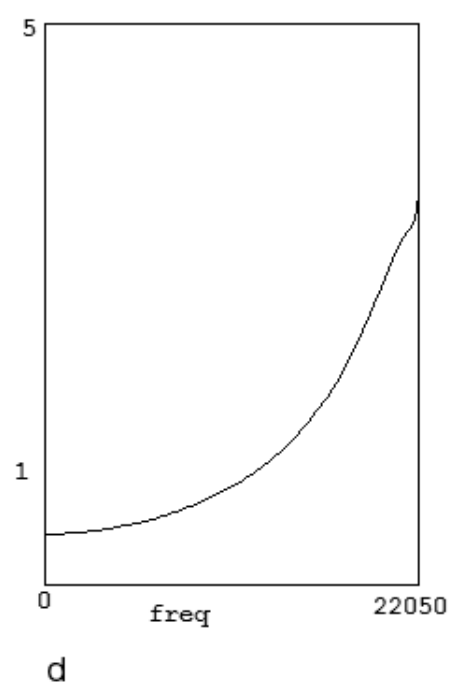
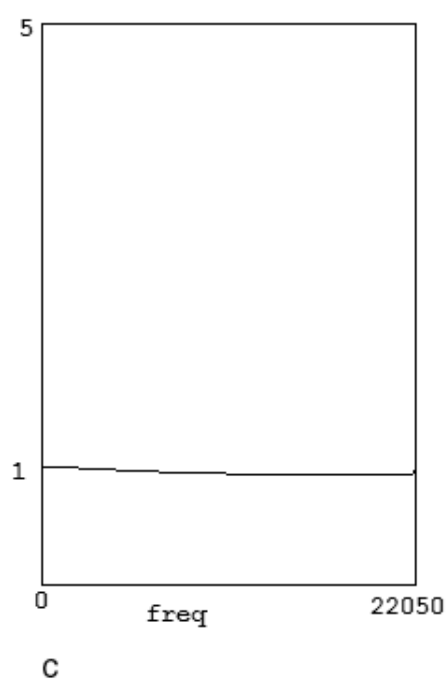


Figura 11.11:

Capitolo 12

Ritardi

Il lettore che abbia seguito la trattazione del capitolo precedente sui filtri, troverà l'argomento di questo capitolo del tutto affine. Infatti come quelli erano basati su linee di ritardi di campioni, gli oggetti di cui qui si tratta sono anch'essi dei dispositivi che implementano ritardi di campioni. In prima istanza li useremo - per così dire - ad *alto livello*, per replicare una porzione di segnale, opportunamente immagazzinata in una certa area di memoria, e rileggerla contestualmente al segnale primitivo creando un effetto di prolungamento del suono che tenderà a diventare quasi un'eco con particolari tempi di *delay*. Successivamente, usati più a *basso livello*, ci accorgeremo che essi permettono di implementare vere e proprie tipologie di filtri e il risultato percettivo sarà di tutt'altra natura. Anche in questo caso dovremo introdurre un po' di notazione matematica per meglio comprendere la natura di questo dispositivo e il perché del suo funzionamento sul piano del risultato sonoro. Ma andiamo con ordine.

12.1 Gli oggetti *delwrite~* e *delread~*

Pd mette a disposizione due oggetti per realizzare linee di ritardo, *delwrite~* e *delread~*. Il primo, come si evince dal suo nome, serve per 'scrivere' ossia memorizzare una certa quantità di ritardo, il secondo per 'leggere' quella porzione in memoria, tutta o una parte di essa.

In particolare, **delwrite~** consta di

- un ingresso in cui riceve un segnale audio che memorizza in una locazione di memoria stabilita dall'argomento e
- due argomenti: il primo stabilisce il nome della locazione di memoria, dunque del ritardo (e questo nome verrà usato da uno o più oggetti *delread~*); il secondo la quantità di tempo (in millisecondi) di memoria da allocare. Questa quantità significa il massimo di tempo che l'oggetto *delread~* potrà usare. E' il caso di ricordare che noi impostiamo l'oggetto in termini di tempi (a noi più congeniale), ma in realtà verrà presa traccia di un ammontare di campioni equivalente a quel tempo.

Dal canto suo **deread~** ha

- un ingresso in cui si può specificare, tramite un numero, quanta parte (anche tutta) della linea di ritardo si vuole usare. La lettura avviene sempre su numeri interi di campioni (non vi è alcuna interpolazione). Se chiediamo a *delread~* di usare più tempo (di memoria) di quanto *delwrite~* ha allocato, la sequenza verrà troncata laddove finisce la memoria; e

- due argomenti il primo è il nome della linea di ritardo cui l'oggetto si riferisce. Deve cioè riferirsi al suo *delwrite~* (ossia dello stesso nome) con la precisazione che vi possono essere più *delread~* per un solo *delwrite~*. Il secondo è il tempo di ritardo espresso in millisecondi. Ovviamente si tratta di un valore iniziale da aggiornarsi a piacere tramite l'ingresso.

Facciamo un'avvertenza preliminare riservandoci di spiegare meglio più avanti. Può capitare - come vedremo in una prossima esercitazione - che un oggetto di lettura sia posto prima di *delwrite~* per dar luogo a ciò che si chiamano linee di ritardo *ricorsive* (o con feedback). Questo termine, che già conosciamo dalla teoria dei filtri, sta a significare che nella linea entra lo stesso segnale processato, attuandosi un ritorno indietro del flusso. In tali casi il ritardo minimo non è 0 ma un intero blocco di campioni che, se non diversamente specificato, è di 64, valore di default in PD. E' possibile modificare questo numero tramite un apposito oggetto (*block~*) di cui parleremo tra breve.

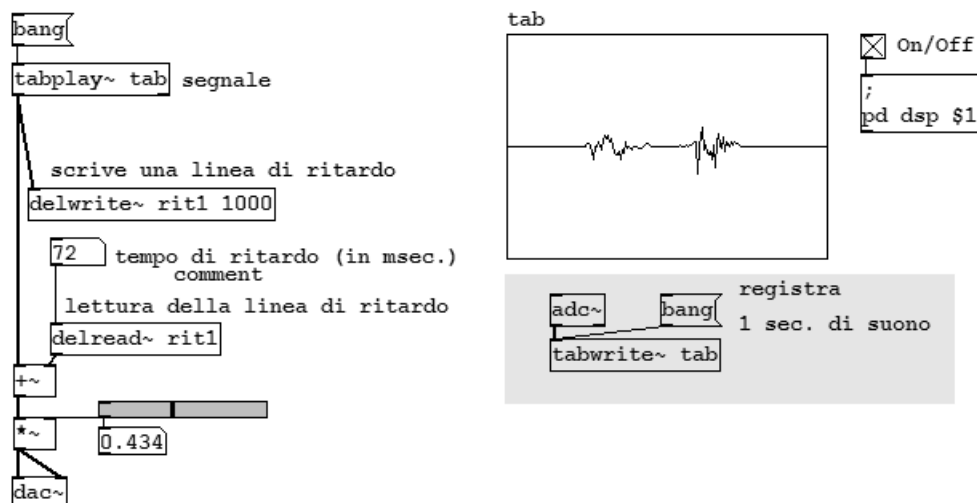


Figura 12.1:

La figura 12.1 illustra una semplice patch in cui sono presenti i due oggetti studiati. Trascuriamo di commentare la parte destra che è di ovvia comprensione. Sulla sinistra un oggetto *tabplay~* attiva la lettura della tabella *tab* preventivamente riempita di un segnale. L'uscita da *tabplay~* viene sdoppiata in due: una di esse entra in *delwrite~* che scrive una linea di ritardo dal nome *rit1* e della durata di 1 secondo. Successivamente l'oggetto *delread~* - di cui è possibile controllare il tempo di ritardo (nell'esempio 72 msec.) - richiama una parte della memoria allocata (72 msec.) e tale flusso viene sommato con l'oggetto *+~* al flusso del segnale originale proveniente da *tabplay~*. Il lettore può divertirsi a far scorrere il controllo di *delread~* producendo vari effetti di 'profondità' del suono fino ad ottenere una rudimentale eco.

12.2 L'oggetto *vd~*

Come *delread~* anche *vd~* legge un segnale da una linea di ritardo con la differenza, però, che in questo caso lo fa in modo più sofisticato: può leggerlo ad un tempo variabile servendosi di un processo di interpolazione a 4 punti. Anche in questo caso deve avere un suo corrispondente

delwrite~ di cui deve richiamare il nome. E questo nome rappresenta anche l'unico argomento. Infatti, mentre *delread~* aveva anche l'argomento del tempo di ritardo in lettura, in questo caso non avrebbe senso: la forza di *vd~* è proprio nell'avere tempi di lettura variabili. Tale informazione gli viene dunque inviata nell'input e potrà essere un segnale audio.

Come già detto, nel caso in cui *vd~* sia posto a monte di *delwrite~*, per realizzare dei feedback, il ritardo deve sottostare, come dimensione minima, a quella del blocco di campioni processati tutti insieme da Pd (di default 64).

12.3 Ordine di computazione della coppia *lettura/scrittura*

Non è per pignoleria soffermarsi a studiare più da vicino cosa succede nei due possibili ordini di computazione: a) la scrittura della linea di ritardo avviene *prima* della lettura; b) la scrittura avviene *dopo* la lettura. Il motivo di ciò sarà chiaro dopo un breve ragionamento. Ricordiamo che Pd computa l'audio attraverso pacchetti o blocchi di campioni che per default ammontano a 64 (ma è un valore modificabile).

12.3.1 Scrittura *prima* della lettura

E' il caso più intuitivo. Supponimo, per pura comodità, di avere un blocco B di 4 campioni, limitandoci a questi perchè vogliamo studiare la più piccola dimensione che può assumere il ritardo. Chiameremo inoltre N il primo campione del blocco. Se mando in scrittura (a *delwrite~*) la mia sequenza, avrò

$$x[N], x[N + 1], x[N + 2], x[N + 3]$$

dove, con N+3 ho finito il mio blocco, in quanto 1+3=4. Con un successivo oggetto di lettura, ad esempio *delread~*, potrò recuperare quella medesima sequenza, ossia

$$x[N], x[N + 1], x[N + 2], x[N + 3]$$

Detto ancora in altro modo. Immaginiamo che il primo campione di B sia il centesimo: avremo in scrittura 100, 101, 102, 103. In lettura saranno sempre gli stessi campioni, quindi 100, 101, 102 e 103.

12.3.2 Scrittura *dopo* la lettura

In questo caso i campioni successivi ad $x[N]$ non sono stati memorizzati nella linea di ritardo proprio perchè, quando *delwrite~* scriveva, non erano ancora stati letti. Ne segue che i campioni più recenti su cui operare si riferiscono a quelli del blocco precedente. Per cui si può affermare, sempre per $B = 4$, che *delread~* legga da $x[N-B]$ a $x[N-1]$ essendo il campione $x[N]$ quello 'attuale'. Leggerà cioè

$$x[N - 4], x[N - 3], x[N - 2], x[N - 1]$$

Quando verrà invocato l'oggetto di scrittura, vi si scriverà pur sempre

$$x[N], x[N + 1], x[N + 2], x[N + 3]$$

Ora scrittura e lettura sono diverse. Usando ancora l'esempio precedente, quando andrò a leggere la sequenza avrò: 96, 97, 98, 99. Ma noi ci troviamo, come minimo, a 100. Dunque

non potrò avere un ritardo inferiore a 4 campioni. Il minimo ritardo ottenibile corrisponderà alla dimensione del blocco B.

Ciò pone dei limiti ai tempi di ritardo che si possono scegliere. Ad esempio se B, com'è il caso più tipico, è 64 samples ciò corrisponde, con una frequenza di campionamento di 44100 Hz, a $64/44100 = 1.45$ msec. A prima vista si tratta di una durata temporale davvero piccola eppure non è così. In alcuni casi, per poter 'intonare' nel modo voluto una linea di ritardo usata come filtro *comb* (ce ne occuperemo nella sezione successiva), si tratta di un valore molto alto che fra poco quantificheremo.

12.4 Realizzare un filtro *comb* con una linea di ritardo

Come si è detto all'inizio di questo capitolo l'aspetto forse più interessante, anche se meno intuitivo, di una linea di ritardo non è quello di produrre effetti simili a riverberi o echi quanto piuttosto di produrre un dispositivo di filtraggio. In questa sezione realizzeremo un particolare tipo di filtro, di cui si è già accennato in 11.3.4, ossia i filtri *comb* o *a pettine*. Ci serviremo di una patch di Miller Puckette con cui viene messa in primo piano la proprietà tipica di un filtro *comb*, di produrre una risposta che enfatizza una certa serie ordinata di frequenze e ne annulla altre. Ciò servirà, in breve, a produrre una trasposizione d'ottava del nostro suono originale. Ma prima è necessaria una breve appendice matematica che riprendiamo dal citato testo di Miller Puckette¹.

12.4.1 Appendice matematica

Riprendendo un concetto accennato nel capitolo precedente, possiamo rappresentare un caso particolare in cui l'uscita di una rete di ritardo è la somma di un input con una sua copia ritardata. Ossia, usando la notazione complessa

$$Z^n + Z^{n-d}$$

intendendo per d un 'certo numero' di campioni di ritardo. Mettendo in evidenza il termine comune, la formula può risciversi

$$(1 + Z^{-d})Z^n$$

Il primo fattore può intendersi come un numero complesso H (per il quale si moltiplica Z^n) che varia² in funzione di una certa frequenza ω . In tal caso abbiamo assunto che

$$H = 1 + Z^{-d} \quad (12.1)$$

Studiando H , cioè come esso varia in funzione della frequenza ω , significa comprendere anche il comportamento (nel dominio della frequenza) della rete di ritardo.

Ma cosa vuol dire aggiungere 1 a Z^{-d} ? Possiamo aiutarci tramite una rappresentazione grafica dove vediamo, nel piano complesso, dove si collocano rispettivamente 1, Z^{-d} e anche H (che qui è la loro somma).

In fig. 12.2 osserviamo che 1 si trova sull'asse x, in quanto numero reale (la parte immaginaria è 0), mentre Z^{-d} , dotato di una parte immaginaria³, si trova in un certo punto della circonferenza che delimita il piano complesso (se ne può constatare la componente angolare

¹M. Puckette, op. cit., pag. 174 - 176.

²Ricordiamo infatti che ogni numero complesso è individuato, tra l'altro, da un certo angolo in un piano, mentre un numero reale è individuabile su una linea retta. Vedere la fig. 12.2.

³In particolare la parte reale sarà $\cos(-d\omega)$ e la parte immaginaria $\sin(-d\omega)$.

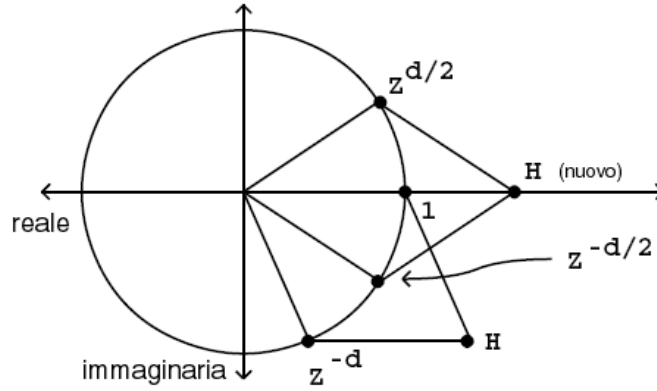


Figura 12.2:

ω rispetto⁴ all'asse destro dell'asse x). La loro somma, che è H , si può rappresentare come il quarto vertice di un parallelogramma che ha per altri vertici 0, 1 e Z^{-d} . Ovviamente la collocazione di H dipenderà dalla *grandezza*⁵ dei 2 numeri complessi originari.

Per semplificare le cose possiamo traslare, in senso antiorario, di metà angolo ω ottenendo rispettivamente, al posto di 1 e Z^{-d} , i numeri $Z^{d/2}$ e $Z^{-d/2}$ anch'essi rappresentati in figura. A questo punto la loro somma è un H che troviamo sull'asse x. Si può dunque riscrivere la 12.1 in questo modo

$$H = Z^{-d/2}(Z^{d/2} + Z^{-d/2}) \quad (12.2)$$

In essa il primo termine individua la traslazione angolare (cioè della fase) di $-d\omega/2$. Il secondo termine, usando la notazione rettangolare⁶ per ciascun numero complesso, può essere riscritto come

$$(\cos(\omega d/2) + i \sin(\omega d/2)) + (\cos(\omega d/2) - i \sin(\omega d/2))$$

dove le due componenti immaginarie si annullano e dunque si avrà

$$2\cos(\omega d/2)$$

che esprime una quantità che oscilla in valori positivi e negativi. Il suo valore assoluto ci fornisce la *grandezza* dell'uscita, ossia

$$|H| = 2 |\cos(\omega d/2)| \quad (12.3)$$

La quantità $|H|$ - che, lo ricordiamo, è la somma di due sinusoidi di uguale ampiezza e la cui fase differisce di un valore ω - è ciò che *esce* dal ritardo, ossia il suo guadagno ad una certa frequenza angolare. Tale quantità è detta *risposta in frequenza*. In sostanza il guadagno dipende dalla frequenza: varierà al variare di quella. E come? Secondo l'oscillazione descritta dalla fig. 12.3⁷ dove si nota la tipica forma di una cosinusoide in valore assoluto.

⁴Più precisamente tale numero ha una parte reale $\cos(-d\omega)$ e una complessa $\sin(-d\omega)$.

⁵Per *grandezza* di un numero complesso si intende la distanza, nel piano complesso, dall'origine al punto a, b che individua quel numero.

⁶Per forma *rettangolare* di un numero complesso si intende il modo di rappresentarlo tramite una componente reale ed una immaginaria. Si contrappone alla forma *polare*.

⁷Anch'essa presa da Puckette cit., pag. 176.

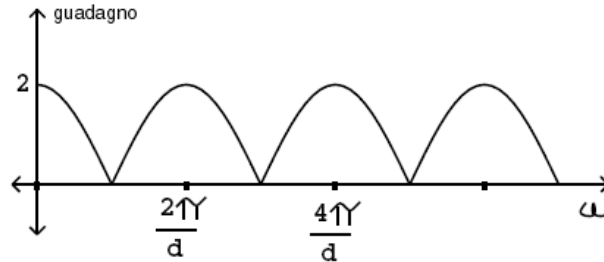


Figura 12.3:

A questo punto si comincia anche a capire in che senso la rete di ritardo che stiamo studiando rappresenta un filtro *comb*. Infatti, come un tale filtro, per certi valori il guadagno si azzera per altri viene addirittura enfatizzato. E ciò avviene secondo una ben precisa regolarità come si osserva dalla figura. Per ogni periodo completo (2π) di ω , partendo da 0, il picco è massimo mentre per ogni periodo completo a partire da π il valore è zero. Il valore di ω sull'asse delle ascisse, in quanto periodo, è di fatto l'inverso della frequenza, ossia una certa quantità di tempo. La figura insomma ci dice che per ritardi che assommano a particolari quantità di tempo (che nella musica digitale è dato da una certa quantità di campioni) la risposta è zero oppure è massima.

Per maggiore chiarezza possiamo ripetere il ragionamento in altro modo. Porre che una quantità di ritardo d sia un periodo completo, ossia $2\pi/\omega$, significa dire che $\omega = 2\pi/d$: a queste impostazioni corrispondono i valori massimi. Poniamo allora di avere un certo segnale periodico la cui frequenza fondamentale sia ω radianti per ogni campione, ossia avanzi ogni volta di un valore ω in una circonferenza ideale. Il suo periodo sarà $1/\omega$. Possiamo allora settare il nostro filtro, con il procedimento appena descritto, in modo che i picchi si abbiano solo per i numeri pari delle armoniche (2, 4, 6, ecc.), mentre per le armoniche dispari vi siano degli zeri. Per ottenere questo dobbiamo scegliere $d = \pi/\omega$ ossia un ritardo che sia metà del periodo. In queste condizioni il segnale che abbiamo prodotto (quello in uscita dalla rete di ritardo così impostata) avrà le parziali 2π , 4π , 6π , ecc., ossia avrà una frequenza fondamentale doppia dell'originale. In un parola, il segnale sarà intonata un'ottava sopra.

12.4.2 Trasporre di un'ottava con un ritardo dinamico

Possiamo ora procedere a studiare la patch *G06.octave.doubler* presente nella documentazione di PD che riportiamo leggermente modificata per una maggiore comprensione (fig. 12.4).

Dopo aver acceso l'audio procediamo per prima cosa a registrare con un microfono un breve campione audio di 1 secondo. All'uopo provvede la parte in alto a destra della patch che registra il segnale, con *tabwrite~*, nella tabella di nome 'tabella' (si vedrà la finestra relativa riempirsi di una forma d'onda).

In alto a sinistra l'oggetto *phasor~* produce una rampa opportunamente scalata da 1 a 44101: si tratta di una scala di indici che vengono mandati in ingresso a *tabread4~* che in tal modo legge la sequenza registrata in 'tabella'. A questo punto il lavoro della lettura è compiuto e infatti una copia del segnale in uscita viene mandato varso l'oggetto *dac~* preventivamente passando per una subpatch di nome *pd* che ha il semplice compito di effettuare uno switch on/off sul segnale originale. Uno analogo si ha alla sua destra per il segnale processato. In tal modo ci riserviamo la possibilità di switchare tra i due segnali e coglierne la differenza.

Veniamo ora alla parte più direttamente interessata al processamento. Per prima cosa il

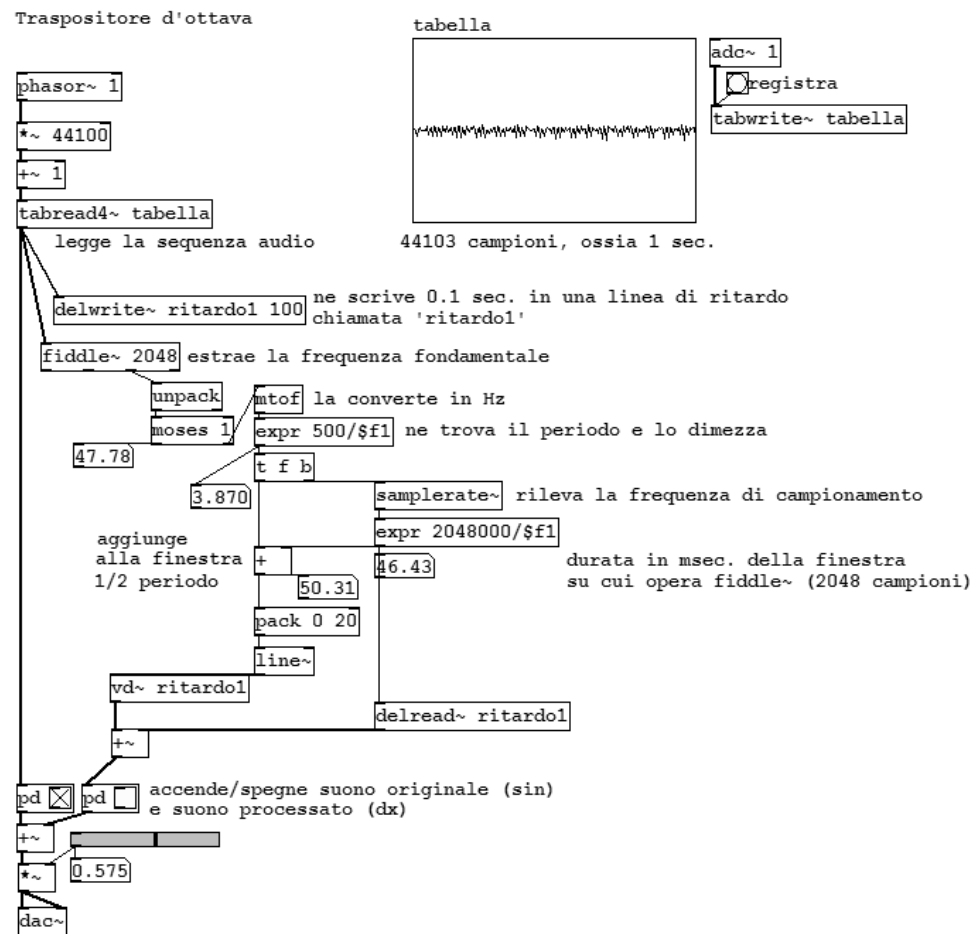


Figura 12.4:

segnale in uscita dalla lettura della tabella viene inviato a *delwrite~* e all'oggetto *fiddle~*. Di quest'ultimo non abbiamo ancora parlato e, in questa sede, ne faremo un accenno solo quanto basta per coglierne la funzione all'interno di questa patch. Verrà esaminato con più profondità nel capitolo successivo. Più in dettaglio esaminiamo punto per punto:

delwrite~ scrive su una linea di ritardo, chiamata 'ritardo1', 100 msec. di suono, ossia un decimo di secondo. Questa quantità è verosimilmente adeguata per gli usi che ne dovranno fare i corrispettivi oggetti di lettura (*delread~* e *vd~*). Dal canto suo

fiddle~ è un oggetto dal compito assai interessante in quanto effettua un rilevamento del segnale in ingresso estraendone la frequenza fondamentale. L'argomento 2048 ci dice che l'oggetto effettua un'analisi continua su pacchetti o, come si dice, su una finestra di 2048 campioni. Inoltre l'oggetto presenta (in questo contesto) 4 outlets sui quali ora non ci soffermeremo se non dicendo che il terzo (da sinistra) manda in uscita due informazioni sul segnale in ingresso: la frequenza rilevata e la sua ampiezza. Per tale motivo a valle di *fiddle~* è posto un oggetto *unpack* che ha il compito di dividere (spacchettare) questo pacchetto di coppie

di dati. Interessandoci solamente l'informazione relativa alla frequenza, e non l'altra, la patch utilizza solo l'uscita di sinistra di *unpack*. Che entra in

moses tarato su 1. Perché? La ragione è semplice: quando *fiddle~* non riesce ad effettuare un rilevamento o, detto con più precisione, non trova una frequenza significativa che rientra nei parametri su cui è chiamato a lavorare, emette degli zero. Questi zero sarebbero un inutile carico di informazioni e devono essere filtrati. Ecco il compito di *moses*, di argomento 1: così impostato fa passare a destra solo i dati numerici che siano 1, o superiori, mentre a sinistra vanno i valori inferiori. Ma ovviamente a sinistra non è collegato a niente perché sono dati irrilevanti.

Va precisato che il valore di frequenza rilevato da *fiddle~* è un dato MIDI: ad esempio, se rileva un Do centrale emetterà 60 e non la frequenza effettiva del Do centrale. Ecco perché subito in cascata troviamo l'oggetto

mtof che provvede proprio ad effettuare l'opportuna conversione in Hz. Ma, ricordando quanto spiegato nella sezione precedente, a noi interessa rilevare la lunghezza del 'periodo' momento per momento del segnale in ingresso; ecco perché viene effettuato un calcolo con

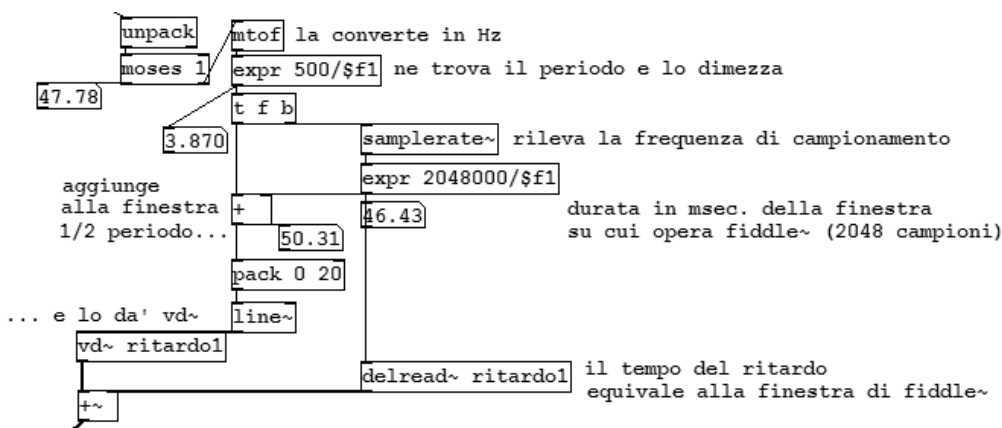
expr Perché $500/\$f1$? Per cominciare, nella variabile $\$f1$ entra il valore della frequenza rilevata da *fiddle~*. Ora il fatto che si trovi al denominatore è dovuto alla relazione che lega, appunto, frequenza e periodo, essendo l'uno il reciproco dell'altro. Infatti $1/f$ ci dà il periodo. Ma il valore in uscita sarebbe in secondi, noi vogliamo avere invece un valore in millisecondi, da qui la necessità di moltiplicarlo, verrebbe da dire, per 1000. Il fatto che invece qui viene moltiplicato per 500 è perché si vuole avere il valore di metà periodo, secondo quanto argomentato nella sezione precedente. Si ricorderà infatti che, per avere una trasposizione d'ottava, è necessario impostare una linea di ritardo su una ben precisa misura di questo ritardo che è appunto di metà periodo del segnale da processare. In definitiva a valle di *expr* esce un numero esprimente la durata in millisecondi di metà periodo del segnale di volta in volta rilevato in ingresso alla patch. Più precisamente questo valore entra in un oggetto

t f b Si tratta di un trigger che, a fronte di un segnale in ingresso, manda in uscita due informazioni. A sinistra, la *f* sta per numero reale, ossia riproduce il numero uscito dal calcolo di *expr*. A destra, la *b* significa che contestualmente viene prodotto un bang il cui compito è di attivare un successivo oggetto.

A questo punto il flusso si scinde in due percorsi che cerchiamo di ricostruire servendoci di una selezione della patch (fig. 12.5).

Percorso che va a *delread~* Questo percorso è finalizzato alla lettura *blocco per blocco* del segnale originale al quale poi, in base alla teoria studiata, sommeremo un ritardo equivalente a mezzo periodo del segnale rilevato. In sostanza, quando viene azionato lo switch (nella subpatch *pd*) che manda in play il segnale processato, non ascoltiamo il segnale - per così dire - in diretta, ma quello che è stato immagazzinato nella rete di ritardo *delwrite~* dal nome 'ritardo 1'. E quanto segnale verrà rilasciato? Di volta in volta verrà rilasciata una parte equivalente alla finestra di analisi su cui opera *fiddle~*. Noi sappiamo che tale finestra è di 2048 campioni, come risulta dall'argomento dell'oggetto. Dobbiamo allora convertire un numero esprimente *campioni* in un numero esprimente un *tempo*, nella fattispecie dei millisecondi. Ecco allora ciò che accade a valle del trigger *t*.

Il bang aziona l'oggetto *samplerate~* che ha il compito di rilevare l'attuale frequenza di campionamento (normalmente 44100 Hz). Questo valore viene passato all'espressione sottostante,



una divisione, come denominatore. Al numeratore invece c'è il numero 2048000 ossia il tempo, espresso in millisecondi (2048*1000), relativo alla finestra di analisi di *fiddle*~. Per sapere a quanto equivalgono 2048 campioni bisogna infatti effettuare una divisione. Ogni campione, alla frequenza di campionamento di 44100 (campioni al secondo), ha una durata temporale di 1/44100 secondi. Questo dato, moltiplicato per 2048, ci dice quanto dura, in secondi, la finestra di analisi. Quindi

$$\frac{1}{44100} * 2048$$

Se infine vogliamo esprimere tale durata non più in secondi, ma in millisecondi, si dovrà moltiplicare per 1000. E questa moltiplicazione è già contenuta nel dividendo che è $2048 \cdot 1000 = 2048000$.

Il valore uscente da tale calcolo costituisce dunque la durata del ritardo su cui impostare $\text{delread}^{\sim}$. In figura si osserva che $\text{delread}^{\sim}$ legge i dati in essa memorizzati nella linea con un ritardo di 46.63 msec.

Il flusso in uscita da $delread^{\sim}$ converge infine in un oggetto somma $(+\sim)$ che effettuerà la somma con l'altro flusso di dati, quello che proviene dalla parte sinistra della patch, contenente il segnale processato, più esattamente traslato di fase (di mezzo periodo). Andiamo allora ad esaminare la porzione di patch che si occupa di questo.

Percorso che va a $vd \sim$ A valle del trigger (t) abbiamo questa volta un oggetto somma che serve appunto a sommare due valori temporali

1. il tempo, calcolato in precedenza, relativo a metà periodo di segnale (di volta in volta estratto da *fiddle*[~]). In figura risulta essere 3.87
2. la durata della finestra di analisi di *fiddle*[~] come esce da *samplerate*[~] e dall'espressione sottostante (nel nostro esempio è 46.43).

La successiva coppia *pack* e $line \sim$ serve solo, com'è intuibile, a smussare gli effetti di una taratura della lettura di ritardo $vd \sim$ estremamente mutevole nel tempo.

I due percorsi si riunificano, infine, tramite un oggetto somma con cui, di fatto, un segmento temporale di segnale *non* processato viene combinato con un campione di segnale a cui si è aggiunto (ritardo) un pezzo equivalente a mezzo periodo del segnale, come rilevato da

fiddle~ (in figura quei 3.87 msec.). Il risultato finale è che, per effetto della traslazione di fase, percepiamo un suono trasposto di un'ottava *sopra* l'originale.

Si intuisce anche il perchè dell'uso diversificato di *delread*~ e *vd*~. Nel primo caso l'oggetto è impostato in modo 'fisso' su un valore che non cambia⁸. Nel secondo caso il valore cambia presumibilmente a grandissima velocità (a seconda del *periodo* della nota di volta in volta prescelta) e ciò richiede l'uso di *vd*~.

12.5 Linee di ritardo come algoritmi di sintesi

Poichè stiamo indagando alcuni aspetti meno intuitivi del tema "linee di ritardo", è il caso di fare un cenno a come esse possono essere impiegate per la implementazione di particolari tecniche di sintesi basate sui cosiddetti *modelli fisici*. Si parte dall'idea che uno strumento musicale può essere pensato, formalmente, come un sistema costituito da una *sorgente* (o eccitatore) e da un *risuonatore* capace di produrre una particolare gamma di parziali. Da un lato, cioè, abbiamo una corda, fissata agli estremi che viene pizzicata e indotta a produrre differenti modi vibratorii, dall'altro una cassa armonica - la cui proprietà, come sanno i musicisti, conferisce valore allo strumento - che fa da risuonatore, esaltando particolari aree formantiche e tagliandone altre. Tale risuonatore si può pensare come un filtro.

Questo principio, descritto in grandi linee, è alla base della sintesi di Karplus-Strong⁹. Più in dettaglio, viene generata una scarica di segnale ad ampio spettro come un rumore bianco (ma può essere anche un'onda quadra o a dente di sega) che viene poi sottoposta ad un filtraggio per bande, com'è tipico dei filtri a pettine o *comb*. Ciò significa, come ormai sappiamo, che il segnale viene inviato, oltre che in uscita, anche in una linea di ritardo di un certo numero N di campioni. L'uscita del filtro viene mixata con il suono al suo ingresso e quindi nuovamente reimpresso nella linea. Il modello è infatti un filtro ricorsivo.

Il risultato è che il suono originale tende progressivamente ad impoverirsi di frequenze alte, trattandosi di un filtro passa basso e venendo a somigliare, con buona approssimazione, al tipico suono di una corda pizzicata in rapida attenuazione. La qualità del suono dipende comunque dalla lunghezza del ritardo, N , e se esso è costante o variabile. Si può anche stabilire la frequenza d'uscita tramite la seguente relazione

$$F = \frac{SR}{N}$$

dove SR è la frequenza di campionamento ed N il ritardo in campioni.

12.5.1 Una patch di sintesi e uso di *block*~

La patch *G04.control.blocksize*, inclusa nella documentazione di PD, presenta un interessante esempio, almeno sotto il profilo didattico, di sintesi ottenuta con una linea di ritardo che ricorda l'algoritmo di Karplus-Strong. Vengono prodotti a caso, grazie all'uso di un oggetto *random* (parte destra della patch), dei valori di frequenza poi tradotti in suono tramite gli oggetti della parte sinistra. Un suono che ricorda quello di una corda pizzicata,.

Studiamo la patch nella nostra implementazione di fig. 12.6.

All'apertura della patch viene attivato (con *loadbang*) un oggetto *metro* che manda dei bang ogni 0.5 secondi. Due box messaggio vengono raggiunti dai bang opportunamente distanziati di un millisecondo (*del 1*). In particolare il bang invia il valore 1 all'oggetto *vline*~ che in questa patch è chiamato a creare la forma d'onda producendo la sorgente sonora. Dopo 1 millisecondo

⁸A meno di non cambiare la frequenza di campionamento, il rapporto tra 2048000 e 44100 rimane immutato.

⁹Si tratta, più esattamente, dei lavori di Alex Karplus e Kevin Strong dei primi anni '80.

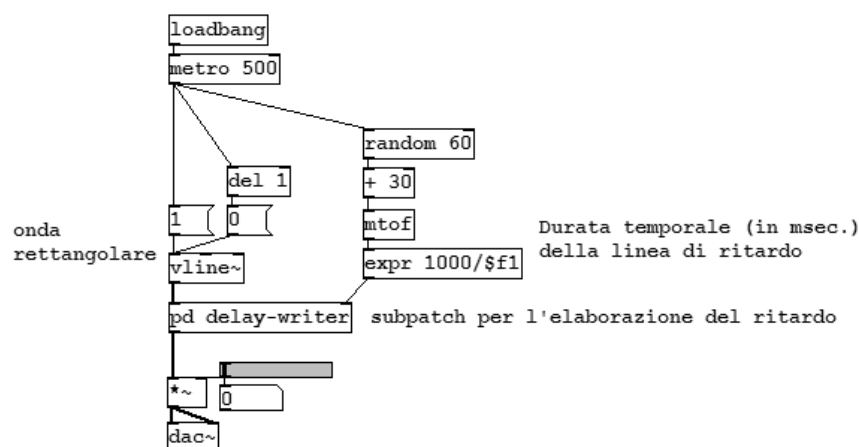


Figura 12.6:

(grazie all'oggetto *del* tarato ad 1) *vline~* è raggiunto dal messaggio che veicola uno 0. In sostanza si è creata una forma d'onda rettangolare, della durata complessiva di mezzo secondo, che presenta un rapido attacco iniziale, un sostegno di appena 1 millisecondo e una ricaduta a zero fino alla fine del periodo. Questo segnale (si tratta di un'onda rettangolare) entra nella subpatch di nome *delay-writer*.

Ma prima di entrarvi guardiamo la parte destra della patch dove un oggetto *random*, attivato sempre da un metronomo di 2 bang al secondo, produce dei valori numerici in un range da 0 a 60 che poi vengono sommati, con + 30 per modificare il range a 30 - 90. L'uscita viene inviata al convertitore *mtof* con cui quei valori, simulati come numeri di nota MIDI, divengono frequenze audio. Per convertire le frequenze in periodi - a noi serve una 'durata' per tarare il ritardo e non la frequenza - è sufficiente farne il reciproco. La moltiplicazione per 1000 serve per esprimere il tempo in millisecondi. Questo secondo valore entra nell'inlet di destra della subpatch.

E giunto il momento di guardarvi dentro (fig. 12.7).

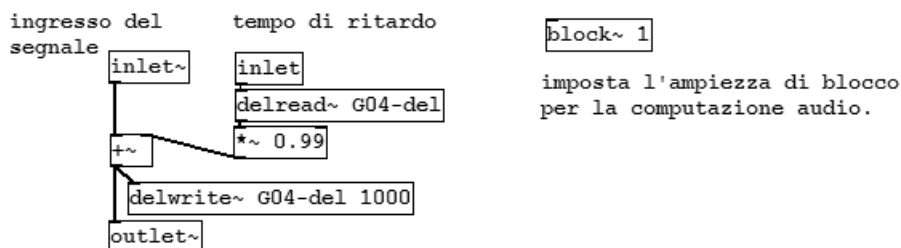


Figura 12.7:

Questa subpatch si occupa di effettuare la computazione del ritardo ricorsivo. Sulla parte sinistra vediamo i due inlet: uno per il segnale in ingresso e l'altro, più a destra, per l'impostazione del tempo di ritardo. Si nota che il segnale in ingresso viene sommato con una 'copia' di se stesso di N campioni precedenti. Questi N campioni sono calcolati sulla base

di quanto era il periodo di ciascuna altezza primitiva¹⁰. Ad esempio per un'altezza di 440 Hz (il La del diapason) si ha una durata di circa 2.2 msec. Il valore del ritardo viene prima moltiplicato per una costante, che nella fattispecie è 0.99.

Nella patch, in quanto implementa un sistema ricorsivo, l'ordine di computazione del filtro è del tipo esaminato in 12.3.2, ossia *prima* lettura *poi* scrittura. Come lì avevamo osservato, in questi casi la taratura del ritardo non può scendere sotto l'ammontare del blocco di computazione. Poichè nel caso di default esso è 64 si ha che il minimo ritardo possibile sarebbe di 64/44100, ossia di 1.45 msec. Da qui la necessità, per un impiego più flessibile, di impostare un valore di blocco inferiore. Per fare questo nella patch ci si è serviti dell'oggetto **block~** con argomento 1. Con questa taratura è possibile un ritardo addirittura di un singolo campione, ossia nell'ordine di 0.02 millesimi di secondo.

Ma vediamo i possibili argomenti dell'oggetto *block~* che sono tre, e precisamente:

1. ampiezza del blocco
2. *overlap*
3. sovra- o sotto-campionamento

Del punto 1 abbiamo già parlato. Quanto al punto 2, l'*overlap*, si tratta di procedimento per cui i campioni computati non vengono presi in sequenza blocco dopo blocco, ma il blocco successivo si compone di una parte (quella finale) di quello precedente. Da qui il termine che vuol dire 'sovrapposizione'. Se ad esempio abbiamo *block~* con argomento 512 e 4 significa che vengono computati blocchi di 512 campioni, ma ad intervalli di 128 alla volta. Ossia, in breve, a 0, a 128, a 256, a 384, a 512, ecc.

Il terzo argomento riguarda la possibilità di imporre un sovracampionamento (*up-sampling*) o, viceversa un sotto-campionamento, di un certo fattore. Ad esempio, aggiungendo agli argomenti precedenti un terzo valore, come 512 4 4, si impone un sovracampionamento di un fattore 4. Ciò può rivelarsi utile in molte circostanze, non ultimo quando vogliamo ridurre o estinguere gli effetti del fold-over. Ma di questo si parlerà più avanti. Quanto al sottocampionamento si introdurranno valori inferiori ad 1. Ad esempio 0.5 significa un sottocampionamento di fattore 2.

L'aspetto da tener presente è che quando si lavora con ampiezze di blocco molto piccole vi è un sensibile aggravio di risorse della CPU. Per questo motivo è utile impostare una sotto routine, ossia una sotto-finestra, che gestisca *solo* quella computazione per la quale è richiesta una dimensione di blocco piccola. Nel nostro caso gli impulsi che eccitano la linea di ritardo vengono computati a parte. Successivamente il segnale elaborato viene reinviato alla patch-madre e da qui mandato in uscita.

Già che siamo in argomento è utile citare anche un altro oggetto - analogo, ma più sofisticato di *block~* - che è **switch~**. Oltre ad impostare la dimensione del blocco di computazione audio, *switch~* permette di accendere o spegnere il DSP relativamente alla finestra in cui si trova. Ciò può comportare un notevole risparmio di risorse. E' sufficiente inviare nel suo inlet un 1 o uno 0 per accendere o, rispettivamente, spegnere la computazione. Si tenga presente che se vi sono altre finestre in subordine a quella dove alloggia *switch~* anche in quelle la computazione verrà regolata di conseguenza. Per ovvi motivi non può esservi più di un oggetto *block~* o *switch~* per finestra.

¹⁰Si tratta di un'altezza ancora virtuale, di cui si definisce (a *random*) la frequenza, ma che non è ancora diventata suono.

12.6 Abbiamo studiato

`delwrite~`, `delread~`, `vd~`, `block~`, `switch~`

Capitolo 13

Strumenti di analisi

Tra le più interessanti risorse rese possibili dalla potenza di calcolo degli attuali computer è senz'altro da considerare l'analisi del suono nelle sue componenti spettrali. Si può ottenere cioè, oramai in tempo (quasi) reale, la traccia dell'energia spettrale di un suono lungo tutto il range di frequenze udibili.

Stiamo parlando della cosiddetta *trasformata di Fourier* che, com'è noto, trasforma una qualunque forma d'onda continua e periodica in una serie infinita di onde sinusoidali semplici in rapporto armonico, ciascuna con una sua ampiezza e fase¹. Più precisamente nel mondo discreto, quello in cui operano i computer, si usa una sua variante, detta appunto *trasformata di Fourier discreta* (d'ora in poi DFT), che permette di procedere al calcolo tramite un numero finito (sempre comunque molto alto) di operazioni.

Ad una prima riflessione emergono subito due problemi: il primo riguarda il fatto che nel mondo reale praticamente mai si ha a che fare con suoni rigorosamente periodici per cui si impone di dirottare il procedimento analitico - senza perdere, ovviamente, in efficacia - verso

¹ La FT scompone il segnale in funzioni esponenziali complesse di differenti frequenze. In termini matematici:

$$X(f) = \int_{-\infty}^{\infty} x(t) * e^{-2j\pi ft} dt \quad (1)$$

dove:

t = tempo

f = frequenza

x = segnale

X denota, invece, il segnale nel dominio della frequenza.

La formula esprime il fatto che il segnale $x(t)$ è moltiplicato con il termine esponenziale ad una certa frequenza f e quindi integrato sopra tutto il tempo. Il termine esponenziale $e^{-2j\pi ft}$ può anche scriversi in notazione complessa:

$$\cos(2\pi ft) + j * \sin(2\pi ft) \dots$$

con una parte reale (coseno della frequenza) ed una immaginaria (seno di f). In sostanza si moltiplica il segnale originale con un'espressione complessa di seni e coseni della frequenza f . Quindi si integra il prodotto, ossia si 'aggiungono' a questo prodotto tutti i punti. Se il risultato del procedimento è un valore grande allora vuol dire che il segnale $x(t)$ ha, proprio in f , una componente spettrale dominante. Viceversa, se il segnale non ha una componente di f , il risultato tenderà a 0.

Poichè l'integrazione è sul tempo e va da $-\infty$ a $+\infty$ consegue che non importa *quando* viene a trovarsi la frequenza f : c'è e basta senza sapere quando. Ne consegue che la Trasformata di Fourier non è adatta per i segnali che variano nel tempo

segnali non periodici. Il secondo ha invece una natura logistica e si esprime valutando il carico di lavoro cui una macchina va incontro per giungere al risultato finale.

13.1 Un po' di teoria

Quanto al primo problema, la natura eminentemente dinamica del segnale (ossia continuamente variabile nel tempo) ha indotto a forzare una periodicità che non c'è, ottenendola artificialmente. In pratica si procede in questo modo: il segnale prima di essere sottoposto al processo di analisi viene *finestrato*, ossia 'rotto' in tante brevi finestre temporali (equispaziate) e su di esse si applica una certa curva di *involuppo*.

Consideriamo la sequenza di operazioni

1. si taglia una sezione di qualche decimo di secondo di segnale audio. Più precisamente preleviamo, in sequenza, pacchetti di un certo numero N di campioni. Chiameremo questo pacchetto *ampiezza di blocco* (*blocksize* o anche *FFT-size*). Poniamo, ad esempio², $N = 1024$. Dovremo assumere che questo pacchetto sia periodico, ma siamo (quasi) certi che non è così: l'inizio e la fine presenteranno valori casuali e non uno *zero-crossing*, ossia quei punti estremi non inizieranno e termineranno a 0 con regolarità periodica. In queste condizioni l'analisi produrrebbe risultati inconsistenti. Quindi
2. si rende periodico ciascun segmento così ottenuto, moltiplicando ogni suo punto (ogni n -esimo *campione*) per una curva particolare, scelta in modo che costituisca un involuppo completo, con tanto di attacco (a partire da zero) e decadimento (fino a zero). L'esempio forse più tipico di tale involuppo è la cosiddetta finestra di Hann, come si vede in figura 13.1. Dopo aver finestrato tutto il nostro suono

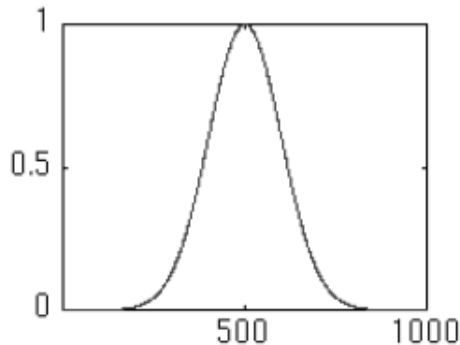


Figura 13.1:

3. si esegue una Trasformata di Fourier per ciascuna finestra. Per via di questa applicazione a piccolissime porzioni di campioni deriva il nome più preciso di *Short-time Fourier Transform* (STFT). La matematica alla base di questo procedimento va aldilà degli scopi di questo libro. E' sufficiente per noi sapere che per ogni blocco di N campioni - finestrati e periodizzati - l'applicazione di una DFT produce una lista di N coppie di valori (chiamate *bins*) corrispondenti alle *ampiezze* e *fasi* di altrettante bande di frequenza per ogni blocco (ossia N bande). In senso stretto sono, rispettivamente, la *parte reale* e *immaginaria* di un numero complesso. Tali bande sono equispaziate lungo tutto il range delle frequenze.

²Vedremo fra breve che è opportuno prendere un valore di N che sia potenza di 2.

Poichè tuttavia si può dimostrare che lo spettro di un segnale reale è sempre simmetrico, si ottengono di fatto solo $N/2$ bande.

La figura 13.2 riassume visivamente questi passaggi

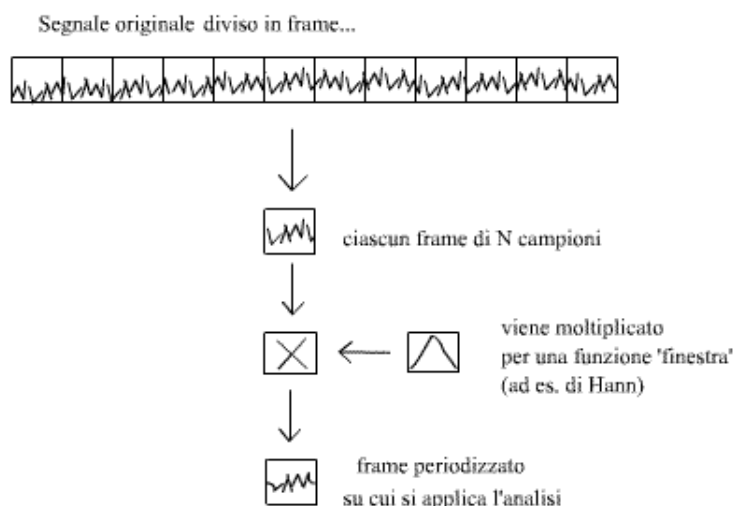


Figura 13.2:

Circa il secondo problema, quello di ridurre il carico di lavoro macchina, si è affermato un geniale algoritmo basato su un calcolo ricorsivo, detto *butterfly*, che ha dato luogo alla *Fast Fourier Transform* (d'ora in poi FFT). Esso riesce, in pratica, ad evitare il forte grado di ridondanza che si presenta nel calcolo della Trasformata di Fourier Discreta (DFT) ma richiede, come condizione, che il numero di campioni N , usati per ogni frame, sia una potenza di 2. L'efficienza³ risulta dal fatto che, invece di avere N^2 operazioni (ossia $N*N$), si scende a $N*\log_2(N)$. La differenza si fa assai sensibile, ovviamente, per valori alti di N . Se, ad esempio, N è 2048 invece di avere 4.096.576 operazioni si scende a sole 22.528. Il risparmio in termini di tempi diventa cruciale quando l'estrazione dello spettro deve avvenire praticamente in tempo reale, come oggi si richiede in molte applicazioni.

13.1.1 Un esempio per capire meglio

Va precisato che, nel procedimento sopra descritto, non emergono i dettagli di *tutte* le frequenze in quanto lo spettro che la FFT ci restituisce è discreto e non continuo. Le frequenze di cui avremo rilevato il doppio profilo (ampiezza e fase) sono cioè equispaziate, in modo più o meno fitto, in base ad un settaggio preventivo dello strumento di analisi, ossia in base alla scelta di N . A spaziatura più fitta (il che vuol dire ad N più grande) corrisponde una maggiore risoluzione in frequenza.

Facciamo un esempio. Abbiamo detto di considerare $N = 1024$, stiamo cioè simulando che il nostro segnale abbia una periodicità di N campioni. Per noi è più intuitivo pensare un periodo in termini temporali e non di campioni, quindi ci chiediamo: a quanto tempo corrisponde

³Senza entrare nei dettagli si può pensare che invece di procedere ad una DFT su N punti si possono calcolare 2 DFT su $N/2$ punti, con un risparmio di risorse di calcolo. Sviluppando il concetto si possono calcolare 4 DFT di ampiezza $N/4$ punti e così via fino a giungere a DFT operanti su solo 2 punti. Combinando opportunamente il flusso dei calcoli si ottiene quindi la drastica riduzione di operazioni che giustifica la qualifica di *fast* (veloce) a questo algoritmo. Cfr. Lindoro Massimo Del Duca, *Musica digitale*, Muzzio Editore, 1987, pag. 155.

un periodo di 1024 campioni? Basta partire dalla frequenza di campionamento (SR) che assumiamo essere 44100 Hz, da cui 1 campione sarà $1/44100$ secondi e, per conseguenza, 1024 campioni saranno (in secondi)

$$1024 * \frac{1}{44100} = 0.02322$$

Il periodo T del nostro segnale (che, come si è detto, viene *forzato* ad essere periodico) è quindi di circa 23 millisecondi, il che corrisponde ad una frequenza (basta fare $1/T$) di circa 43 Hz. Cosa ci dice questo dato? Che verrà effettuata un'analisi (una FFT) 43 volte per ogni secondo.

L'algoritmo della FFT operando su N campioni reali nel tempo li converte in $(N/2) + 1$ campioni complessi in frequenza, con relativa parte reale e immaginaria. Essa ci restituisce le frequenze in modo simmetrico rispetto alla frequenza di Nyquist: se in un segnale abbiamo una frequenza di 1000 Hz, ipotizzando un Nyquist a 22050 (SR/2), la FFT rileverà una frequenza a 1000 e a 43100.

Ne consegue che per N=1024 non si hanno 1024 coppie di valori ma, in senso stretto, 513. In realtà possiamo trascurare la prima uscita in quanto essa rappresenta la componente continua del segnale che dovrebbe essere nulla. Quindi avremo di fatto 512 *bins*. Le rimanenti non sono che la riflessione intorno alla frequenza di Nyquist.

Pertanto nel nostro esempio ogni banda avrà un 'passo' di $22050/512$, ossia 43.06 Hz. La prima parte da 0, la seconda a

$$\frac{1}{1024} * 44100 = 43.066 Hz$$

la terza a

$$\frac{2}{1024} * 44100 = 86.132 Hz$$

la decima a

$$\frac{9}{1024} * 44100 = 387.597 Hz$$

e così via⁴.

Questo vuol dire che la FFT calcola *ampiezza* e *fase* (un *bin*) di una possibile frequenza⁵ che si troverà nel range tra 0 e 43 Hz, poi rileverà *ampiezza* e *fase* di una (possibile) frequenza slittata di 43.06 Hz, (ossia tra 43 e 86) Hz e poi, ancora, un'altra slittata di 43.06 Hz, fino ad arrivare alla fine del range che è la cosiddetta *frequenza di Nyquist*, ossia SR/2 (per noi 22050 Hz).

In sostanza abbiamo 'fatto finta' che il segnale presenti parziali periodiche e in rapporto armonico tra loro, ossia progrediscano per numeri interi di una frequenza fondamentale, nel

⁴Per comodità possiamo continuare a calcolare la larghezza di banda con SR/N e non come

$$\frac{SR}{2} / \frac{N}{2}$$

in quanto il risultato non cambia.

⁵Va ribadito, per essere precisi, che la FFT non calcola, come talvolta si dice impropriamente, la frequenza presenti nello spettro di un segnale, in quanto esse sono, per così dire, già fissate in anticipo al momento della scelta di un blocco di computazione (*blocksize*), per noi N. Essa piuttosto ci dice qual è l'energia, più precisamente l'ampiezza, di *quella* (possibile) frequenza. Diciamo che la 'trova' nel momento in cui l'ampiezza è diversa da 0. Inoltre trova, di quella possibile energia, anche la fase. La coppia *ampiezza* e *fase* è - concettualmente - l'analogo della coppia complessa Reale/Immaginario. Ciò che muta, di quella coppia espressa in modalità diverse, è solo la codifica.

nostro caso 43 Hz. Ciò non vuol dire, però, che tra 0 e 43 Hz o tra 43 e 86 Hz il segnale debba presentare necessariamente un valore diverso da 0. Se, ad esempio, sottoponiamo ad analisi un segnale di frequenza 90 Hz verrà rilevata un'energia nella terza banda di frequenze, mentre nella prima e seconda banda si avrà 0.

Poichè nei calcoli sopra riportati N si trova al denominatore (per noi 1024), si evince che per N più grande la *risoluzione della frequenza* sarà maggiore, per N più piccolo sarà minore. Con 'risoluzione' intendiamo il grado della capacità dello strumento di analisi di controllare variazioni di frequenza. Infatti, se invece di $N=1024$ avessimo $N=512$, le regioni di frequenza salgono a 86.13 Hz la prima, a 172, 26 Hz la seconda, e così via. La frequenza del segnale dell'esempio precedente, di 90 Hz, cadrà ora nella seconda regione e non più nella terza, 'mescolandosi' però con altre eventuali frequenze presenti fino a 172 Hz. Abbiamo quindi 'peggiolato' la risoluzione in frequenza dell'analisi.

13.1.2 Conflitto tra risoluzione di frequenza e di tempo

In base a quanto detto potrebbe essere scontata la conclusione che è meglio puntare a valori di N molto grandi. A parte il fatto che questo comporta oneri di calcolo maggiori (abbiamo visto che i calcoli aumentano all'aumentare di N) sorgono problemi in quanto vi è un sostanziale conflitto tra la risoluzione in frequenza e quella nel tempo: volendo ottenere un'ottima risoluzione in frequenza si avrà una scarsa risoluzione temporale. E con quali conseguenze? Semplicemente che se il segnale varia molto velocemente nel tempo rischieremo di non poterne controllare tutte le evoluzioni in quanto l'analisi viene applicata a *tutto ciò che è dentro* ciascuna finestra.

Formalmente la risoluzione temporale è data dall'inverso della formula precedente. Se la risoluzione in frequenza si esprime con SR/N ora dovremo porre N/SR : infatti $1/SR$ è il tempo (la durata) di ciascun campione che moltiplichiamo per N per avere la durata degli N campioni.

Facciamo come esempio una comparazione con due differenti valori di N :

$N = 1024$ La risposta in frequenza è $44100/1024 = 43.06$ Hz, e il Periodo T di ciascun blocco $1024 \cdot 1/44100 = 0.023$ sec. (abbiamo moltiplicato gli N campioni per la 'durata' di 1 campione)

$N = 4096$ La risposta in frequenza è $44100/4096 = 10.706$ Hz, e il Periodo T di ciascun blocco $4096 \cdot 1/44100 = 0.092$ sec.

Conclusione Per un blocco di 1024 campioni abbiamo una buona risoluzione temporale, capace di cogliere evoluzioni del segnale nell'ordine di 23 millesimi di secondo. Nel contempo la risoluzione in frequenza è di 43 Hz quindi se nel segnale vi sono affollamenti di parziali (parliamo ovviamente soprattutto per le basse frequenze) distanti meno di 43 Hz rischiamo di produrre risultati non corretti, perchè l'analisi effettuerà una media tra le frequenze presenti in quel range. Viceversa con un blocco di 4096 campioni aumentiamo la risoluzione in frequenza a soli 10.76 Hz, ma nel contempo peggioriamo quella temporale a poco meno di un decimo di secondo.

13.1.3 Overlapping

Accenniamo infine alla tecnica dell'*overlapping* che letteralmente significa 'sovrapposizione'. Anche se ne abbiamo già parlato, in 12.5.1, ne ribadiamo il concetto: effettuando la periodizzazione del segnale con l'applicazione di una *funzione-inviluppo*, ad esempio la finestra di Hann, si perdono informazioni agli estremi della regione perchè ne vengono smussati i valori (per farli tendere a 0). Ciò porterebbe ad un degrado inaccettabile di performance dell'analisi.

Per sopperire a questo problema si procede in questo modo: si prendono sempre pacchetti di N campioni (cui sottoporre una FFT), ma senza saltare ogni volta N campioni. Si preleva cioè un blocco dopo un numero di campioni $M < N$. Ad esempio⁶ se $N = 300$, si può prelevare - con $M = 200$ - un blocco successivo a 200, (e non a 300), un'altro a 400 e così via, come illustra la fig. 13.3.

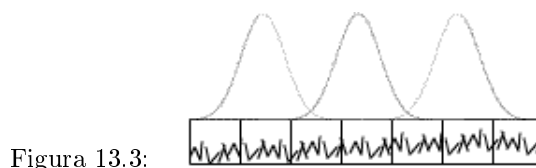


Figura 13.3:

In questo modo, combinando con opportune tarature le analisi delle finestre sovrapposte, si riducono sensibilmente gli errori di finestratura.

13.2 Tipologie d'uso di $\text{fft} \sim$ ed $\text{rfft} \sim$

Affrontiamo più da vicino la materia studiando e commentando alcune patch di Miller Puckette contenute nella versione base di PD che presentano l'uso degli oggetti $\text{fft} \sim$ ed $\text{rfft} \sim$.

13.2.1 $\text{fft} \sim$

Cominciamo con lo studio di una prima patch, dal nome *I01.Fourier.analysis*, opportunamente riscritta (fig. 13.4). In essa sperimentiamo come l'oggetto $\text{fft} \sim$ calcoli l'energia frequenziale di un segnale prodotto da un oggetto $\text{osc} \sim$ di cui noi stessi impostiamo la frequenza. Si tratta certamente di una patch inutile da un punto di vista pratico, ma interessante sotto il profilo didattico.

Esaminiamola da vicino.

La parte alta della patch può apparire a prima vista un po' strana. Invece di assegnare in modo del tutto arbitrario una frequenza ad $\text{osc} \sim$ facciamo in modo di servirgli un valore che risponda ad un certo requisito: più esattamente che sia un multiplo del rapporto tra la frequenza di campionamento e 64. Perché?

Noi sappiamo che il processo della fft consiste nel ripartire tutto il segnale in finestre la cui consistenza 'simula' una periodicità. Questa finestra in PD è composta, di *default*, da 64 campioni la cui durata temporale è di $64 * 1 / 44100 = 1.45$ msec. Ma per esprimere la frequenza di questo segnale periodico devo calcolare il suo reciproco, ossia $44100 / 64 = 689.06$ Hz circa. Stiamo dunque 'simulando' che il nostro segnale abbia una frequenza di 689.06 Hz. Se ora forniamo un suono da analizzare che ha davvero quella frequenza non si tratterà più di una simulazione, con i problemi che ne derivano⁷. Con una presa di campioni 689.06 volte al secondo siamo sicuri di analizzare un periodo completo di suono e non vi è la necessità di effettuare 'correzioni' di sorta.

Per capire meglio facciamo anche il ragionamento inverso. Se voglio analizzare un segnale la cui frequenza è 689.06 Hz vuol dire che esso ha un periodo di 0.00145 sec. il che equivale a

⁶Assumiamo, per semplicità e chiarezza, $N = 300$ ma ricordiamo che la FFT richiede un dimensionamento pari a potenze di 2.

⁷Si è già detto che, nei casi più comuni, si deve effettuare una correzione agli estremi del segnale con un'opportuna funzione-finestra (come la finestra di Hann).

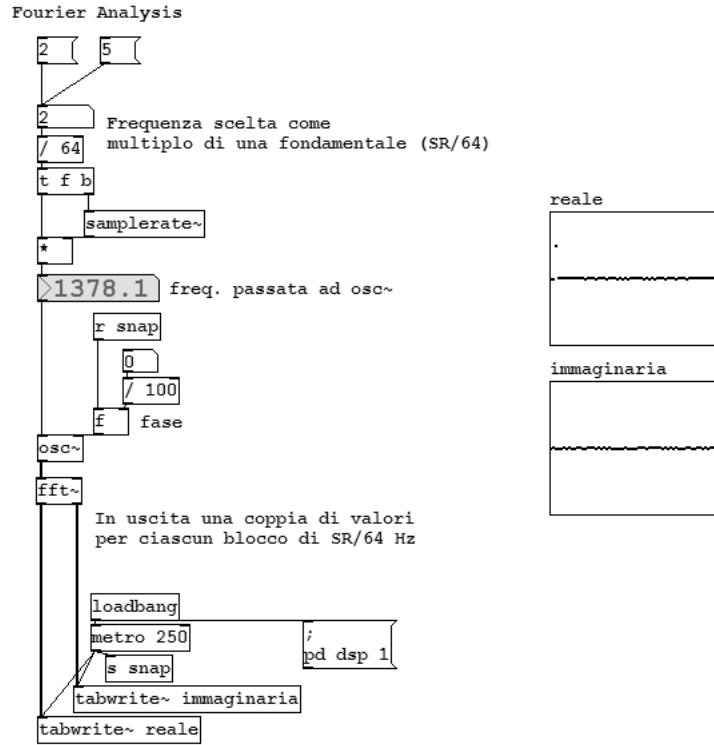


Figura 13.4:

64 campioni (ovviamente ad una SR di 44100 Hz). Prendendo, come fa PD di *default*, 64 campioni come parametro di *fft* vuol dire che analizzo il segnale esattamente periodo per periodo. Analogamente si dirà per multipli di 64 campioni: avrò semplicemente periodi multipli.

Ecco allora la ragione dei calcoli nella patch. Posto un intero N (qui con valori predefiniti 2 e/o 5) avrò:

$$\frac{44100}{64} * N$$

che si può riscrivere

$$\frac{N}{64} * 44100$$

Nel box numerico colorato avrò dunque la frequenza passata ad *osc~*, una frequenza che so già essere adeguata al valore dell'ampiezza di analisi (64).

Più in basso il valore della frequenza entra in un oggetto *osc~*, alla destra del quale posso anche impostare una fase, in centesimi di ciclo (da 0 a 1).

Finalmente l'uscita di *osc~* entra nell'oggetto *fft~* che ha 2 ingressi, ma qui è usato solo quello di sinistra. I due ingressi accettano, rispettivamente, il valore *reale* e *immaginario* di un segnale complesso. Qui, come si vede, la parte immaginaria è zero (non entra alcun valore).

Anche l'uscita da *fft~* è doppia: una per la parte reale e una per l'immaginaria. Più esattamente usciranno $(N/2) + 1$ parti reali ed $(N/2) - 1$ parti immaginarie⁸. Ad esempio, nel nostro caso con $N = 64$ si avranno 33 parti reali e 31 immaginarie.

⁸A DC e alla frequenza di Nyquist non c'è alcuna parte immaginaria.

Ricordiamo che per 'parte reale e immaginaria' intendiamo concettualmente l'analogo di *ampiezza e fase*. Volendo 'ricostruire' (come più avanti vedremo) un segnale a partire dai dati computati da un oggetto *fft~* e utilizzando un banco di oscillatori pari alla metà dell'ampiezza di blocco⁹, la coppia 'parte reale e immaginaria' ci darà le impostazioni per un oggetto *osc~* che ripropone quella particolare frequenza (con quell'ampiezza e quella fase). Si faccia attenzione, tuttavia, che non è possibile, per un problema di codifica, inserire direttamente in un oggetto *osc~* l'uscita di *fft~*. L'unico oggetto che 'comprende' quella codifica è l'oggetto *ifft~*, dove la *i* sta per *inversa-FFT*¹⁰.

Riprendendo l'esame della patch, i valori così trovati vengono raccolti da due oggetti *tabwrite~*, chiamati rispettivamente *reale* e *immaginaria*. L'oggetto *metro*, settato ad 1/4 di secondo, serve per monitorare 4 volte al secondo il risultato dell'analisi.

I 2 riquadri a destra ci indicano visivamente questi risultati. Si nota immediatamente la simmetria dei valori in uscita da *fft~* assumendo come centro la frequenza di Nyquist. Ogni valore a sinistra si riflette specularmente a destra. Variando, infine, i valori della fase si nota che la parte immaginaria si incrementa da un valore iniziale piatto. Ad un quarto di ciclo (inserendo 25, ossia 1/4 di 100) si nota che la parte immaginaria si è sollevata mentre è diventata piatta quella reale. Analogamente a 3/4 di ciclo.

13.2.2 Breve approfondimento: valori Reali e Immaginari

Si è detto che l'uscita di *fft~* - ma anche, come vedremo, della sua variante semplificata *rfft~* - è costituita da una coppia complessa di valori, *Reale* e *Immaginario* (usualmente notati con *Re* e *Im*) che corrisponde, concettualmente, ad *ampiezza e fase* di una sinusoide. La differenza risiede nella codifica, che attiene ad un sistema di *coordinate polari*: l'ampiezza è data dalla distanza dall'origine del piano complesso e la fase è l'angolo dato dalle coordinate polari. Le conversioni da una coppia all'altra sono date dalle seguenti formule

$$amp = \sqrt{(re^2 + im^2)} \quad (13.1)$$

$$fase = \arctan\left(\frac{im}{re}\right)$$

Come si è detto, se vogliamo rendere disponibili queste informazioni per ricostruire dei segnali audio in senso stretto dobbiamo ricorrere all'oggetto che compie l'operazione inversa, quindi *ifft~* e, rispettivamente, *irfft~*.

In effetti, aldilà di semplici operazioni di carattere analitico - per le quali, peraltro, esistono oggetti più specifici, come vedremo avanti - l'utilità di una *fft* consiste principalmente nella possibilità di lavorare nel dominio della frequenza per operare un editing del segnale originario. All'uopo può essere utile tener presente alcune proprietà¹¹ che riguardano la manipolazione di *ampiezza e fase*.

a) se dobbiamo riscaldare un'ampiezza si osserva che:

$$amp(x * Re, x * Im) =$$

$$\sqrt{(x * Re)^2 + (x * Im)^2} =$$

⁹L'analisi spettrale effettuata da una FFT si può infatti descrivere come l'estrazione di quei dati (coppie di *ampiezze e fasi*) che ci permetterebbero di ricostruire il suono di origine tramite la somma di un blocco di *blocksize/2* onde sinusoidali aventi quelle particolari specifiche.

¹⁰Analogamente, il corrispettivo di *rfft~* (che vedremo più avanti) sarà *irfft~*.

¹¹Vedi Frank Barknecht, *Beginner's Guide to the FFT-objects in Pd* in <http://footils.org/cms/show/60>.

$$\begin{aligned} \sqrt{x^2(Re^2 + Im^2)} &= \\ \sqrt{x^2} * \sqrt{Re^2 + Im^2} &= \\ |x| * amp(Re, Im) \end{aligned}$$

b) riscaldando, analogamente, la fase si può invece dimostrare che non si produce alcun cambiamento. Infatti:

$$phs(x * Re, x * Im) = atan\left(\frac{x * Im}{x * Re}\right) = atan\left(\frac{Im}{Re}\right) = phs(Re, Im) \quad (13.2)$$

13.2.3 Finestra di Hann ed *rfft* ~

Nella patch precedente abbiamo studiato un caso di applicazione della *fft* basata su un blocco la cui dimensione è stata scelta accuratamente in modo da esimerci dall'impiego di una funzione-finestra correttiva. Volendo invece sottoporre ad analisi un segnale di una frequenza qualunque dovremo ricorrere all'artificio di una sua periodizzazione forzata tramite una particolare finestra. Per studiare più da vicino il procedimento ricorriamo ad una patch presente nell'help di PD (*I02.Hann.window*), lievemente ritoccata. In essa troviamo una parte principale ed una in subordine (sub-patch). Cominciamo dalla prima (vedi fig. 13.5).

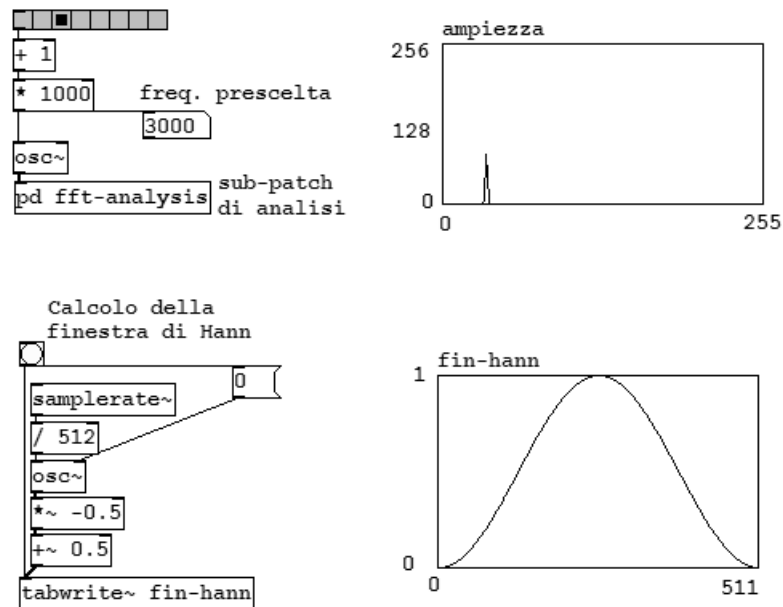


Figura 13.5:

In alto a sinistra impostiamo semplicemente una frequenza a piacere, tramite un oggetto moltiplicatore. Ovviamente il lettore potrà aggiungere un box numerico, connesso con l'inlet sinistro di *osc~*, per introdurre la frequenza che desidera. Il segnale computato da *osc~* entra dunque nella sub-patch il cui nome è *fft-analysis*.

La parte in basso, a sinistra della patch principale, ci propone un algoritmo per creare una curva di Hann che risponda alle nostre necessità: che sia cioè costituita da 512 punti. L'oggetto *sample_rate~* si occupa di rilevare la frequenza di campionamento su cui è impostata PD. Nel nostro caso, che è il più comune, sarà 44100 Hz. Dividendo per 512 si ha 86.132 Hz, frequenza che viene fornita all'oscillatore *osc~*.

Se questo dato può sembrarci strano riflettiamo come segue: una frequenza di 86.132 Hz corrisponde ad un periodo di $1/86.132$, cioè di 0.01161 secondi. A quanti campioni equivale questo periodo di tempo? Basta dividere per il tempo di durata di 1 campione, ossia effettuare il calcolo:

$$\frac{0.01161}{\frac{1}{44100}} = 512$$

Otteniamo insomma proprio i 512 campioni che ci servono. Proseguendo, da *osc~* escono valori oscillanti tra -1 e +1. Moltiplichiamo allora tali valori per -0.5 per avere una oscillazione tra 0.5 a -0.5. Una ulteriore somma di 0.5 porterà l'oscillazione tra 1 e 0, come si vede in figura. Questi valori vengono immagazzinati nella tabella di nome *fin-hann* (che sta per *finestra di hann*). La rappresentazione di questa curva è visibile nel grafico in basso alla patch principale. Si può osservare che la curva si compone di 512 punti, quanti sono i campioni del blocco di segnale che, di volta in volta, viene fatto oggetto di analisi.

Rivolgiamoci ora alla sub-patch *fft-analysis* (fig. 13.6). Ciò che avviene qui dentro è praticamente questo: viene preso un blocco di campioni alla volta il cui numero è di 512, valore imposto grazie all'oggetto *block~*. Questo insieme di valori numerici (che rappresentano *digitalmente* il segnale) viene moltiplicato per l'altro insieme di valori, che parte da zero e finisce a zero, quello della curva di Hann.

I due blocchi devono, ovviamente, corrispondersi in modo preciso e ciò viene assicurato da un oggetto di nome *tabreceive~*. Questo oggetto non fa altro che prendere i valori che custodisce in se stesso (sono i valori custoditi in *fin-hann*, quelli della curva di correzione) e 'spararli' in uscita in concomitanza di ogni blocco di computazione di PD. Poichè anche questo blocco è di 512 campioni, si avrà che 512 campioni del segnale audio si uniscono con i 512 campioni contenuti *fin-hann*. E il gioco è fatto.

Per effetto della moltiplicazione, campione per campione, il segnale sarà portato a zero alle due estremità del blocco, mentre rimarrà immutato al centro. Questo segnale, così corretto dalla finestratura, viene quindi fatto entrare nell'oggetto *rfft~*. Riflettiamo dunque sul fatto che è stato necessario 'preparare' il materiale-segnale da inviare al dispositivo che effettuerà l'analisi.

Di quest'ultima si incarica *rfft~*. Il nome dell'oggetto significa 'fft reale' e infatti, proprio per questo, presenta un solo ingresso (*reale*). Si può notare la differenza con l'oggetto *fft~* che possiede invece 2 ingressi, accettando un segnale *complesso*, fatto di coppie di valori, uno reale e l'altro immaginario. Sul fronte opposto abbiamo 2 uscite, a sinistra la parte reale, a destra quella immaginaria.

Perchè questa 'variante' di una più convenzionale procedura di *fft*? Perchè in questo modo si risparmiano risorse di computazione (più precisamente, la metà). Questo oggetto, infatti, assume che la componente immaginaria sia zero, pertanto vengono computati solo $(N/2) + 1$ canali, gli altri vengono determinati per simmetria.

Il calcolo che troviamo più in basso è l'applicazione della formula 13.1 studiata in precedenza. I valori della componente reale e quella immaginaria, pensati nello spazio bidimensionale dei numeri complessi, vengono elevati al quadrato, quindi sommati. Alla fine se ne estrae la radice quadrata. Possiamo pensare alla misura dell'ipotenusa di un triangolo ideale i cui cateti sono, rispettivamente, la parte reale e quella immaginaria.

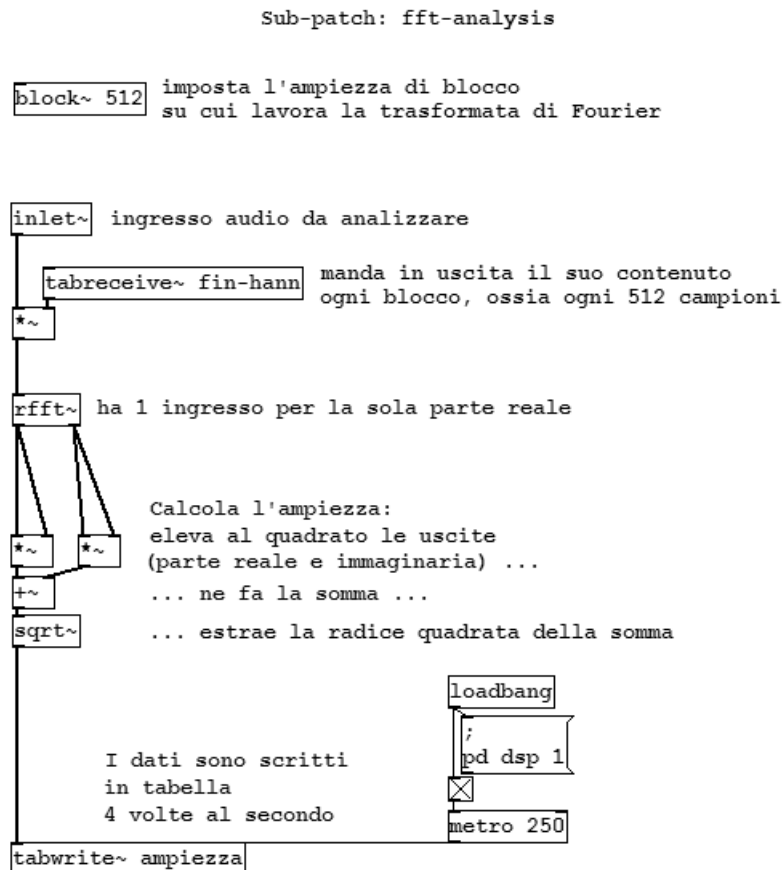


Figura 13.6:

A questo punto non resta che mandare i valori così ottenuti in un oggetto che possa offrircene una rappresentazione grafica. Di questo si occupa l'oggetto `tabwrite~` che crea un array di nome 'ampiezza' scrivendo alla velocità di 4 volte al secondo (1 volta ogni 0.25 secondi).

13.2.4 Ricostruzione del segnale audio. Uso di `irfft~`

Per completare il nostro percorso studiamo ora la patch, assai utile sotto il profilo didattico, *103.resynthesis.pd*, tratta sempre dalla documentazione interna di PD. Essa ci propone un esempio assai pratico di impiego di un dispositivo di analisi e risintesi: si tratta di un filtro grafico in cui, tramite mouse, è possibile stabilire un profilo di guadagno lungo tutte le frequenze udibili (da 0 a 22050) da applicarsi ad un segnale audio campione che, in questo caso, è un rumore bianco.

Nella nostra versione riscritta (fig. 13.7) presentiamo in forma semplificata la patch madre: in primo piano la finestra grafica in cui agire con il mouse. Sulla destra un'altra sub-patch di nome *Hann-window* provvede a creare, separatamente, la finestra di Hann. Ne abbiamo già parlato quindi la trascureremo.

A sinistra, l'oggetto `noise~` entra nella sub-patch *fft-analysis*. Studiamola da vicino (fig. 13.8).

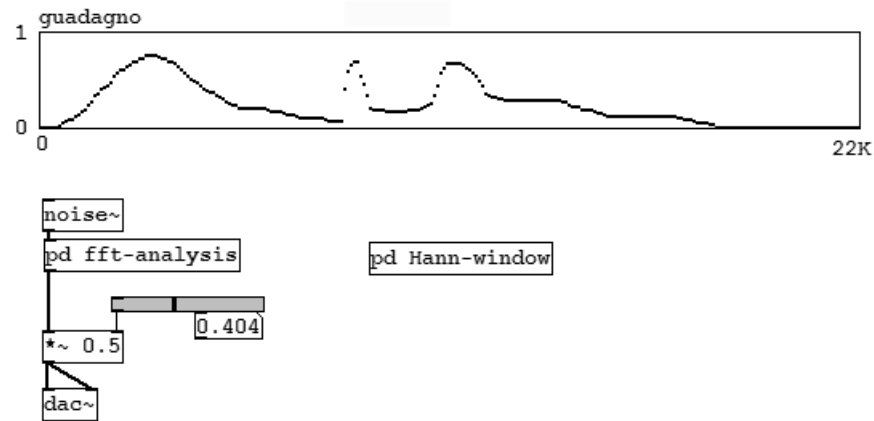


Figura 13.7:

In alto vediamo la definizione di blocco (di campioni) su cui PD andrà ad operare. Ricordiamo che, con tale dichiarazione, si influenza solo la sub-patch e non la patch madre. Rispetto all'uso fatto in precedenza, l'oggetto *block~* presenta ora 2 argomenti. Il primo (che già conosciamo) stabilisce il numero di campioni per blocco, il secondo l'*overlap*, qui fissato a 4. Ciò significa che in questo caso vengono computati sempre blocchi di 512 campioni ma ad intervalli di 128 alla volta, ossia $512/4$. I $3/4$ del blocco vengono quindi a sovrapporsi, ogni volta aggiungendosi un set 128 campioni 'nuovi'. Questo 'arrangiamento' dei campioni ai fini della computazione viene effettuato dall'oggetto *inlet~*. Corrispondentemente l'oggetto *outlet~* dovrà rimettere insieme, di volta in volta, gli ultimi quattro segmenti (giunti al suo input) in modo da ricostruire il segnale originale.

Più in basso, dopo l'ingresso del segnale, avviene la moltiplicazione blocco per blocco con la finestra di Hann (opportunamente implementata in una tabella di 512 punti). La scritta *1a. volta* ci informa che questa finestratura avverrà anche un'altra volta, nella parte bassa della sub-patch. Ne parleremo più avanti.

A questo punto il segnale è pronto per entrare nell'oggetto *rfft~* perchè venga analizzato. Le coppie di valori in uscita sono da intendersi, lo ripetiamo, come l'ampiezza e la fase relative a ciascun blocco o - per tornare alla nostra metafora di un banco di oscillatori - relative a ciascun oscillatore (di frequenza SR/N e suoi multipli interi).

A questo punto interviene il riscalamento che è, se vogliamo, la parte cruciale della funzionalità di tutta la patch. Infatti, a seconda di come avevamo impostato il guadagno nel relativo box grafico della patch madre, qui si moltiplicano i valori di ciascun blocco - grazie a *tabreceive~* che gestisce la tabella *guadagno* - per una certa quantità oscillante tra 0 e 1. Laddove il valore, per quel blocco, è impostato a 1, la coppia ampiezza/fase passa inalterata, laddove invece è minore di 1 o è 0, i valori della coppia verranno riscalati corrispondentemente. Richiamiamo, a questo proposito, quanto detto in 13.2.2: il riscalamento riguarderà, di fatto, le ampiezze e non le fasi perchè quest'ultime rimangono immutate (si riveda la formula 13.2).

I risultati vengono poi elevati alla quarta potenza per aumentare la rilevanza delle modifiche. Più in basso, la divisione per 768 merita una breve spiegazione. Gli oggetti *fft~* ed *rfft~* sono progettati per compiere i loro calcoli senza alcuna normalizzazione, così i valore in uscita sono, rispettivamente, N ed $N/2$ volte più grandi¹². Pertanto si impone una divisione per ristabilire i valori originari. Nel caso di utilizzo (insieme) della coppia *rfft~* ed *irfft~* si dovrà dividere per

¹² N ed $N/2$ è il numero delle computazioni ricorsive

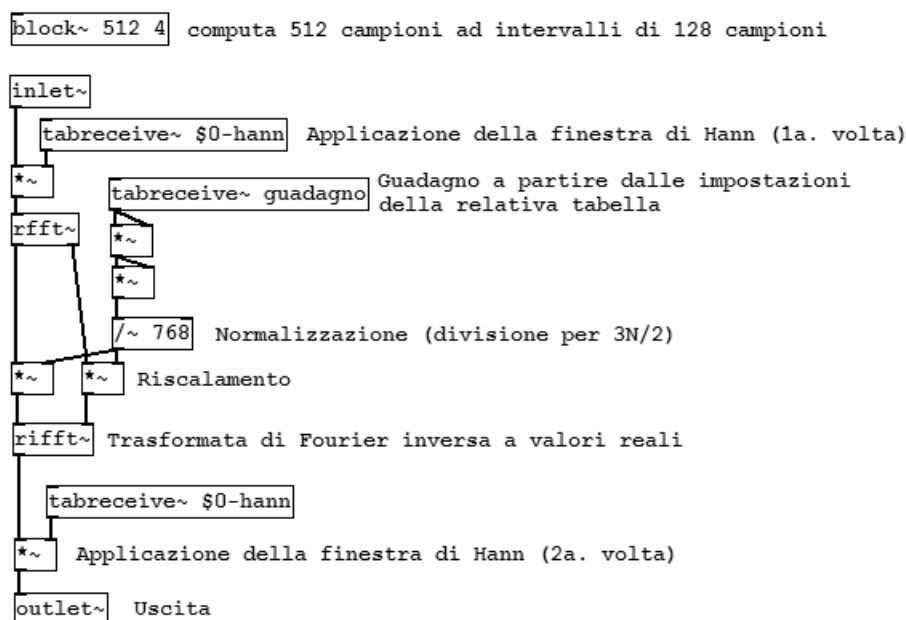


Figura 13.8:

N. Ma in questo caso non basta. Dato l'*overlap* di 4 si rende necessaria una ulteriore divisione per $3/2$ e questo perchè per 2 volte, una su *rfft~* e l'altra su *irfft~*, ogni blocco conteneva i $3/4$ del precedente. Quindi $3/4 * 2 = 3/2$. In sostanza il numero 768 deriva da $N * 3/2$, ossia $512 * 3/2$.

Proseguendo verso il basso, l'uscita da *rifft~* viene, come abbiamo visto, ulteriormente riscalata sulla finestra di Hann e questo perchè l'operazione della *rfft inversa* può produrre valori spuri agli estremi di ciascun blocco. Infine il risultato, che ormai è tornato ad essere un segnale audio a tutti gli effetti, viene spedito su *outlet~* e da qui, dopo un'ulteriore impacchettamento di cui si è detto, mandato alla finestra madre per l'uscita audio.

13.3 Un rilevatore di frequenza e di ampiezza: *fiddle~*

Assai utili e ricchi ormai di una cospicua letteratura¹³ sono i dispositivi di rilevamento dell'altezza a partire da un suono in ingresso¹⁴. Da un lato tali tracciamenti possono essere di supporto alle partiture per analisi statistiche, dall'altro - e con implicazioni certamente più interessanti - trovano un eccellente impiego nei processi di composizione interattiva uomo/macchina. Un sistema di rilevamento via software può infatti diventare un dispositivo di controllo per gestire vari parametri compositivi, come ad esempio le caratteristiche di un filtro, la spazializzazione, la traslazione d'altezza, l'indice di una modulazione di frequenza, e così via.

Per tali svariati compiti PD mette a disposizione un oggetto assai interessante, di nome **fiddle~**, che in teoria non fa parte della dotazione standard, ma ormai presente in tutti gli

¹³Si veda il classico Curtis Roads, *The Computer Music Tutorial*, MIT Press, 2000, Parte IV Cap. 12.

¹⁴Ovviamente deve trattarsi di un segnale che abbia la caratteristica di presentare frequenze in qualche modo stabili. Non avrebbe senso sottoporre a tali procedure d'analisi suoni puramente elettronici a largo spettro.

ultimi rilasci del programma (si trova in genere nella cartella *extra*). L'oggetto lavora bene con segnali monofonici, ma anche - sebbene in modo meno affidabile - con quelli polifonici fino a 3 voci.

Esaminiamolo da vicino.

Argomenti Gli argomenti che è possibile inserire in *fiddle*~ sono 4. Ovviamente si può lasciare anche senza argomenti: in tal caso avranno valore le impostazioni di default. Da sinistra a destra si scriveranno:

- **Numero di punti.** Indica il numero dei punti della finestra di analisi, ossia il numero di campioni di ciascun blocco su cui avviene la computazione. Deve essere una potenza di 2, più precisamente tra 128 e 2048. Di default è 1024.
- **Numero di altezze.** Indica il numero delle altezze separate che è chiamato a rilevare. Può essere da 1 a 3: di default è 1.
- **Numero di picchi di analisi.** Poiché il processo di rilevamento del pitch si effettua su una certa quantità di singoli contributi frequenziali (detti picchi), con questa impostazione possiamo determinare il numero massimo di picchi che andranno considerati. Tale numero può oscillare tra 1 e 100: di default è 20.
- **Numero dei picchi in uscita.** Diciamo a *fiddle*~ di quante singole componenti, presenti nel suono oggetto di analisi, vogliamo avere traccia.

Uscite Il numero di uscite dell'oggetto è funzione degli argomenti prescelti. Il minimo è 4, ma possono aumentare se, ad esempio, abbiamo inserito in argomento un numero di altezze maggiore di 1. Si tratta di uscite sia *discrete* che *continue*. Le prime riguardano il caso in cui venga trovata un'altezza (pitch) e un attacco stabili. Cosa si intenda per 'stabile' dipende da come vengono impostati alcuni parametri in ingresso, che vedremo fra breve. Le seconde sono invece quelle che vengono aggiornate continuamente, ad una velocità che dipende dal periodo di analisi (il 'numero di punti' della finestra che abbiamo appena visto). Questi ultimi dati potrebbe essere utili per impostare un processo di risintesi del suono, magari dopo aver apportato qualche modifica 'in tempo reale'.

Più in particolare, le uscite da sinistra a destra sono:

- il rilevamento di una nuova altezza stabile (entro i parametri stabiliti con opportuno messaggio in ingresso)
- un bang ogni qualvolta rileva un suono dall'attacco deciso (entro i parametri stabiliti con opportuno messaggio in ingresso)
- da 0 a 3 coppie di valori (a seconda della relativa impostazione negli argomenti) indicanti *altezza* e *ampiezza*. Poiché tecnicamente è una lista di dati è necessario, per 'spacchettarla', utilizzare l'oggetto *unpack* e porre, alle sue due uscite, altrettanti box numerici, a sinistra per l'altezza, a destra per l'ampiezza
- l'ampiezza complessiva del segnale rilevata in modo continuo, espressa in dB
- il rilevamento continuo delle componenti sinusoidali di un segnale, nel numero impostato dal relativo argomento. Di default è 0. Se, ad esempio, si è messo 3 in argomento, l'analisi del La 440 di un pianoforte mostrerà la parziale di frequenza 440 con la sua relativa ampiezza, la parziale 880 con relativa ampiezza e la parziale 1320 sempre con la sua ampiezza.

Altri ingressi Nell'unico ingresso dell'oggetto, oltre ovviamente al segnale da analizzare, si possono inviare un insieme di messaggi che servono ad impostare gli ambiti entro cui verranno effettuati i rilevamenti. In particolare:

- **amp-range (basso) (alto)**. Imposta un range di controllo d'ampiezza (in decibel) tramite una coppia di valori, basso/alto. Scrivendo, ad esempio, "amp-range 40 60" intendiamo che se l'ampiezza del segnale è più *basso* di 40 dB non vengono rilevate nè altezze nè attacchi. La soglia *alta* - nell'esempio, 60 dB - è invece il minimo perchè venga rilevata un'altezza 'stabile'
- **vibrato (durata) (semitono)**. Accetta un'altra coppia di valori che indicano, rispettivamente, una certa *durata temporale* (espressa in millisecondi) entro la quale la nota rilevata non deve deviare più di un *certo intervallo* (espresso in *semitoni*) perchè possa essere considerata stabile. Se eccede quell'intervallo sarà invece considerata una nuova nota e mandata in uscita nell'outlet relativo (quello più a sinistra)
- **re-attack (tempo) (dB)**. Questa impostazione è analoga alla precedente, ma riguarda l'attacco. Perchè venga rilevato un nuovo attacco è necessario che la potenza del segnale sia superiore ad un certo valore in *dB* indicato entro un certo *tempo* in millisecondi
- **npartial (n)**. Nel calcolo per il rilevamento dell'altezza le parziali superiori vengono pesate in rapporto inverso al loro numero d'ordine. Con questo messaggio impostiamo il numero di parziali, *n*, che saranno soggette a questo riscalamento. In uno specifico articolo¹⁵ su *fiddle*, Puckette e altri riportano il seguente rapporto: la *j*-esima parziale (su *n* parziali) avrà un peso di

$$\frac{n}{n+j}$$

- **npoints (N)**. Con *N* una potenza di 2, possiamo impostare un valore (in numero di punti) alla finestra di analisi, indipendentemente che vi sia o non vi sia la dichiarazione di uno specifico argomento (vedi sopra)
- **auto (1 o 0)**. Imposta su Sì (1) o No (0) la modalità auto. Se è 0: il rilevamento dell'altezza o delle componenti sinusoidali viene controllato direttamente dall'utente, tramite un apposito messaggio *bang* (vedi item sotto). Se è settato su 1: i valori vengono registrati automaticamente per ogni periodo di analisi. Per default è 1. Si tenga inoltre presente che se *auto* è su 0 le due uscite 'discrete' (quelle più a sinistra) sono disabilitate
- **bang**. Messaggio utile solo in corrispondenza della modalità *Auto* 0. Attiva il rilevamento del valore corrente
- **print**. Manda in uscita, su terminale, i parametri controllati dai messaggi precedenti.

13.4 Esercitazioni con *fiddle~*

Presentiamo due semplici esempi di impiego dell'oggetto *fiddle~* in relazione alle due possibili direzioni d'uso: 1) un tracciatore d'altezze e di picchi di intensità che segua una performance e 2) un estrattore in tempo reale di parametri audio (frequenza e ampiezza) da riutilizzare per una successiva una sintesi (analisi e risintesi).

¹⁵Miller S. Puckette, Theodore Apel, David Zicarelli, *Real-time audio analysis tools for Pd and MSP*, <http://crca.ucsd.edu/~msp/Publications/icmc98.ps>.

13.4.1 Tracciatore d'altezze e di picchi

Un primo semplice impiego dell'oggetto *fiddle~* può essere quello di collegarlo all'uscita di un segnale: nel nostro esempio è la voce acquisita tramite *adc~*. L'obiettivo della nostra patch sarà quello di mostrarci le altezze di intonazione della nostra voce e, con maggiore dettaglio, di determinare la distanza temporale tra ogni altezza rilevata. Inoltre, come appendice alla patch, provvediamo a tener traccia di quest'ultimi rilevamenti scrivendoli su un apposito file o mostrandoli su terminale. Per fare questo ci serviremo dell'oggetto *textfile*.

Date le finalità della patch, ci servirà soltanto la componente 'discreta' delle uscite di *fiddle~* per cui non agganceremo nulla ai 2 outlets più a destra.

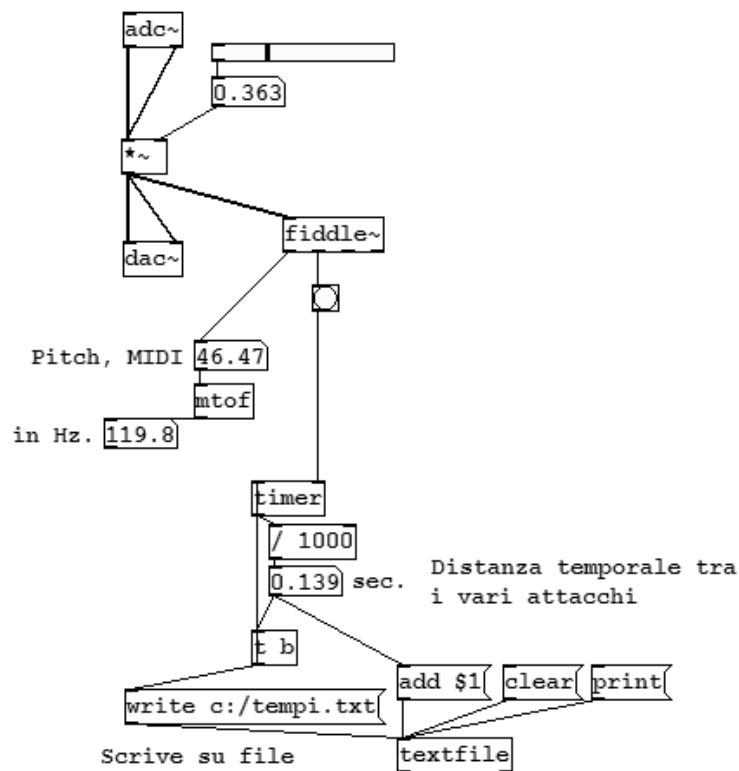


Figura 13.9:

Si noterà che gli outlets effettivi sono solo 4 perchè abbiamo lasciato *fiddle~* senza argomenti (o, più precisamente, sono operativi quelli di default). L'uscita più a sinistra ci fornisce l'altezza stabile rilevata, dapprima in Nota MIDI, poi convertita in Hz con l'apposito oggetto *mtof*.

La seconda uscita produce, come sappiamo, un bang ogni volta che rileva un picco di energia. Useremo questi messaggi come ingresso per una subroutine che calcola la distanza temporale tra ogni bang. L'oggetto *timer*, ricevendo i bang nell'inlet destro, fa partire il cronometro che conta il tempo intercorso dall'ultimo bang. Poichè, tuttavia, i tempi vengono ad aggiungersi uno sull'altro è necessario inviare un bang (tramite *t b*) dopo ogni ciclo, indirizzato all'inlet sinistro (di *timer*) che ha proprio il compito di azzerare il cronometro.

Più in basso l'oggetto *textfile* è sovrastato da una serie di messaggi dei quali uno, *add*

`$1`, aggiunge in automatico una stringa relativa a ciascun rilevamento effettuato. Infine `print` manda i risultati su terminale, mentre `write <percorso>`, scrive gli stessi su un apposito file.

13.4.2 Analisi e risintesi

Il campo dei processi di analisi e risintesi è indubbiamente di grande interesse soprattutto oggi quando la potenza delle macchine rende possibile computazioni in tempo reale prima inimmaginabili. Il concetto, assai semplice, alla base di questo approccio consiste nell'inviare in ingresso un qualunque segnale (nel nostro caso sarà la nostra stessa voce); esso viene sottoposto ad un processo di analisi, usando `fiddle~`, quindi i dati di analisi vengono fatti oggetto di un certo tipo di manipolazione il cui risultato offrirà infine i parametri per una risintesi del suono. Il tutto in tempo reale.

Osserviamo allora la patch di fig. 13.10.

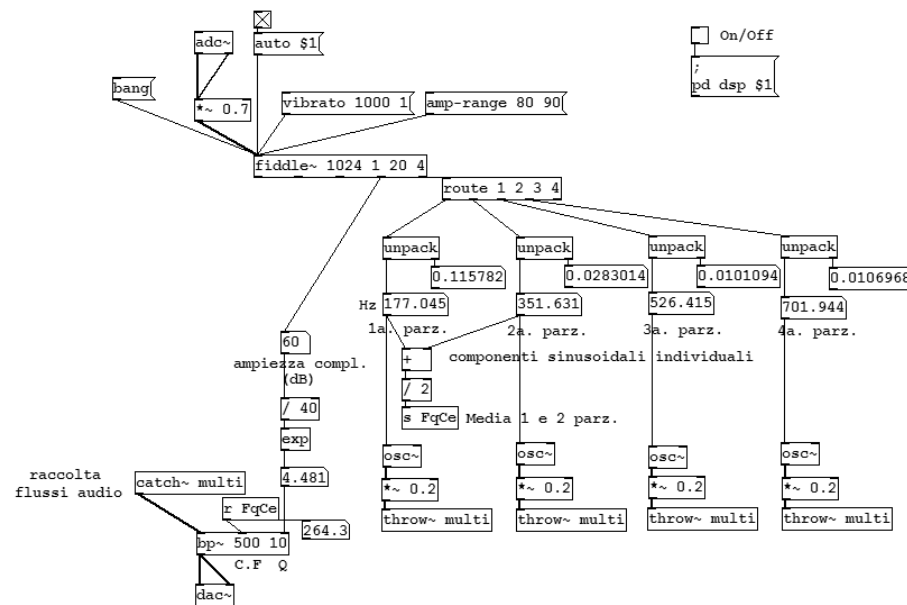


Figura 13.10:

In alto, il segnale (supponiamo la nostra stessa voce da un microfono) entra nell'oggetto `fiddle~` che, all'uopo, si avvale di alcune impostazioni con messaggi di `vibrato` e `amp-range`. Essi hanno la funzione di stabilizzare per quanto possibile il segnale da rilevare. Osserviamo inoltre il messaggio `auto $1`, da usarsi in abbinamento con il `bang`, sempre in ingresso a `fiddle~`. Con la spunta nell'oggetto `toggle`, che significa 1, `fiddle~` effettua l'analisi sul continuo del flusso audio. Togliendo invece la spunta potremo divertirci a rilevare le prime 4 parziali della nostra voce agendo semplicemente sul messaggio `bang`. Invece di scorrere velocemente, i dati rimangono bloccati sui box numerici sottostanti fino ad un nostro successivo `bang`.

Si noterà che, a causa dei 4 argomenti, `fiddle~` presenta 5 uscite. Faremo uso solo delle ultime 2: la penultima rileva l'ampiezza generale del segnale e l'ultima le 4 parziali (4 è infatti l'impostazione dell'ultimo argomento). I dati di quest'ultima sono coppie di valori rappresentanti a) una frequenza e b) la sua relativa ampiezza. Da qui la necessità di servirsi dell'oggetto `unpack`.

A questo punto, ottenuti i dati, non dobbiamo fare altro che pensare ad una loro possibile

manipolazione per ottenere le impostazioni per la sezione di risintesi. Quella pensata (a titolo puramente dimostrativo) per questa patch è presente nella sua parte bassa e consta di un banco di 4 oscillatori, ciascuno dei quali ricostruisce le singole parziali in uscita da *route*, e di una semplice sezione di filtraggio che utilizza due parametri variabili in tempo reale, a seconda delle caratteristiche del segnale principale in ingresso (la nostra voce). Come si ricoderà, l'oggetto *bp~* è un filtro passa-banda che richiede, oltre al segnale da processare (inlet di sinistra), una frequenza centrale (ingresso di mezzo) e un valore di Q (inlet di destra). In dettaglio:

- la frequenza centrale viene ottenuta facendo la media tra i valori in Hz delle prime due parziali (il dato viene inviato dall'oggetto *s FqCe*)
- il valore di Q viene ottenuto dall'ampiezza generale del segnale in ingresso e una semplice manipolazione: diviso per 40 diventa poi l'esponente di una base *e* (da cui l'oggetto *exp*). Ciò ha il compito di ottenere dati in crescita non lineare ma esponenziale e quindi aumentare le caratteristiche dinamiche del filtro.

Non ci resta, per concludere, che fare cenno a due ultimi oggetti che ancora non conosciamo: **throw~** e **catch~**. Si tratta di oggetti che operano in coppia, contraddistinti da un unico nome argomento. Alla base vi è l'idea di creare una connessione a distanza di un certo flusso di dati audio. Vi possono essere tanti oggetti *throw~* con un particolare nome, come nel nostro caso, ed un unico oggetto *catch~* che abbia il medesimo nome. I loro stessi nomi sono indicativi: *throw* ci rimanda l'idea di un 'gettare' uno o più segnali audio a distanza; *catch* qualcosa che lo (li) raccoglie o 'afferra'. Tramite il messaggio *set <nome flusso>* si può anche ridirezionare il flusso verso un altro oggetto *catch~*.

13.5 Abbiamo studiato

fft~, rfft~, ifft~, irfft~, fiddle~, throw~, catch~, textfile

Indice analitico

abs, 27
adc , 77
astrazione, 20
atomo, 18

b, 37
bang, 22
bendin, 63
bendout, 63
biquad , 109
blocco, 128
block , 125
borax, 67
bounds, 53
box messaggio, 21
box numerico, 25
box symbol, 22
bp , 104
butterfly, 129

catch , 144
change, 43
chord, 68
clear, 104
clip, 30
codifica MIDI, 58
Comb filters, 101
cos , 92
cputime, 36
crossover, 102
ctlin, 62
ctlout, 62

dac , 77
dbtorms, 29
delay, 33
delwrite , 114
deread , 114
DFT, 127
dispersione, 101
div, 27

dsp, 77

env , 104
exp, 27

f, 34, 39
f2note, 106
Fast Fourier Transform, 129
feedback, 110
feedforward, 110
FFT-size, 128
fft , 133
fiddle , 120
Filtri Shelving, 102
FIR, 96
float, 30
frequenza di taglio, 99
ftom, 29

hip , 103
Hradio, 50

iemlib, 75
IIR, 96, 109
int, 30

Karplus-Strong, 123
key, 46
keyname, 46
keyup, 46

line, 37
line , 87
loadbang, 64
log, 27
lop , 104

makenote, 56
max, 28
maxlib, 66
maxsize, 83
metro, 35

- mfread, 71
- midin, 64
- midout, 64
- min, 28
- mod, 27
- moses, 42
- mtof, 29

- noise , 106
- normalizza, 90
- notein, 59
- noteout, 57

- open, 80
- openpanel, 71
- osc , 79
- overlap, 125
- overlapping, 131

- pack, 39
- patch, 14
- pgmin, 61
- pgmout, 61
- phasor , 91
- piano complesso Z , 97
- pipe, 34
- pitch, 68
- poli, 98
- pow, 27

- Q, 104

- random, 28, 49
- read, 83
- readsf , 81
- realtime, 36
- receive, 17
- resize, 83
- rfft , 136
- ripple, 100
- rmstodb, 29
- route, 44
- rpole , 109

- s, 34
- samphold , 94
- samplerate , 136
- select, 40
- send, 17
- Short-time Fourier Transform, 128
- sinesum, 89
- skip, 83
- soundfiler , 82
- spigot, 42
- start, 80
- stop, 80
- switch , 125
- symbol, 21
- sysexin, 64

- t, 37
- table, 48
- tabosc4 , 89
- tabplay , 86
- tabread, 50
- tabread4, 52
- tabread4 , 90
- tabreceive , 136
- tabwrite, 49
- tabwrite , 84
- textfile, 142
- throw , 144
- timer, 36
- toggle, 40
- trasformata di Fourier, 127
- trasformata z , 99
- trigger, 37

- unpack, 38

- vcf , 107
- vd , 115
- Vline , 87
- Vradio, 50

- writesf , 79

- xeq, 69
- xlabel, 54
- xticks, 53

- ylabel, 54
- yticks, 53

- zeri, 98